Honeypot IDS – Exploit test-scenario report

Generated: 2026-10-02 19:47:32
Target: ids (local) (https://localhost)
Honeypot layer: WordPress proxy level
Score decay half-life: 300s

Each exploit below is run from a freshly reset threat state (Redis flushed + reverse_proxy shared dicts flushed) so its score is that CVE's own isolated value. The score/route are read from reverse_proxy's X-Threat-Score / X-Route-Target headers; the screenshot is the embedded browser at that URL. Every request is scored through the pipeline in **Figure 2** and routed as in **Figure 1** – so a score below reflects Figure 2, and its production/honeypot route reflects Figure 1.

Contents

1. Individual exploits
   a. WooCommerce Payments admin bypass (CVE-2023-28121)
   b. Abandoned Cart Lite checkout token forgery (CVE-2023-2986)
   c. Drag & Drop file upload MIME-type bypass (CVE-2025-4403)
   d. CWMP unauthenticated options update (CVE-2025-2266)
   e. Gift Voucher file upload RCE (CVE-2025-47577)
   f. Advanced Form Integration SQL injection (CVE-2024-2387)
   g. PagSeguro Connect path traversal (CVE-2025-10142)
   h. WP File Upload directory traversal (CVE-2024-50508)
   i. wp-config.php disclosure (GENERIC-WP-001)
   j. Installer file access (GENERIC-WP-002)
   k. User enumeration via ?author= (GENERIC-WP-003)
   l. XML-RPC probe (GENERIC-WP-004)
   m. Bricks Builder unauthenticated RCE (CVE-2024-25600)
   n. WP Automatic SQL injection (CVE-2024-27956)
   o. Ultimate Member SQL injection (CVE-2024-1071)
   p. BookingPress SQL injection (CVE-2022-0739)
   q. LayerSlider SQL injection (CVE-2024-2879)
   r. REST API content injection (defacement) (CVE-2017-1001000)
   s. Script tag in a search form (GENERIC-XSS-001)
   t. Event-handler XSS (GENERIC-XSS-002)
   u. SQL injection in a search form (GENERIC-SQLI-001)
   v. Time-based blind SQL injection (GENERIC-SQLI-002)
   w. SQL injection in the login form (POST body) (GENERIC-SQLI-003)
   x. Wordlist scan: sensitive paths (GENERIC-ENUM-001)
   y. Wordlist scan: plugin and theme discovery (GENERIC-ENUM-002)
   z. Insecure direct object reference: user IDs (GENERIC-IDOR-001)
   aa. Insecure direct object reference: order IDs (GENERIC-IDOR-002)
   ab. Scanner tool: sqlmap (GENERIC-TOOL-001)
   ac. Scanner tool: Nikto (GENERIC-TOOL-002)
   ad. Scanner tool: Metasploit (GENERIC-TOOL-003)
   ae. Scanner tool: Gobuster (GENERIC-TOOL-004)
   af. Scanner tool: WPScan (GENERIC-TOOL-005)
2. Threat increase – repeated exploits
3. Threat decay

4. Test categorization

Reading the probability metrics. A **95% confidence interval (CI)** is the range the true value would fall in 95 times out of 100 if the measurement were repeated: a wide interval means few or noisy samples, a narrow one means the figure is well pinned down. A **p-value** is the probability of seeing a difference at least this large if there were really no effect; below 0.05 it is conventionally called *significant*, i.e. unlikely to be chance. Rates use the Wilson interval, which stays within 0–100% even for small samples or 0% / 100% results. Where a mean over a time series (CPU, memory) has a CI, it uses an effective sample size that discounts correlated neighbouring samples, so it is not over-confident. **P(≥x)** is the share of a recording spent at or above x: the chance that a randomly picked moment was that busy. **SD** (standard deviation) is the typical distance of a value from the mean.

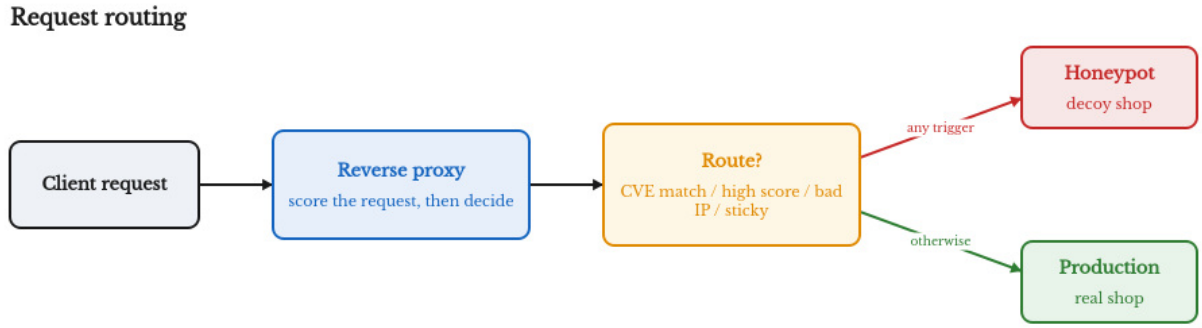


Figure 1. Request routing: the reverse proxy scores each request and diverts it to the honeypot on any trigger (Figure 2), otherwise to production.

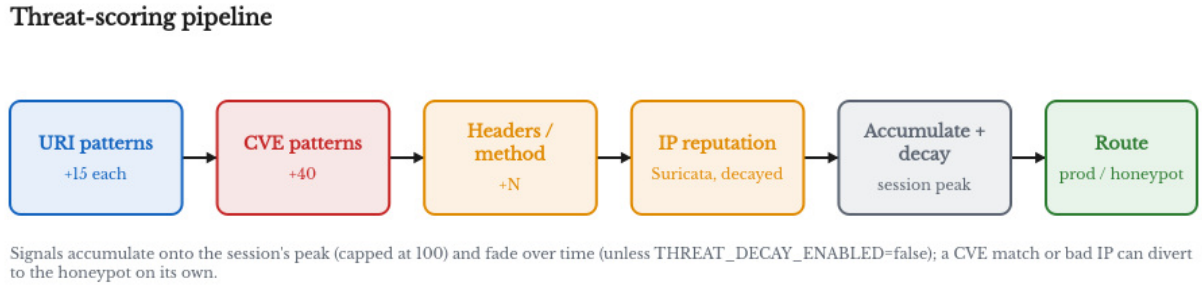


Figure 2. Threat-scoring pipeline: the signals whose sum is the score shown for each exploit below; it accumulates onto the session peak and decays over time.

Honeypot bypasses: none. This report ran on the WordPress proxy level, where a diverted request is served by a separate honeypot eshop (its own WordPress files, PHP runtime and database), so every exploit below is contained once it is routed to the honeypot.

1. Individual exploits

WooCommerce Payments admin bypass
CVE-2023-28121 · severity HIGH
`GET /wp-json/wp/v2/users`
CVE exploit · Plugin / theme · woocommerce-payments · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 200
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+25), Suspicious headers (+50), CVE PATTERNS

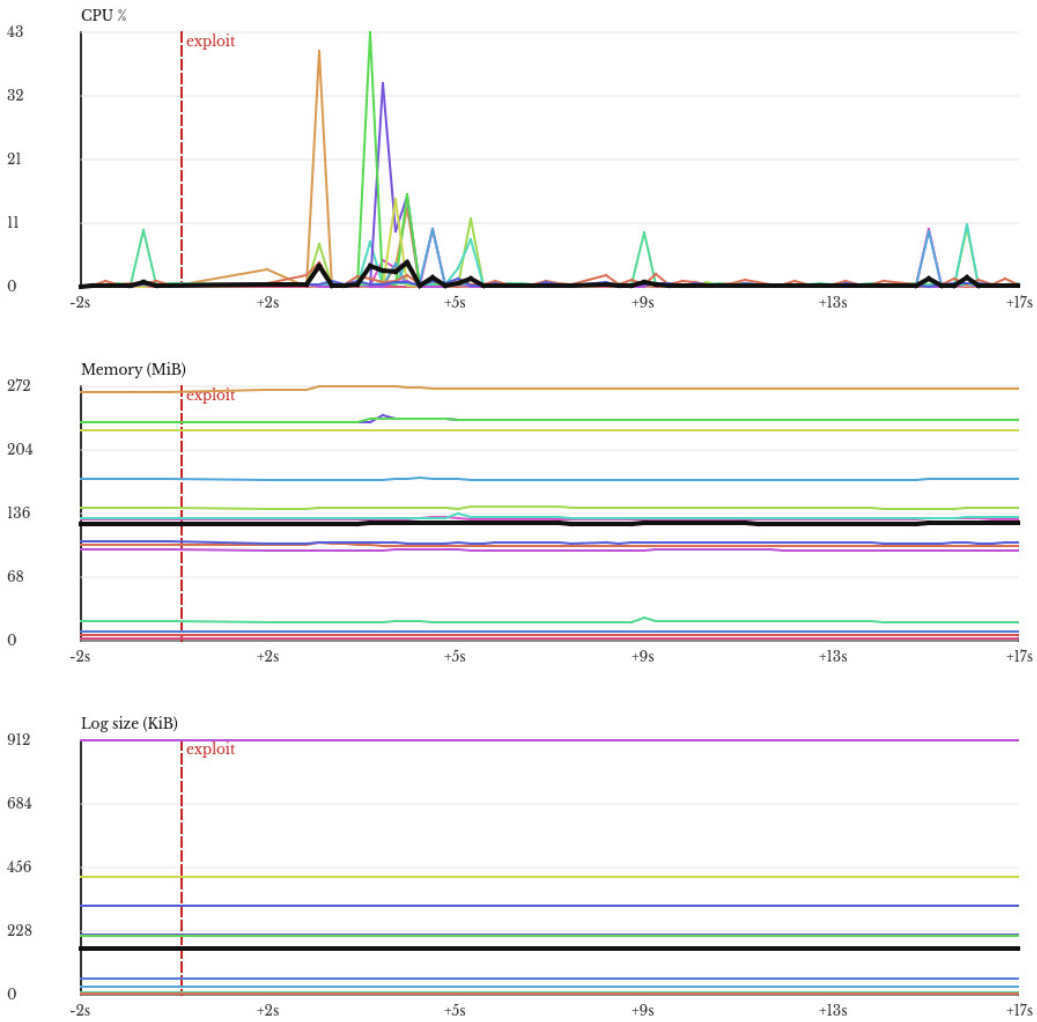
DETECTED (+40), Automation detected (+5)

Pretty print

{"code":"rest_forbidden","message":"Sorry, you are not allowed to do that.","data":{"status":401}}

Resource usage – ~2 s before to ~15 s after the exploit

Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

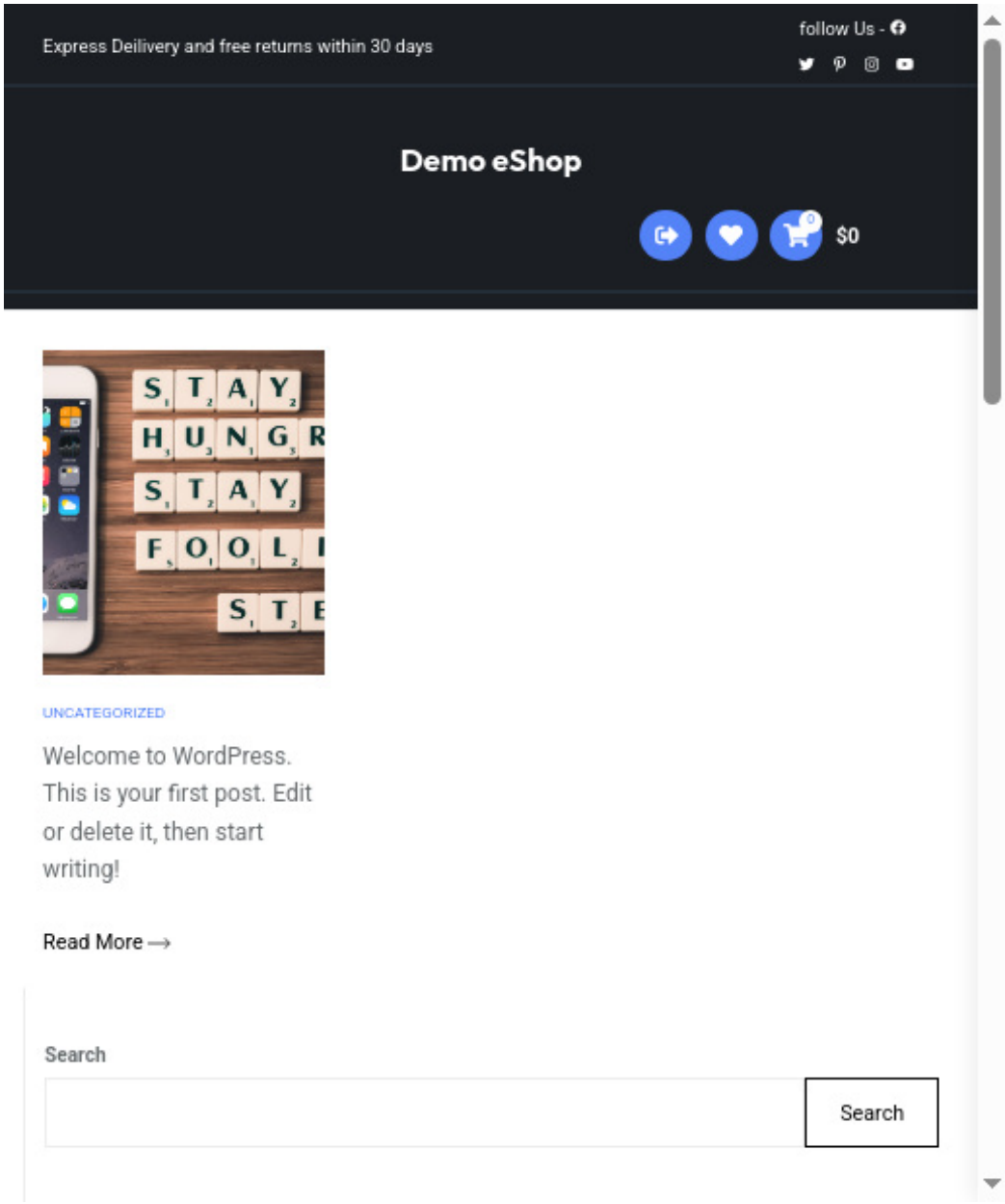


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.2% (95% CI 0.0–0.3%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.1–0.9%)**	55 samples after the exploit
CPU change after the exploit	**+0.4%, p = 0.084**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**4.0%, 17.3 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**124.6 MiB (95% CI 124.5–124.6 MiB)**	8 samples before the exploit
Memory, container average, after	**124.9 MiB (95% CI 124.7–125.1 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.4 MiB, p = 0.002**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**125.5 MiB, 71.4 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

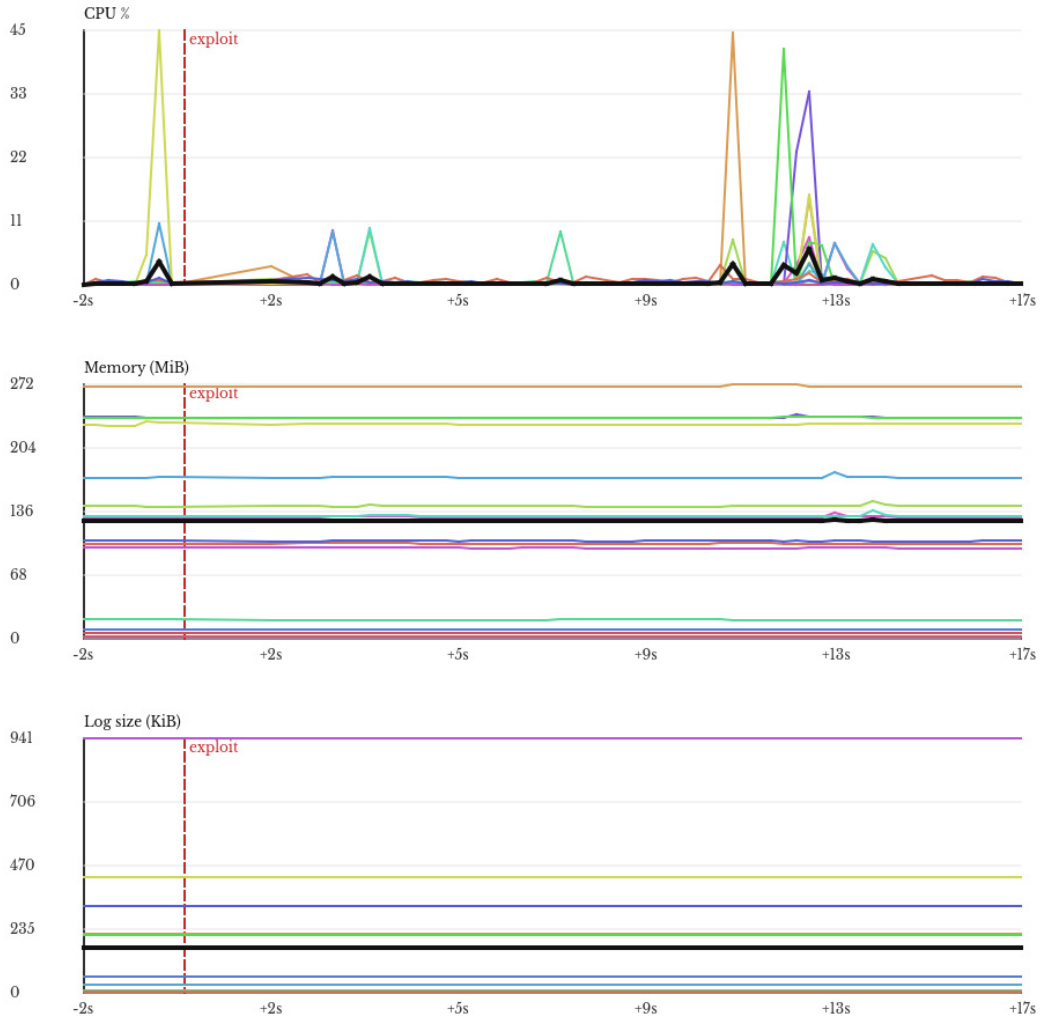
Abandoned Cart Lite checkout token forgery
CVE-2023-2986 · severity HIGH
`GET /?wcal_action=checkout_link&validate=x`
CVE exploit · Plugin / theme · woocommerce-abandoned-cart · Backend

Score: 60/100 Route: HONEYPOT reason: cve_pattern_match HTTP/2 200
Diverted to honeypot by rule "cve_pattern_match" – a CVE/exploit match (or IP reputation) diverts regardless of the numeric score.
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+15), CVE PATTERNS DETECTED (+40), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.

CPU and memory were read from the containers' cgroup files every 0.25 s.

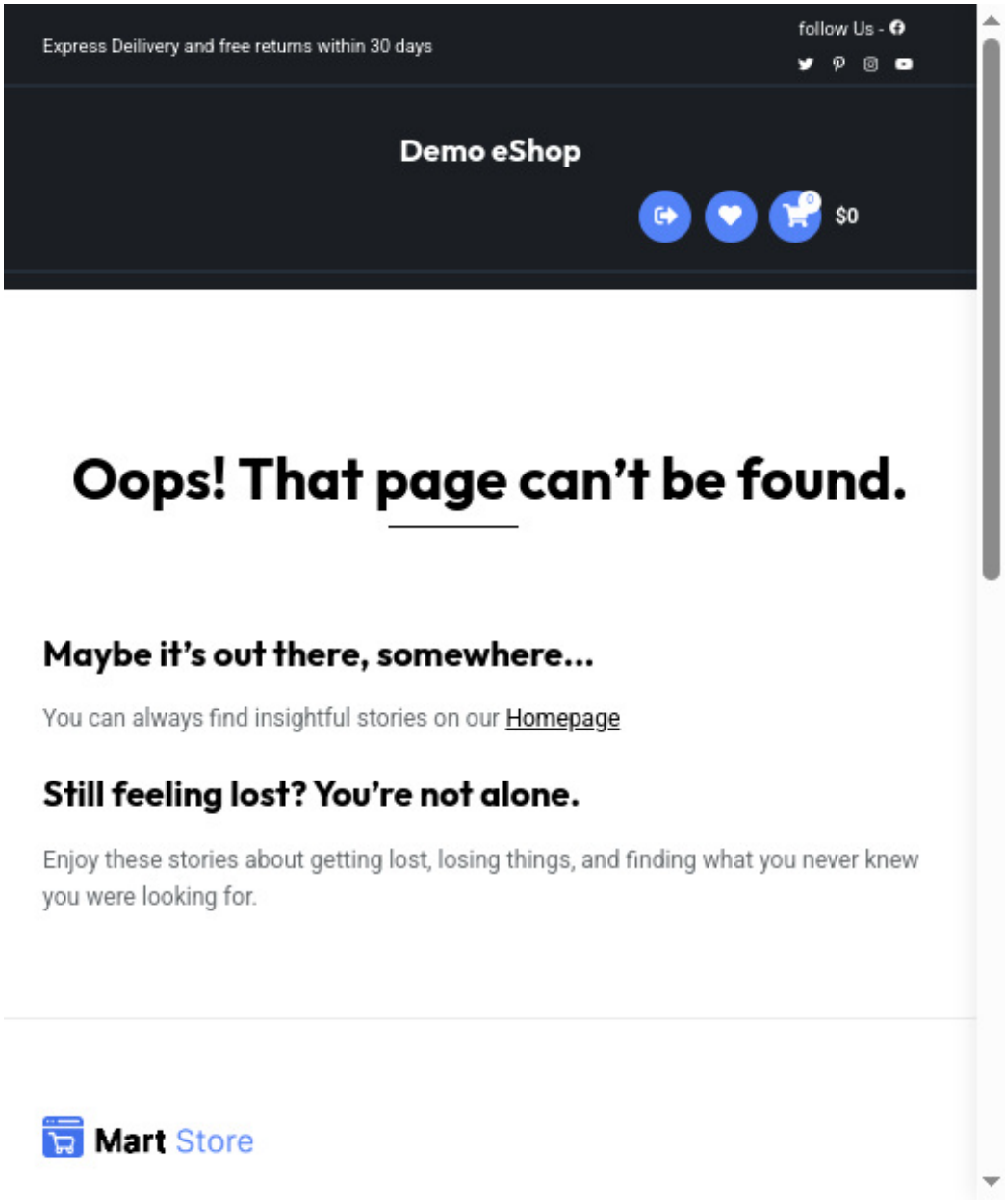


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

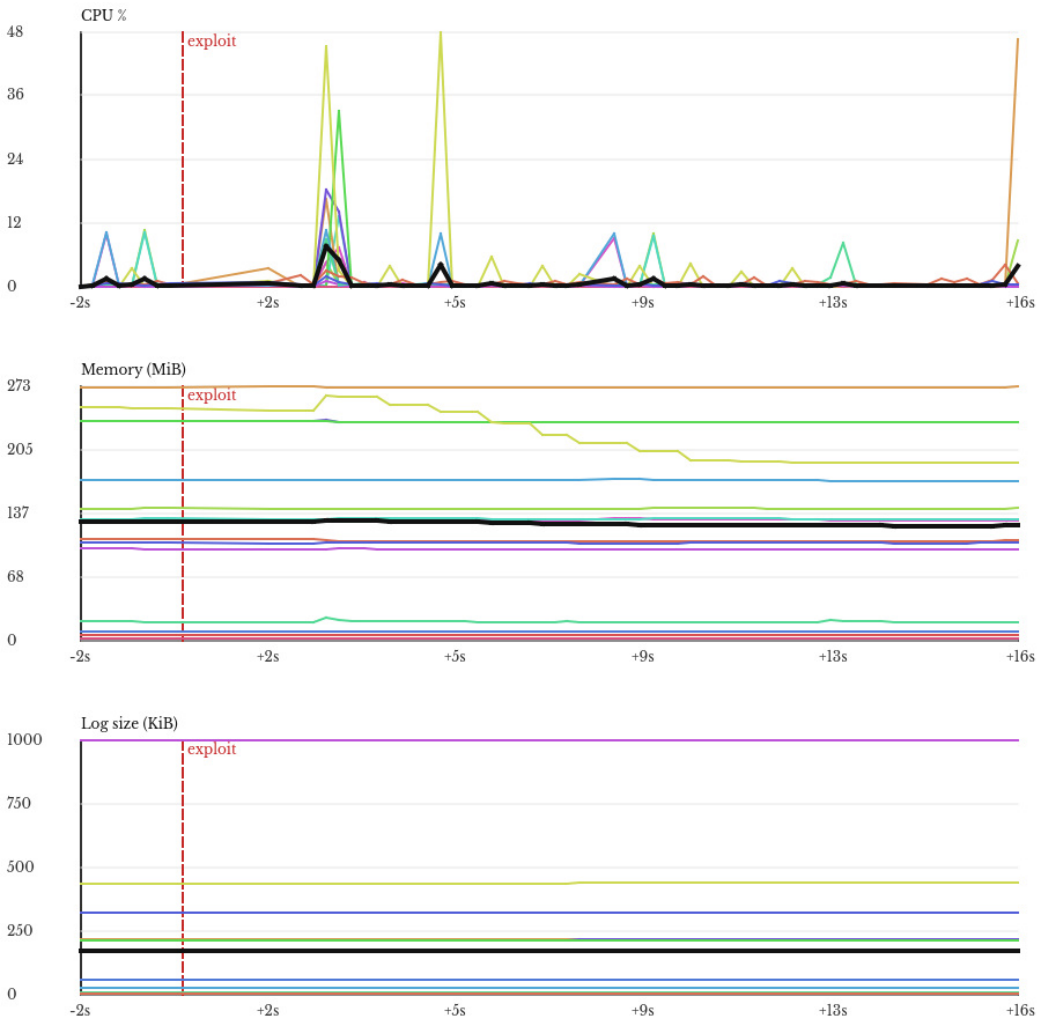
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.6% (95% CI 0.0–1.7%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.1–0.9%)**	55 samples after the exploit
CPU change after the exploit	**-0.1%, p = 0.859**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**6.3%, 4.3 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**125.3 MiB (95% CI 125.3–125.4 MiB)**	8 samples before the exploit
Memory, container average, after	**125.3 MiB (95% CI 125.1–125.4 MiB)**	55 samples after the exploit
Memory change after the exploit	**-0.1 MiB, p = 0.398**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**126.2 MiB, 11.8 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Drag & Drop file upload MIME-type bypass
CVE-2025-4403 · severity CRITICAL
`GET /?action=dnd-wc-upload-file&name=shell.php`
CVE exploit · Plugin / theme · drag-and-drop-multiple-file-upload · Backend

Score: 45/100 Route: HONEYPOT reason: cve_pattern_match HTTP/2 404
Diverted to honeypot by rule "cve_pattern_match" – a CVE/exploit match (or IP reputation) diverts regardless of the numeric score.
Analyzer score verdict: elevated but below threshold
Signals: CVE PATTERNS DETECTED (+40), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

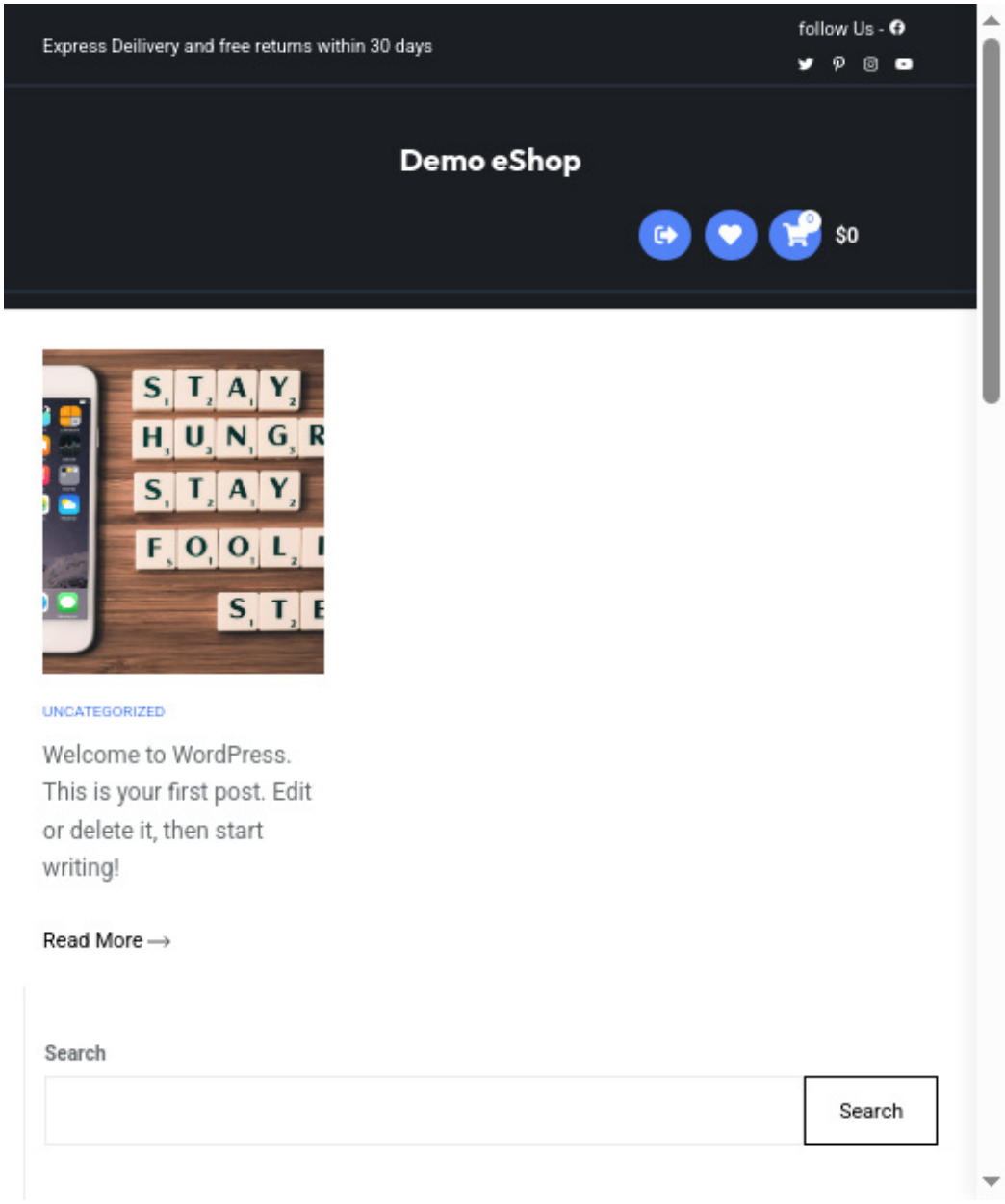


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

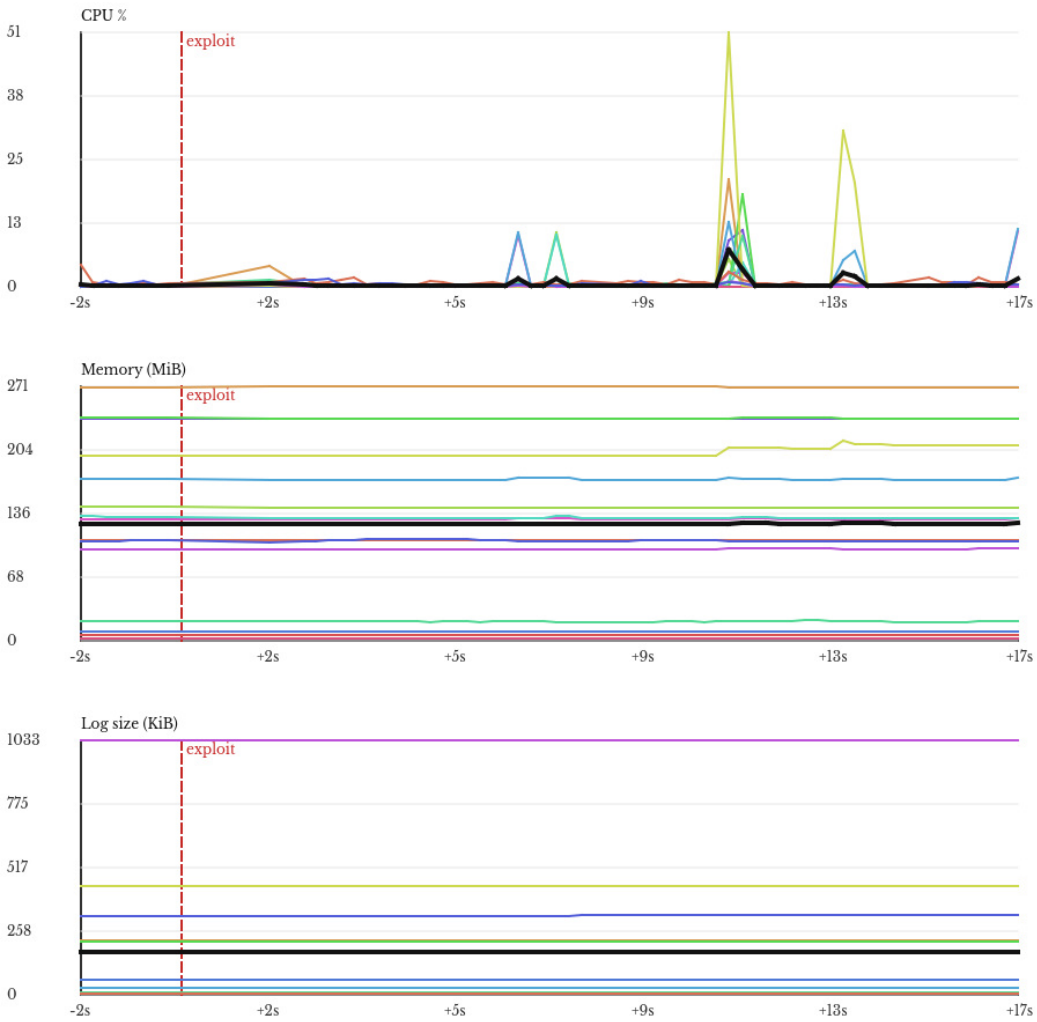
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.5% (95% CI 0.0–1.0%)**	8 samples before the exploit
CPU, container average, after	**0.6% (95% CI 0.1–1.1%)**	55 samples after the exploit
CPU change after the exploit	**+0.1%, p = 0.707**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**7.6%, 11.4 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**127.6 MiB (95% CI 127.5–127.7 MiB)**	8 samples before the exploit
Memory, container average, after	**124.8 MiB (95% CI 107.2–142.4 MiB)**	55 samples after the exploit
Memory change after the exploit	**-2.8 MiB, p = 0.292**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**128.7 MiB, 21.7 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

CWMP unauthenticated options update
CVE-2025-2266 · severity CRITICAL
`GET /?action=cwmpUpdateOptions&option=default_role&value=administrator`
CVE exploit · Plugin / theme · cwmp · Backend

Score: 45/100 Route: HONEYPOT reason: cve_pattern_match HTTP/2 200
Diverted to honeypot by rule "cve_pattern_match" – a CVE/exploit match (or IP reputation) diverts regardless of the numeric score.
Analyzer score verdict: elevated but below threshold
Signals: CVE PATTERNS DETECTED (+40), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

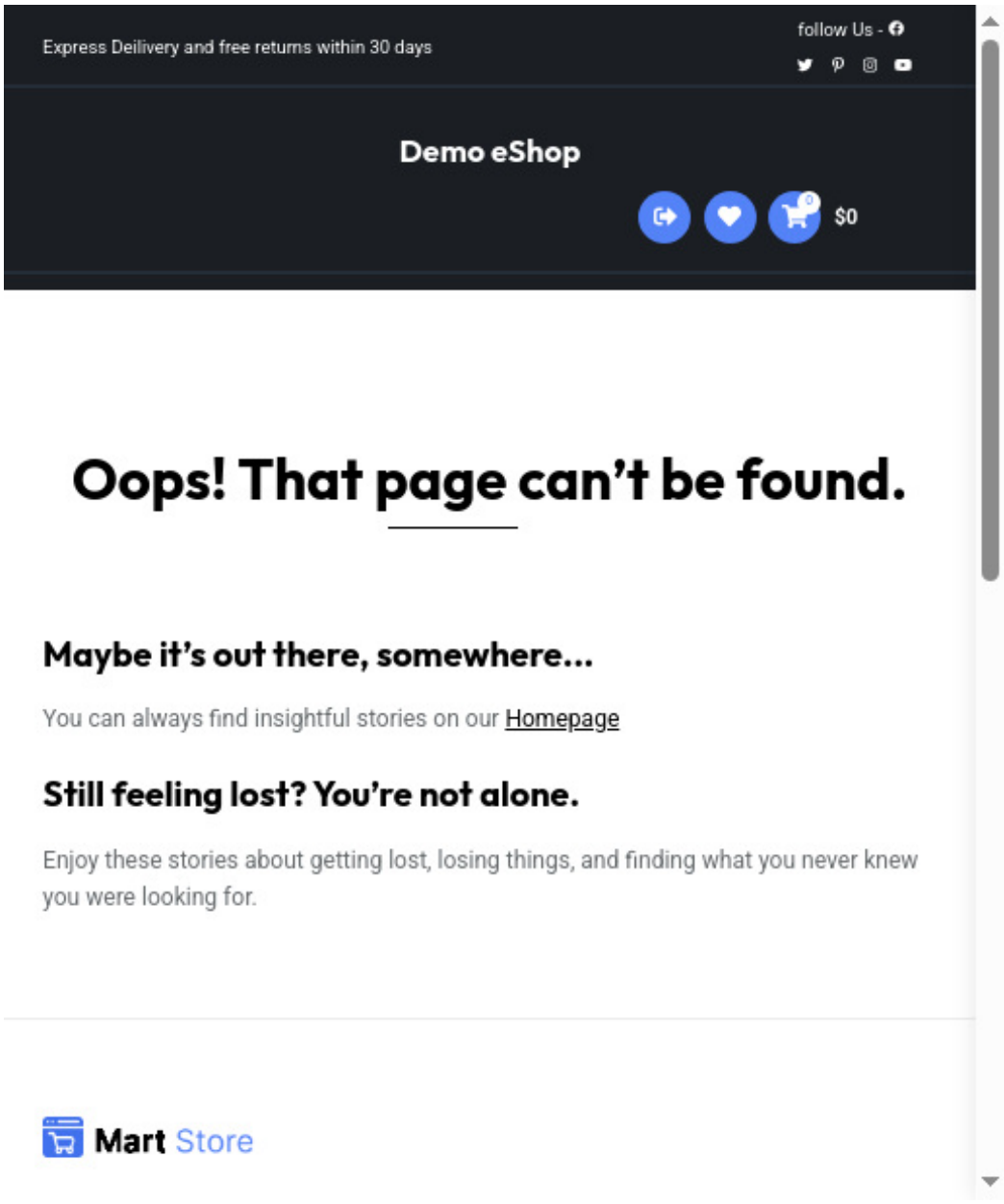


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

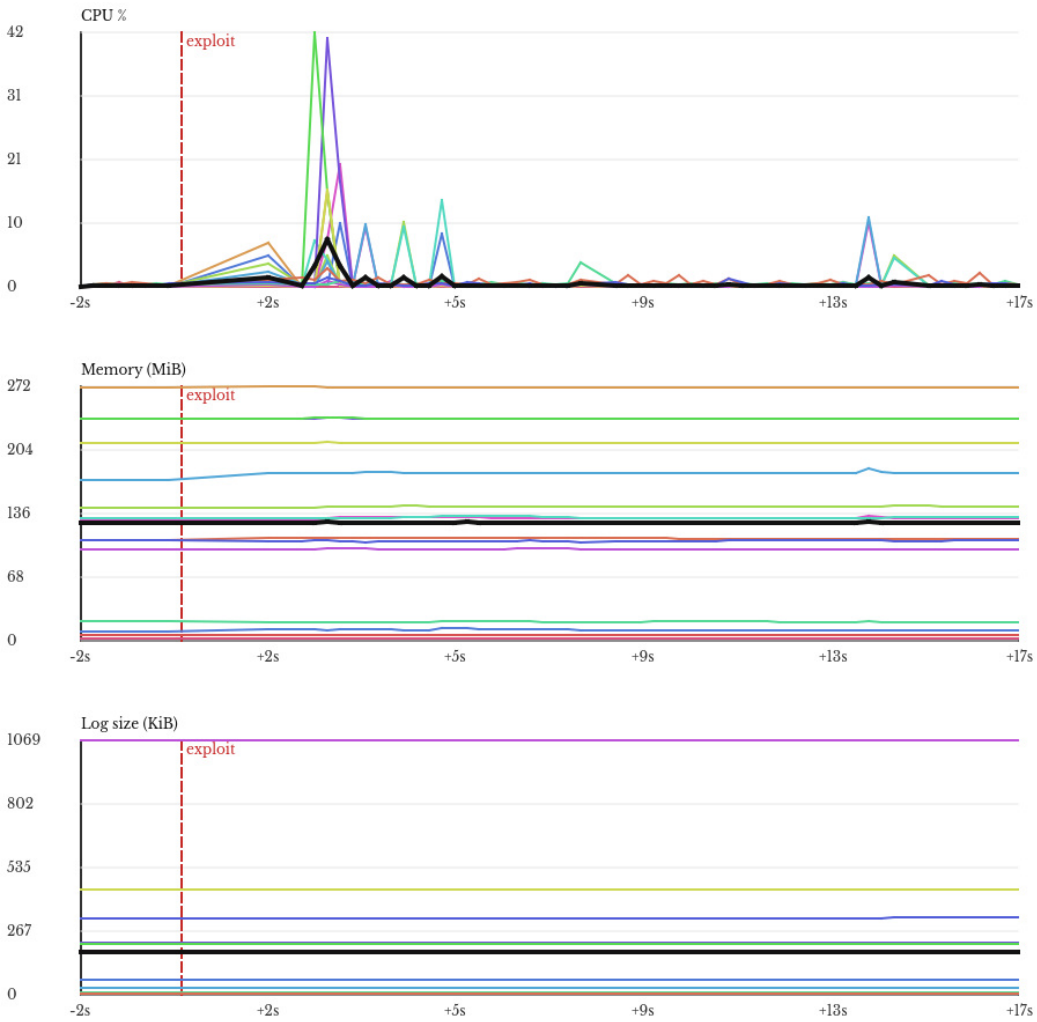
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.1–0.2%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.1–0.9%)**	55 samples after the exploit
CPU change after the exploit	**+0.3%, p = 0.121**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**7.2%, 100.8 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**124.1 MiB (95% CI 123.9–124.2 MiB)**	8 samples before the exploit
Memory, container average, after	**124.2 MiB (95% CI 123.5–124.8 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.1 MiB, p = 0.586**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**125.0 MiB, 54.9 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Gift Voucher file upload RCE
CVE-2025-47577 · severity CRITICAL
`GET /?action=mwb_wgm_preview_mail&attachment=shell.php`
CVE exploit · Plugin / theme · gift-voucher · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 404
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+15), CVE PATTERNS DETECTED (+80), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

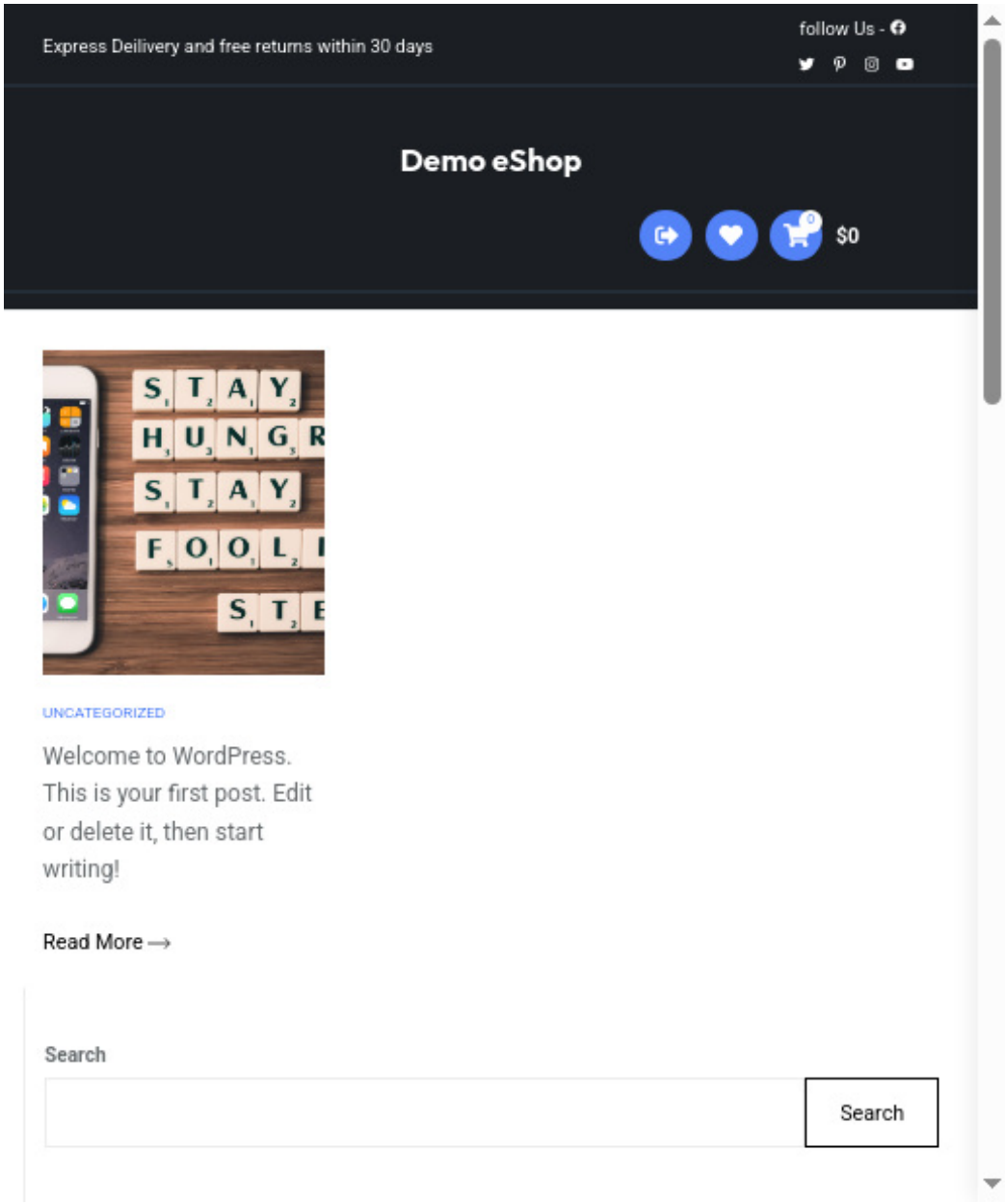


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.0–0.1%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.0–1.1%)**	55 samples after the exploit
CPU change after the exploit	**+0.4%, p = 0.158**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**7.7%, 185.4 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**124.8 MiB (95% CI 124.7–124.8 MiB)**	8 samples before the exploit
Memory, container average, after	**125.7 MiB (95% CI 125.5–125.9 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.9 MiB, p < 0.001**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**126.0 MiB, 110.7 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

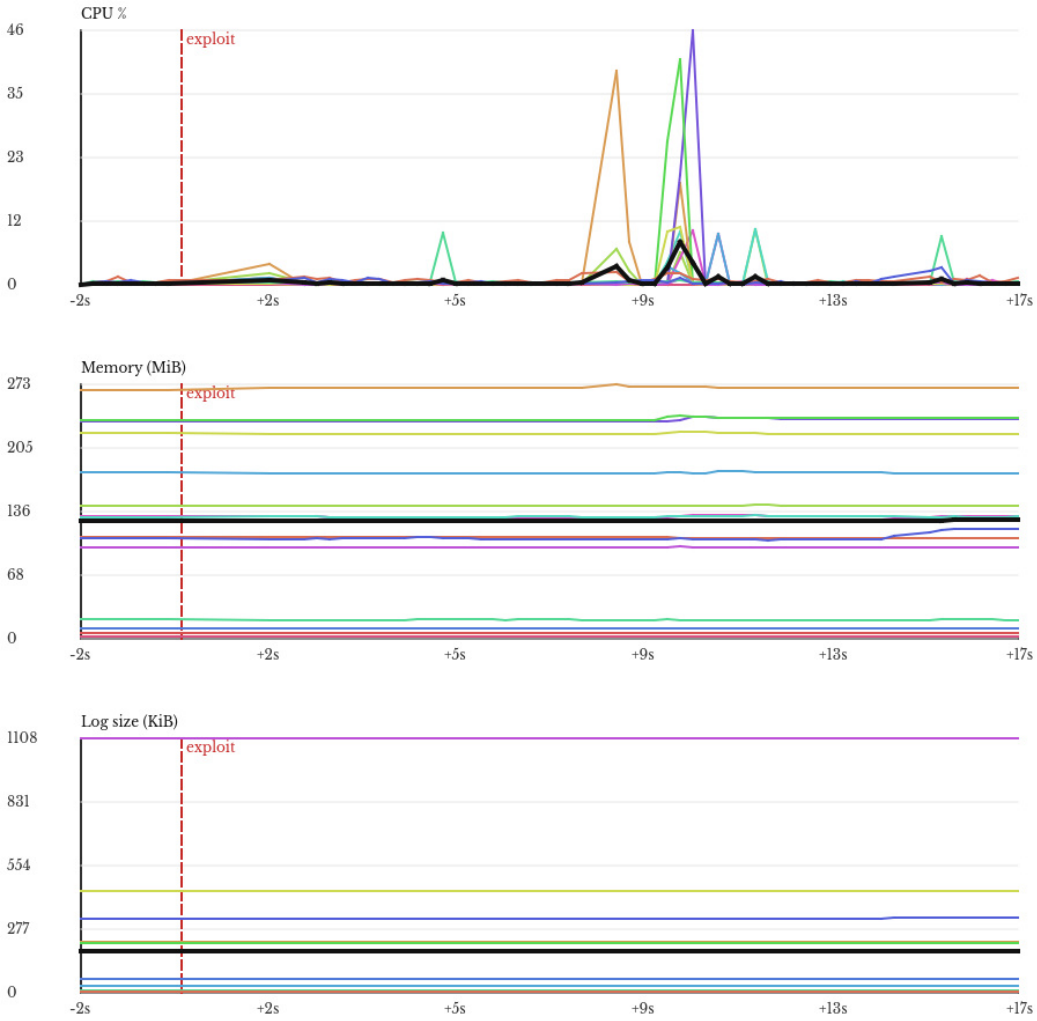
Advanced Form Integration SQL injection
CVE-2024-2387 · severity HIGH
`GET /?page=advanced-form-integration-log&integration_id=1'%20UNION%20SELECT%20user_login%2Cuser_pass%20FROM%20wp_users--%20-`
CVE exploit · Plugin / theme · advanced-form-integration · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 200
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+100), CVE PATTERNS DETECTED (+40), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.

CPU and memory were read from the containers' cgroup files every 0.25 s.

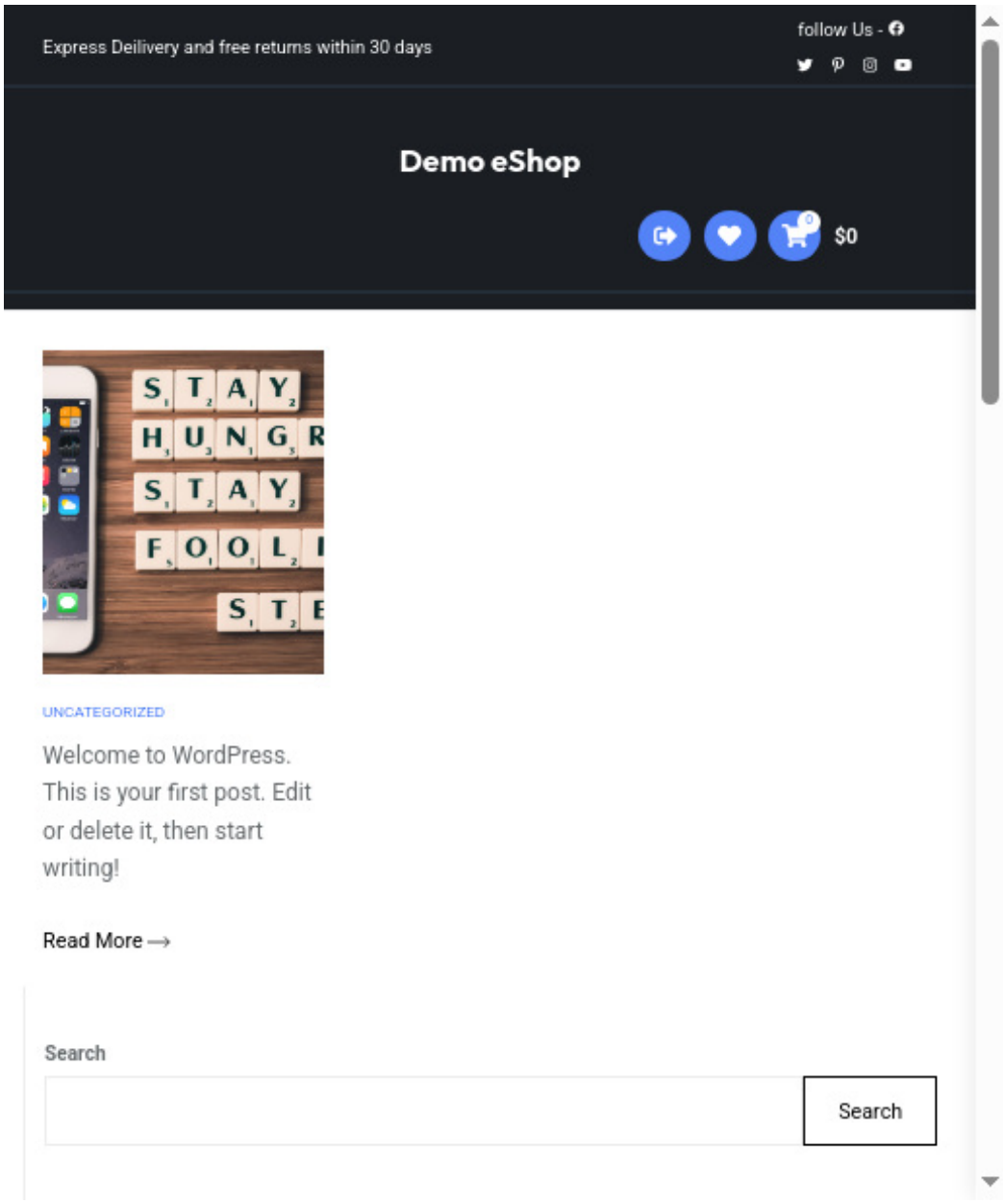


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

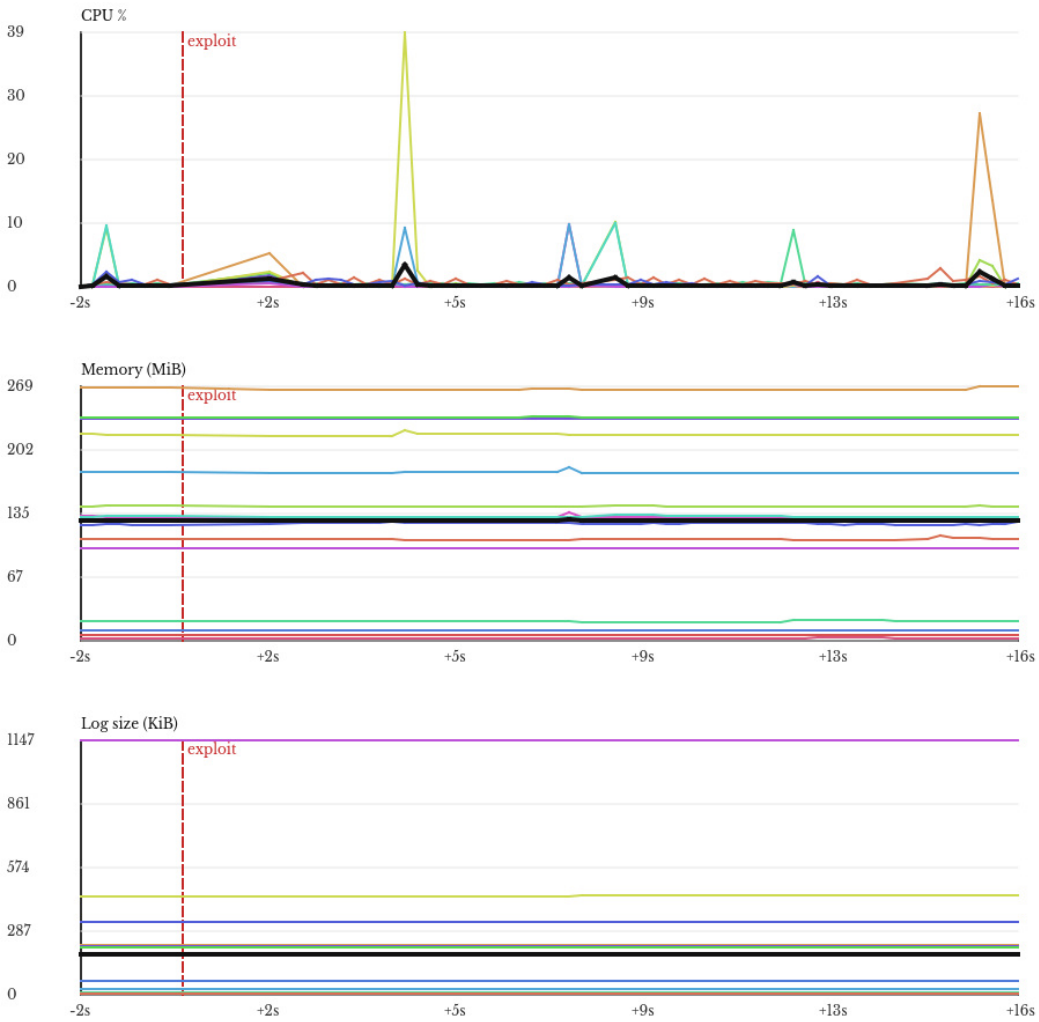
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.1–0.2%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.0–1.2%)**	55 samples after the exploit
CPU change after the exploit	**+0.4%, p = 0.166**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**7.8%, 147.0 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**125.6 MiB (95% CI 125.6–125.6 MiB)**	8 samples before the exploit
Memory, container average, after	**125.8 MiB (95% CI 123.8–127.7 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.2 MiB, p = 0.622**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**126.5 MiB, 67.2 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

PagSeguro Connect path traversal
CVE-2025-10142 · severity HIGH
`GET /?action=pc_added_uploaded_image&file[path]=../../../../etc/passwd`
CVE exploit · Plugin / theme · pagseguro-connect-woocommerce · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 200
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+80), CVE PATTERNS DETECTED (+80), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

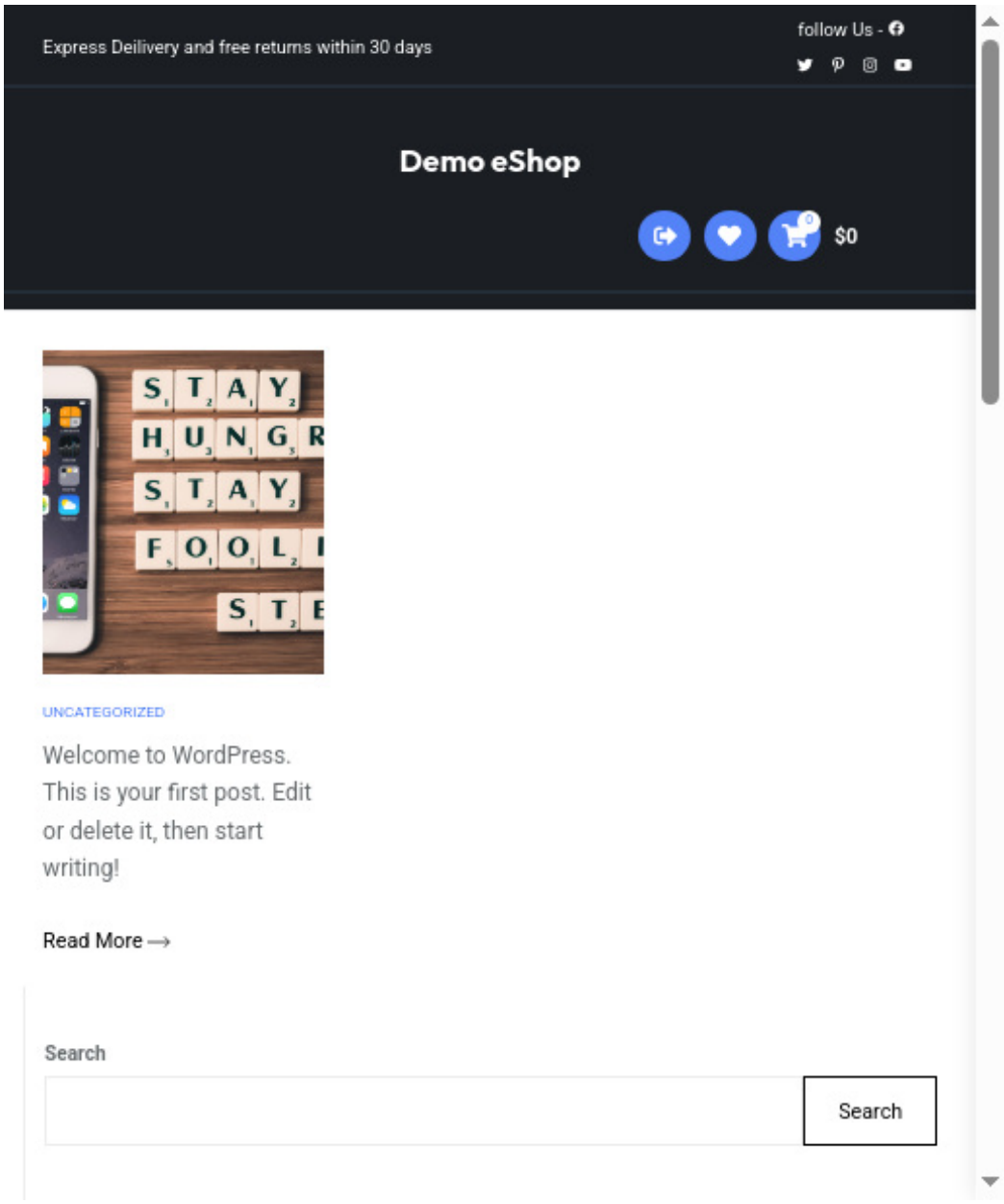


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

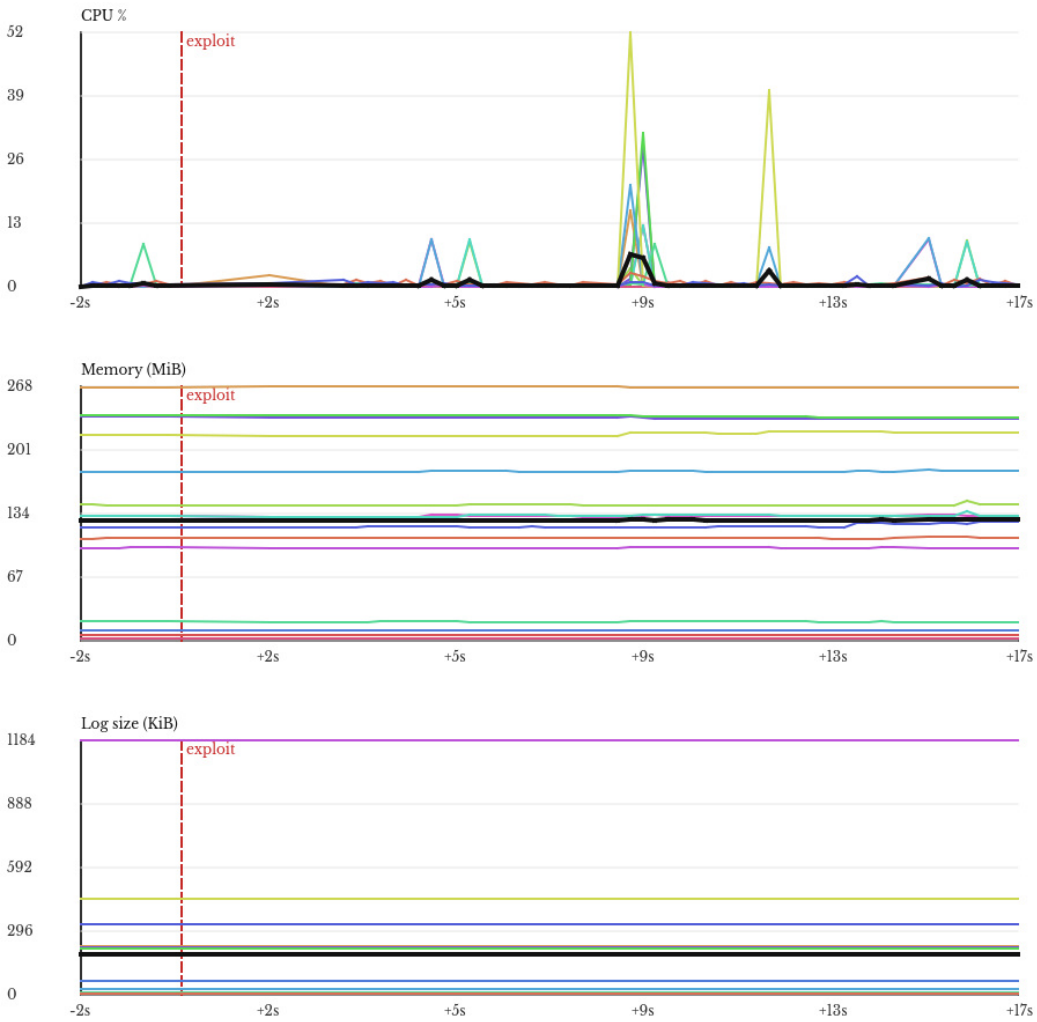
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.3% (95% CI 0.0–0.7%)**	8 samples before the exploit
CPU, container average, after	**0.3% (95% CI 0.1–0.5%)**	55 samples after the exploit
CPU change after the exploit	**+0.1%, p = 0.760**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**3.4%, 6.4 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**126.7 MiB (95% CI 126.6–126.8 MiB)**	8 samples before the exploit
Memory, container average, after	**126.5 MiB (95% CI 126.4–126.5 MiB)**	55 samples after the exploit
Memory change after the exploit	**-0.2 MiB, p < 0.001**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**127.2 MiB, 10.0 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

WP File Upload directory traversal
CVE-2024-50508 · severity HIGH
`GET /?action=wordpress_file_upload&file[path]=../../../../etc/passwd`
CVE exploit · Plugin / theme · wp-file-upload · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 200
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+65), CVE PATTERNS DETECTED (+40), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.



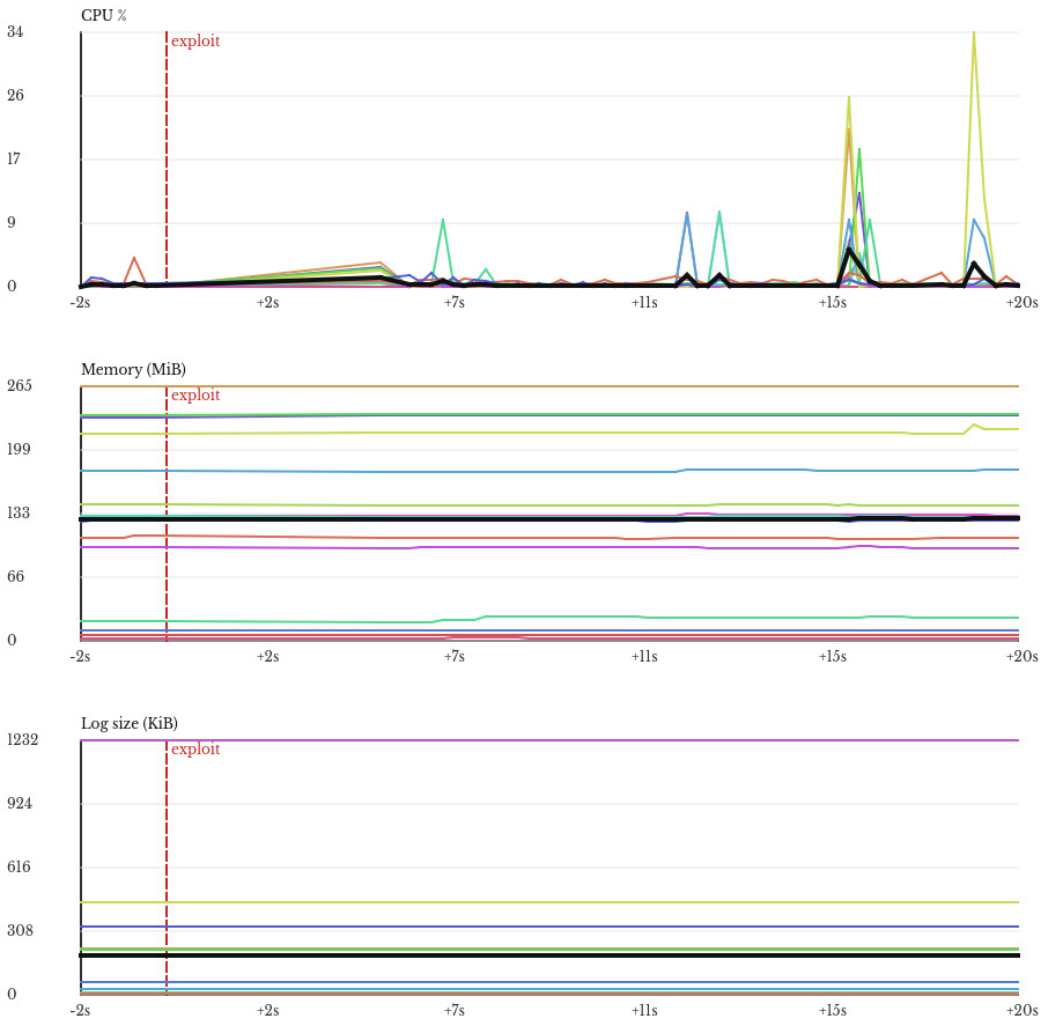
Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.2% (95% CI 0.0–0.3%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.0–1.1%)**	52 samples after the exploit
CPU change after the exploit	**+0.4%, p = 0.210**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**6.6%, 31.4 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**126.2 MiB (95% CI 126.2–126.2 MiB)**	8 samples before the exploit
Memory, container average, after	**126.4 MiB (95% CI 126.2–126.5 MiB)**	52 samples after the exploit
Memory change after the exploit	**+0.2 MiB, p = 0.039**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**127.2 MiB, 29.2 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

wp-config.php disclosure
GENERIC-WP-001 · severity CRITICAL
`GET /wp-config.php`
Generic attack · WordPress core · Backend

Score: 35/100 Route: PRODUCTION HTTP/2 200
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+30), Automation detected (+5)

Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

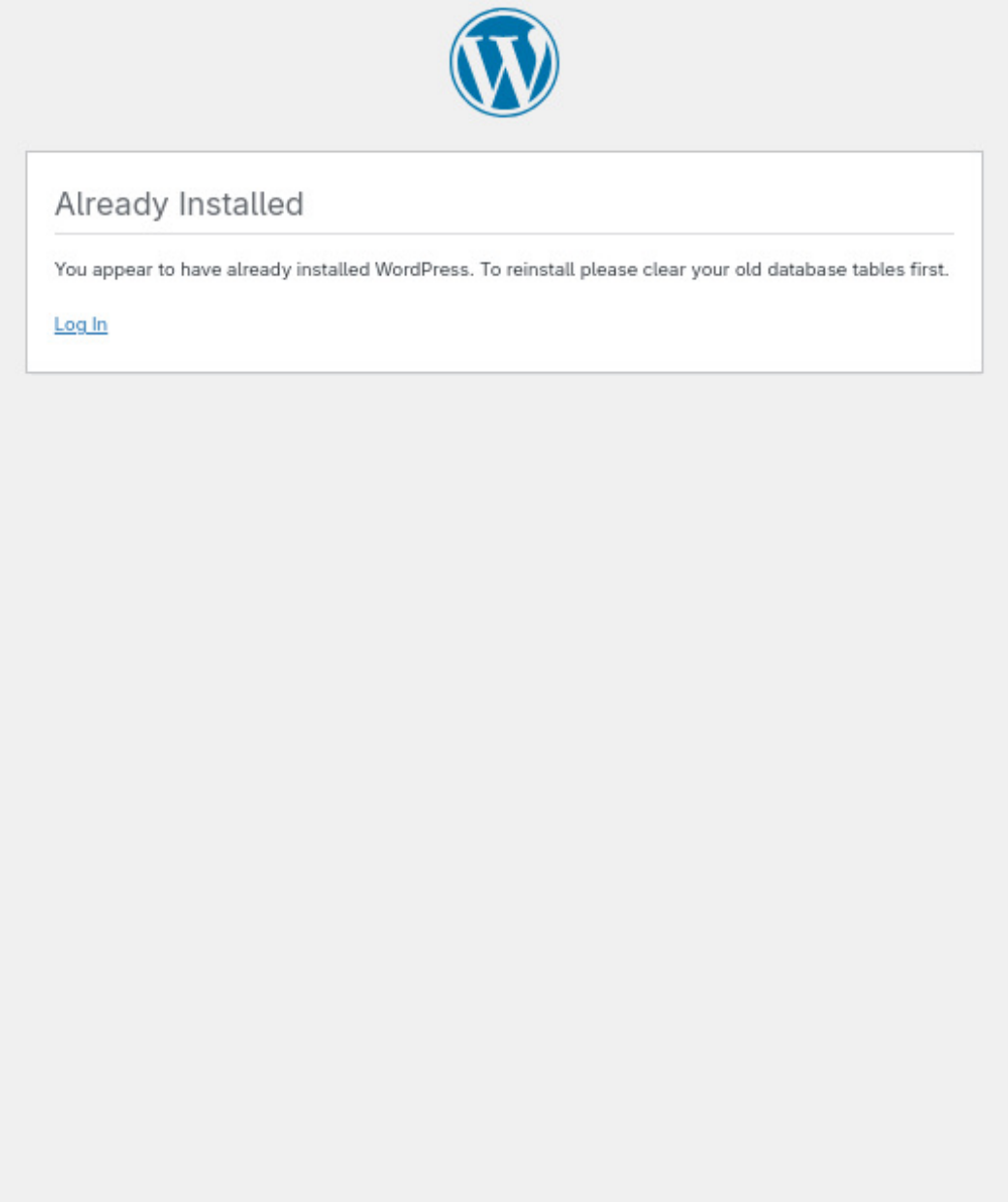


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

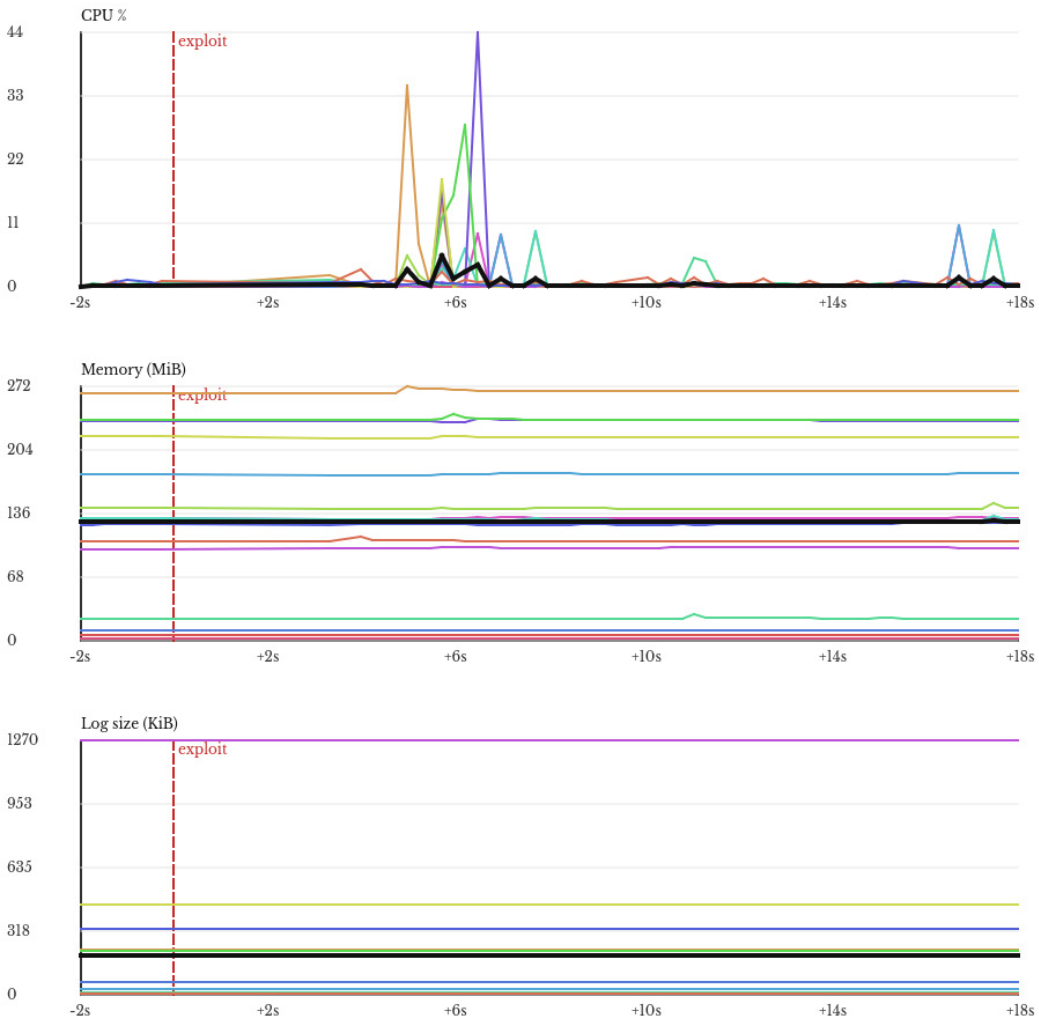
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.0–0.2%)**	8 samples before the exploit
CPU, container average, after	**0.4% (95% CI 0.1–0.7%)**	55 samples after the exploit
CPU change after the exploit	**+0.3%, p = 0.103**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**4.8%, 45.8 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**126.1 MiB (95% CI 125.7–126.6 MiB)**	8 samples before the exploit
Memory, container average, after	**126.4 MiB (95% CI 126.3–126.6 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.3 MiB, p = 0.009**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**126.9 MiB, 13.5 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Installer file access
GENERIC-WP-002 · severity HIGH
`GET /wp-admin/install.php`
Generic attack · WordPress core · Backend

Score: 75/100 Route: PRODUCTION HTTP/2 200
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+70), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

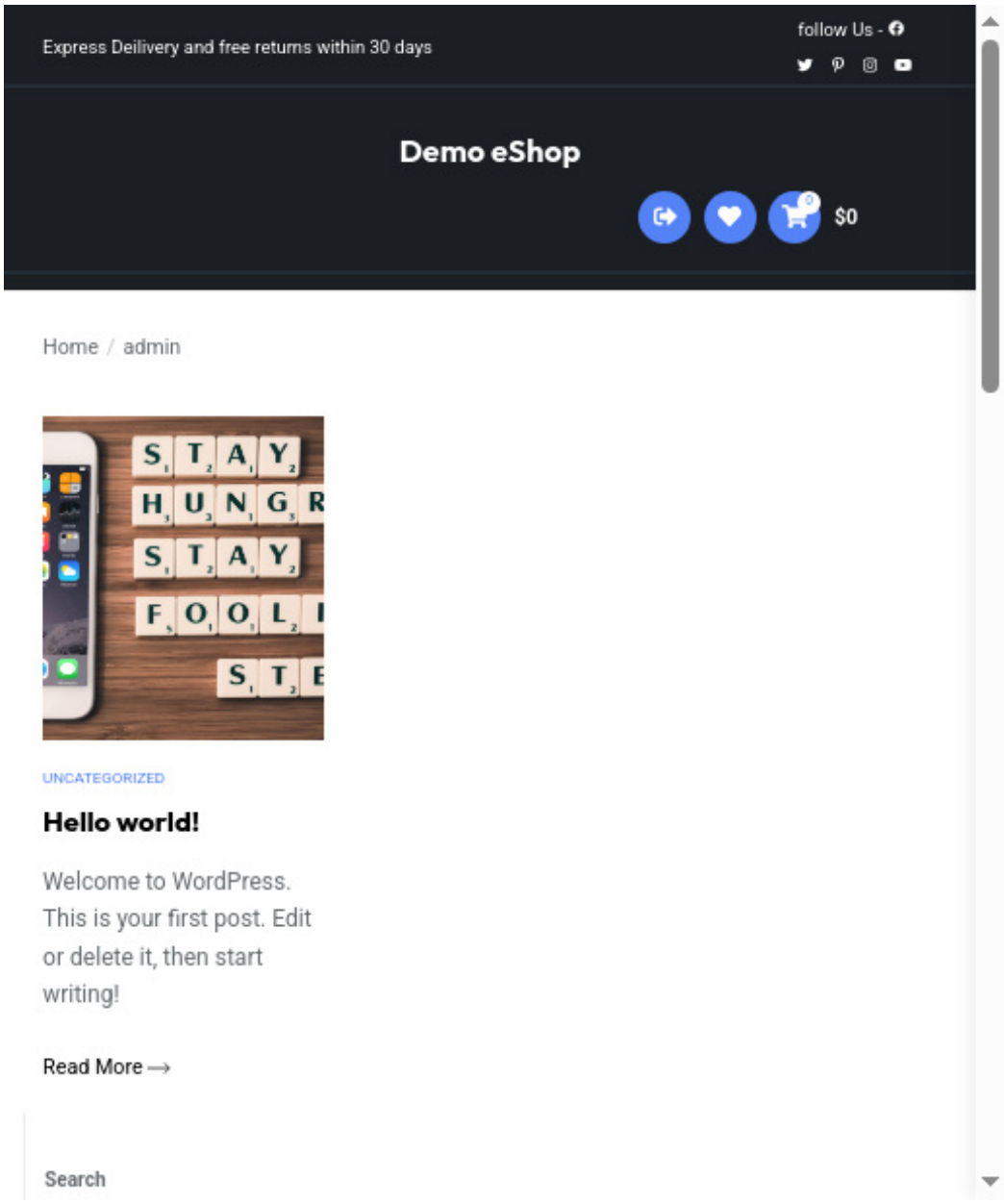


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

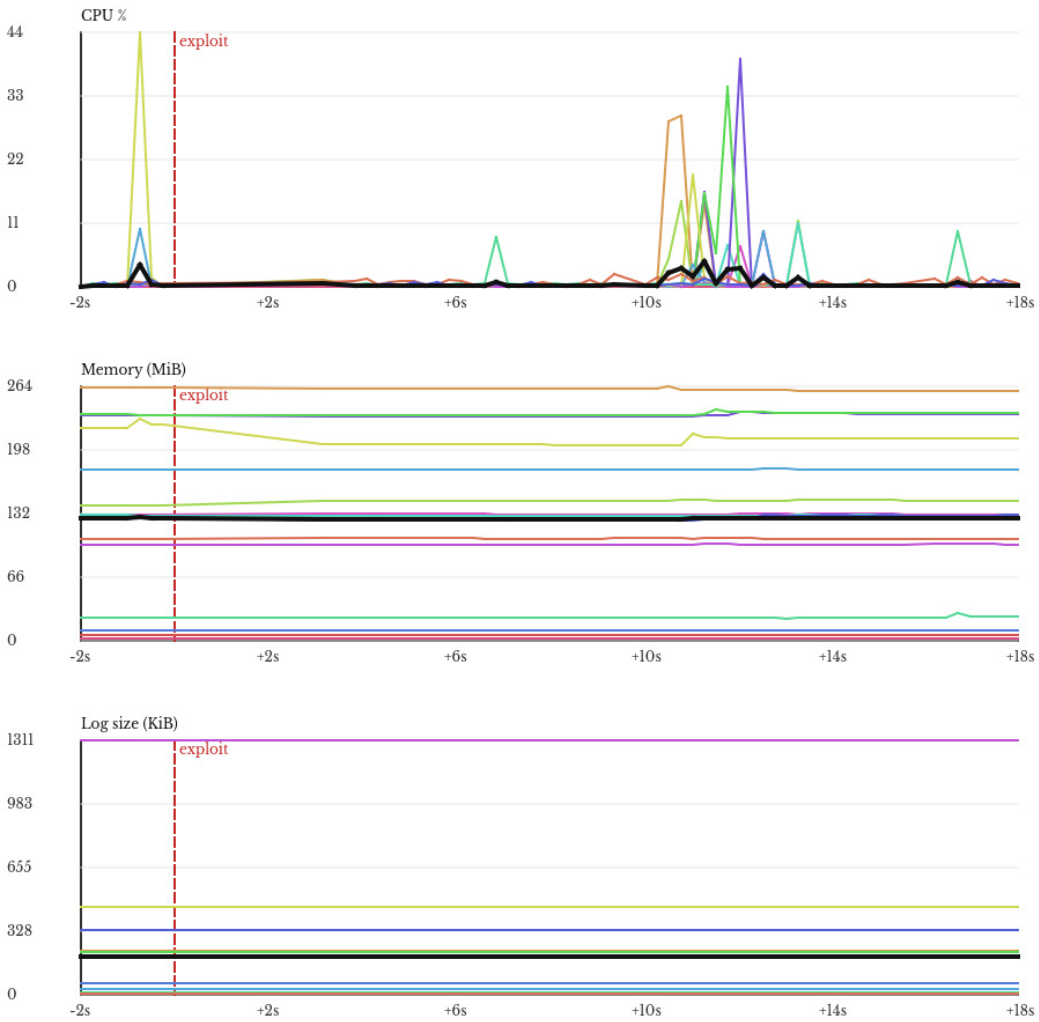
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.0–0.2%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.2–0.8%)**	55 samples after the exploit
CPU change after the exploit	**+0.4%, p = 0.017**	mean after vs mean before; unlikely to be chance (significant at 5%)
CPU peak after the exploit	**5.4%, 110.8 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**126.4 MiB (95% CI 126.3–126.6 MiB)**	8 samples before the exploit
Memory, container average, after	**126.6 MiB (95% CI 126.5–126.7 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.2 MiB, p = 0.011**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**127.4 MiB, 47.8 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

User enumeration via ?author=
GENERIC-WP-003 · severity MEDIUM
`GET /?author=1`
Generic attack · WordPress core · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 301
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+95), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.



Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.6% (95% CI 0.0–1.6%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.1–0.9%)**	55 samples after the exploit
CPU change after the exploit	**-0.1%, p = 0.903**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**4.2%, 2.9 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**126.8 MiB (95% CI 126.5–127.0 MiB)**	8 samples before the exploit
Memory, container average, after	**125.8 MiB (95% CI 121.0–130.6 MiB)**	55 samples after the exploit
Memory change after the exploit	**-1.0 MiB, p = 0.220**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**126.6 MiB, -0.6 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

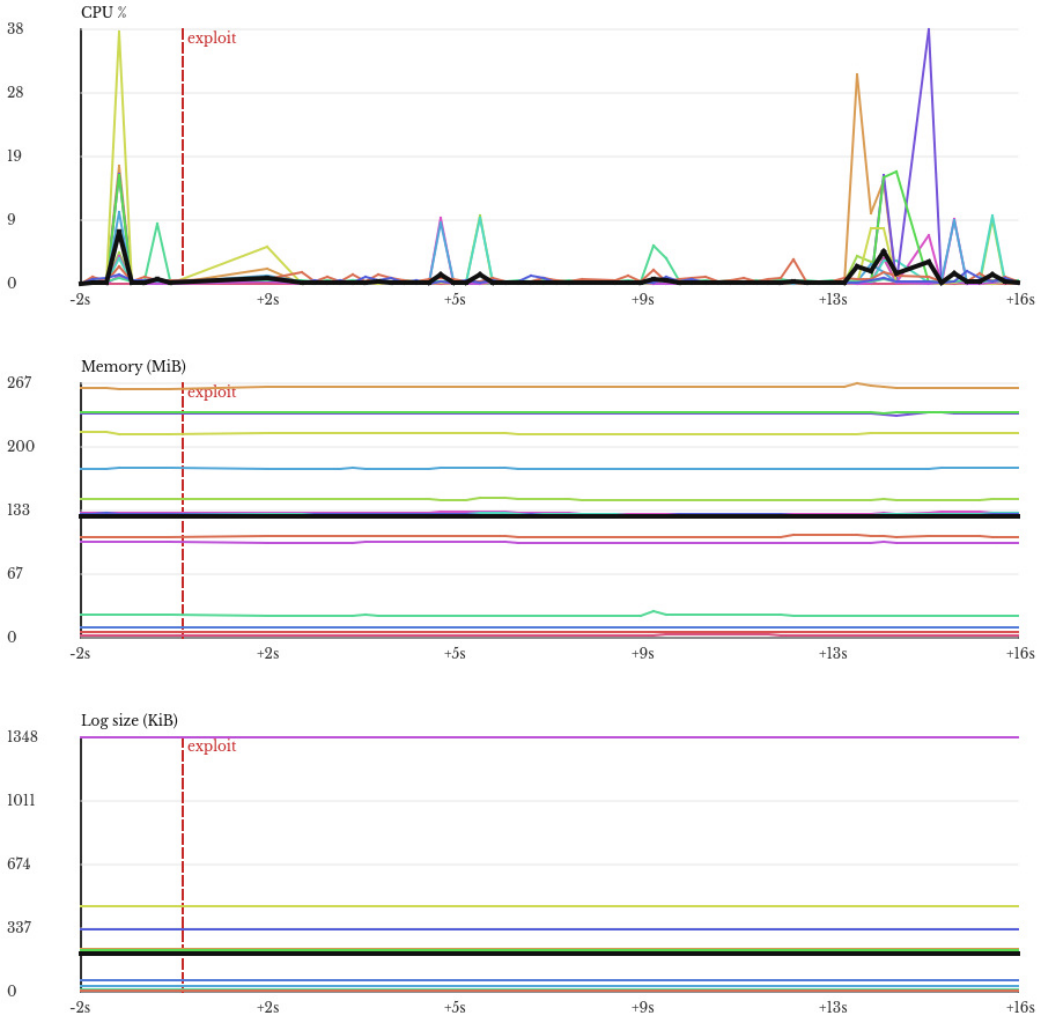
XML-RPC probe
GENERIC-WP-004 · severity MEDIUM
`POST /xmlrpc.php`
Generic attack · WordPress core · Backend

Score: 40/100 Route: HONEYPOT reason: rapid_automation_detected HTTP/2 200
Diverted to honeypot by rule "rapid_automation_detected" – a CVE/exploit match (or IP reputation) diverts regardless of the numeric score.
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+30), Suspicious headers (+5), Automation detected (+5)

```
XML-RPC services are disabled on this server.
```

Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the

footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.



Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**1.1% (95% CI 0.0–3.4%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.1–0.8%)**	55 samples after the exploit
CPU change after the exploit	**-0.7%, p = 0.514**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**4.8%, 1.4 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**126.8 MiB (95% CI 126.7–127.0 MiB)**	8 samples before the exploit
Memory, container average, after	**126.7 MiB (95% CI 126.4–127.0 MiB)**	55 samples after the exploit
Memory change after the exploit	**-0.1 MiB, p = 0.218**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**127.1 MiB, 6.6 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is

		rarely normal fluctuation

Bricks Builder unauthenticated RCE
CVE-2024-25600 · severity CRITICAL
`POST /wp-json/bricks/v1/render_element`
CVE exploit · Plugin / theme · bricks · Backend

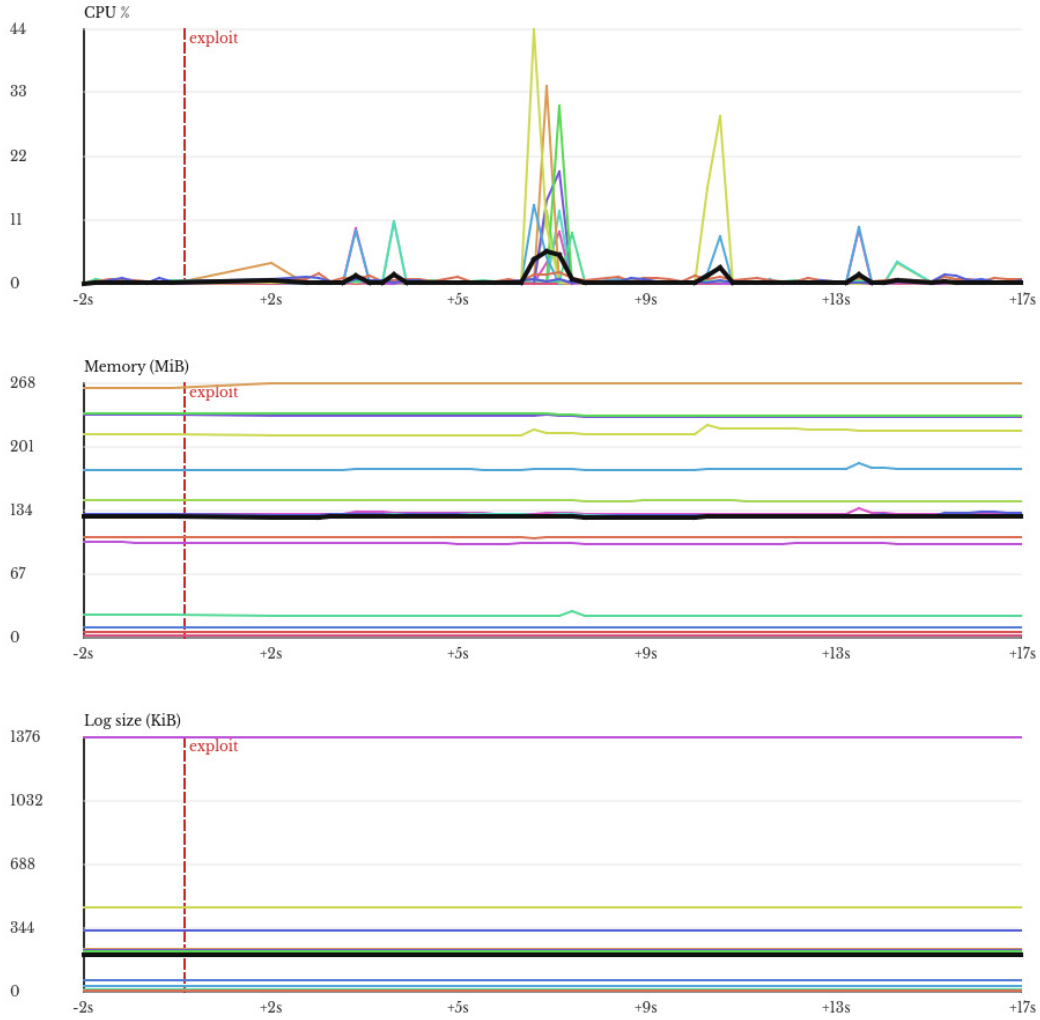
Score: 89/100 Route: HONEYPOT reason: high_threat_score HTTP/2 404
Analyzer score verdict: elevated but below threshold
Signals: Suspicious headers (+5), CVE PATTERNS DETECTED (+40), Automation detected (+5)

Pretty print

{"code":"rest_no_route","message":"No route was found matching the URL and request method.","data":{"status":404}}

Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the

footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

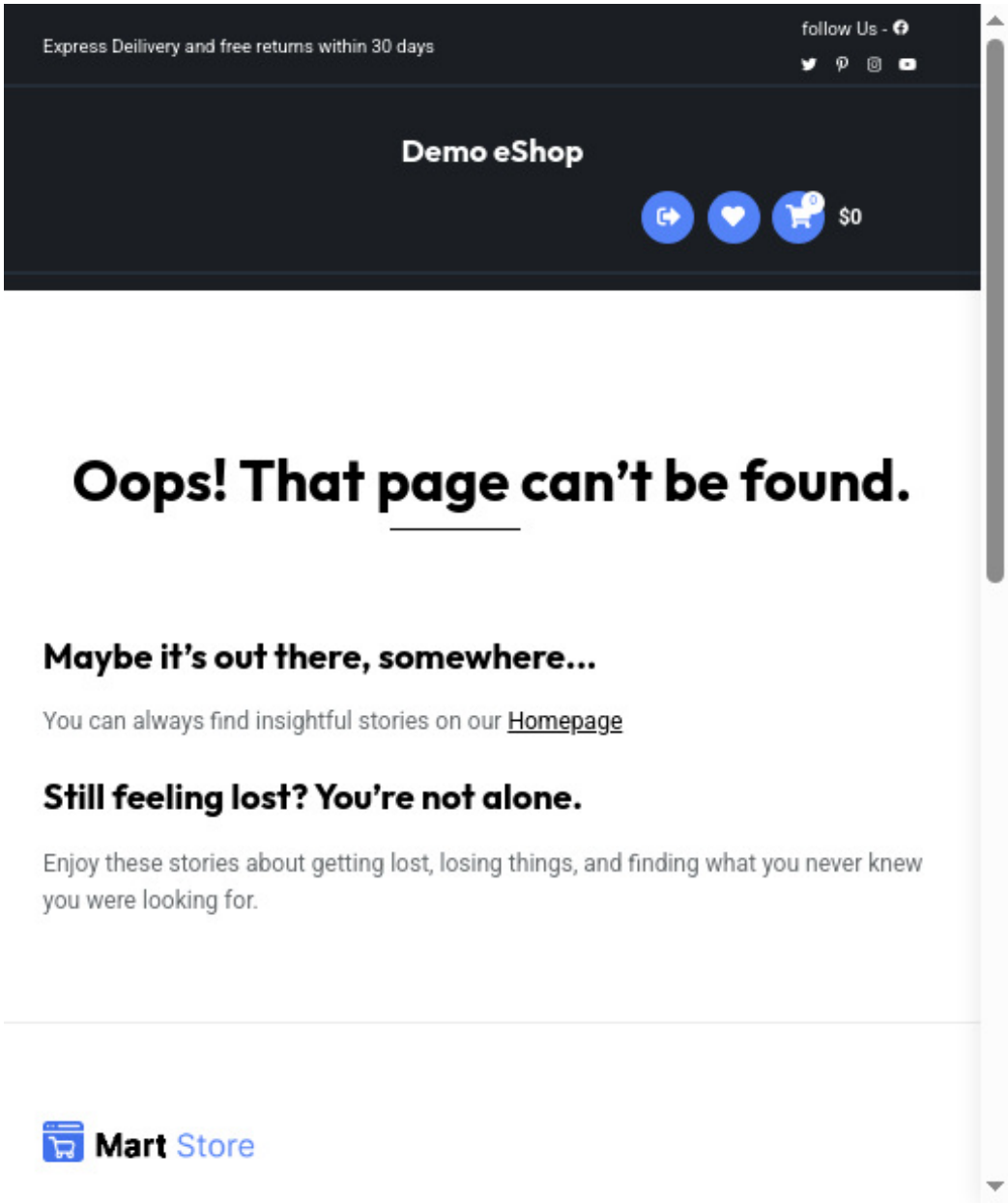


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.0–0.1%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.0–1.2%)**	55 samples after the exploit
CPU change after the exploit	**+0.5%, p = 0.173**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**5.5%, 103.5 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**126.8 MiB (95% CI 126.4–127.2 MiB)**	8 samples before the exploit
Memory, container average, after	**127.0 MiB (95% CI 126.9–127.1 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.2 MiB, p = 0.021**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**127.7 MiB, 20.3 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

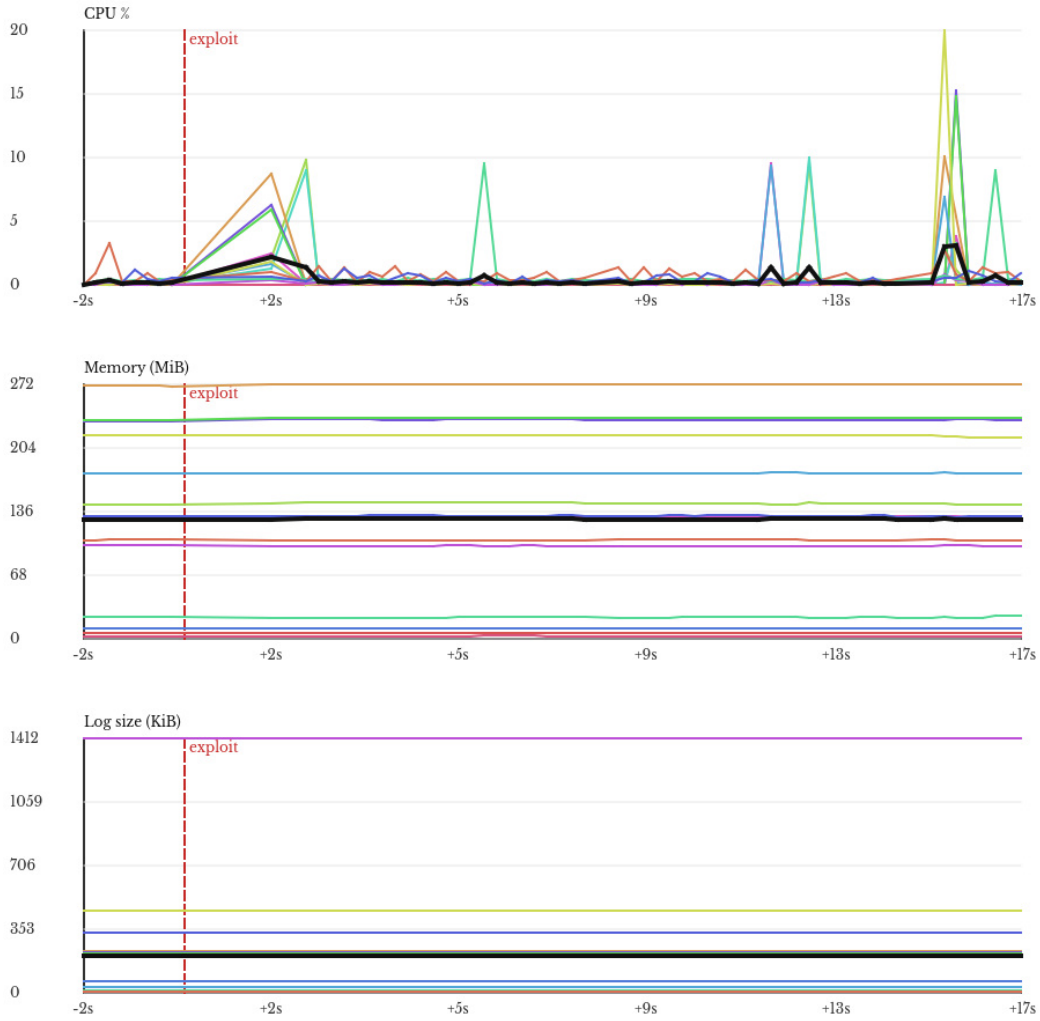
WP Automatic SQL injection
CVE-2024-27956 · severity CRITICAL
`GET /wp-content/plugins/wp-automatic/inc/csv.php?q=1'%20UNION%20SELECT%20user_login%2Cuser_pass%20FROM%20wp_users--%20-`
CVE exploit · Plugin / theme · wp-automatic · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 404
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+80), CVE PATTERNS DETECTED (+40), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.

CPU and memory were read from the containers' cgroup files every 0.25 s.



Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.0–0.2%)**	8 samples before the exploit
CPU, container average, after	**0.3% (95% CI 0.1–0.6%)**	55 samples after the exploit
CPU change after the exploit	**+0.2%, p = 0.083**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**3.0%, 34.2 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**127.1 MiB (95% CI 127.1–127.1 MiB)**	8 samples before the exploit
Memory, container average, after	**127.6 MiB (95% CI 127.5–127.7 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.5 MiB, p < 0.001**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**127.7 MiB, 26.0 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

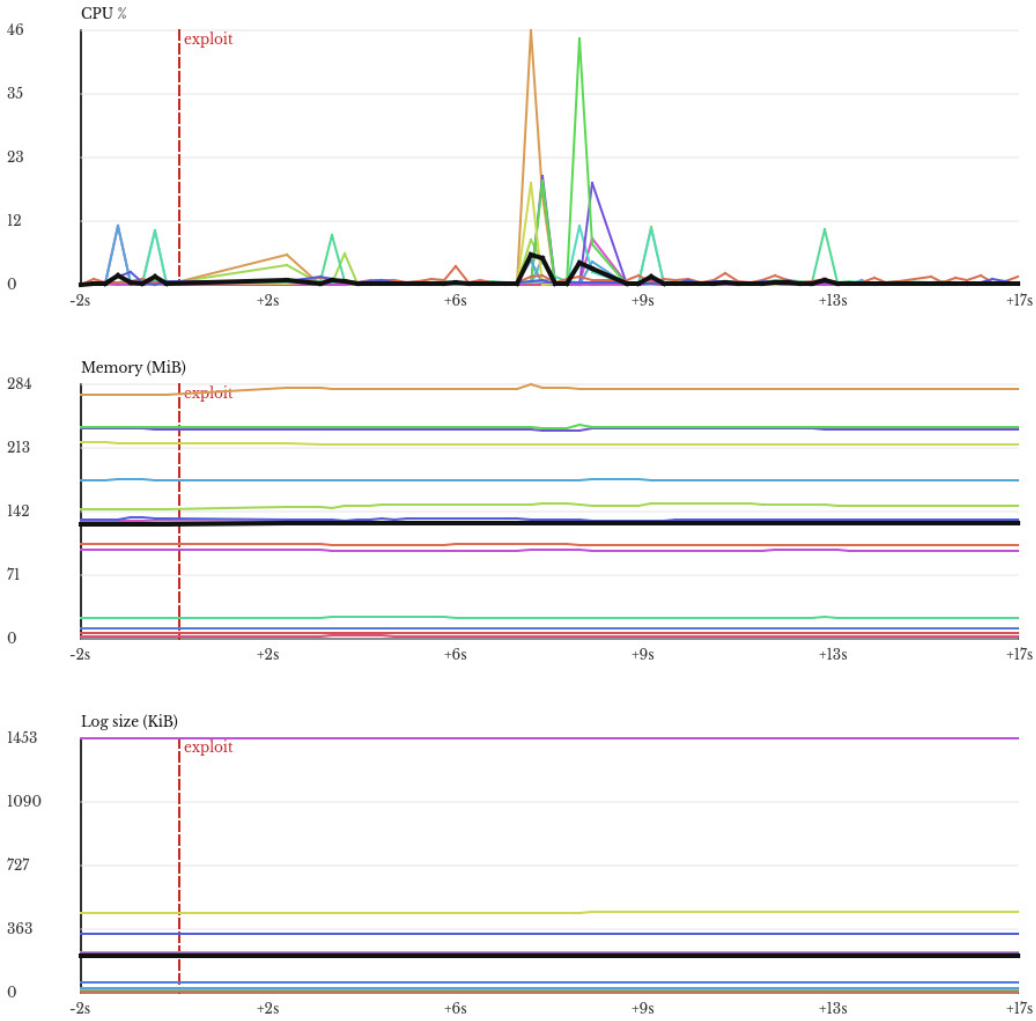
Ultimate Member SQL injection
CVE-2024-1071 · severity CRITICAL
`GET /wp-admin/admin-ajax.php?action=um_get_members&directory_id=1'%20UNION%20SELECT%20user_login%2Cuser_pass%20FROM%20wp_users--%20-`
CVE exploit · Plugin / theme · ultimate-member · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 400
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+80), CVE PATTERNS DETECTED (+40), Automation detected (+5)

```
0
```

Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.

CPU and memory were read from the containers' cgroup files every 0.25 s.



Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.4% (95% CI 0.0–1.0%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.0–1.0%)**	55 samples after the exploit
CPU change after the exploit	**+0.0%, p = 0.884**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**5.3%, 7.7 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**127.8 MiB (95% CI 126.9–128.7 MiB)**	8 samples before the exploit
Memory, container average, after	**128.2 MiB (95% CI 128.1–128.3 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.4 MiB, p = 0.045**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**128.7 MiB, 8.6 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

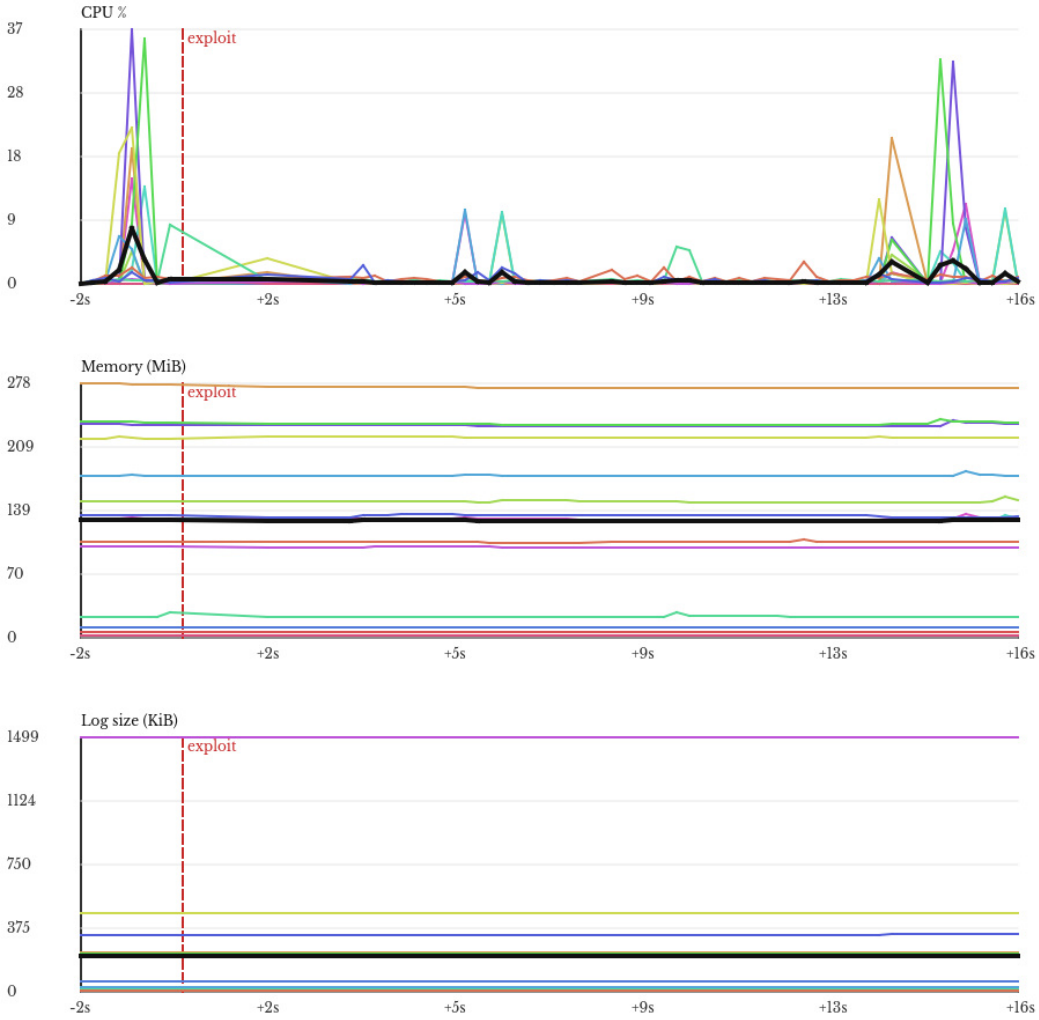
BookingPress SQL injection
CVE-2022-0739 · severity CRITICAL
`GET /wp-admin/admin-ajax.php?action=bookingpress_front_get_category_services&total_service=1'%20UNION%20SELECT%20user_login%2Cuser_pass%20FROM%20wp_users--%20-`
CVE exploit · Plugin / theme · bookingpress-appointment-booking · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 400
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+80), CVE PATTERNS DETECTED (+40), Automation detected (+5)

```
0
```

Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the

footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.



Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**1.8% (95% CI 0.0–5.7%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.1–0.8%)**	51 samples after the exploit
CPU change after the exploit	**-1.3%, p = 0.380**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**3.3%, 0.6 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**128.3 MiB (95% CI 128.1–128.5 MiB)**	8 samples before the exploit
Memory, container average, after	**127.6 MiB (95% CI 127.3–127.9 MiB)**	51 samples after the exploit
Memory change after the exploit	**-0.7 MiB, p < 0.001**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**128.4 MiB, 0.3 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

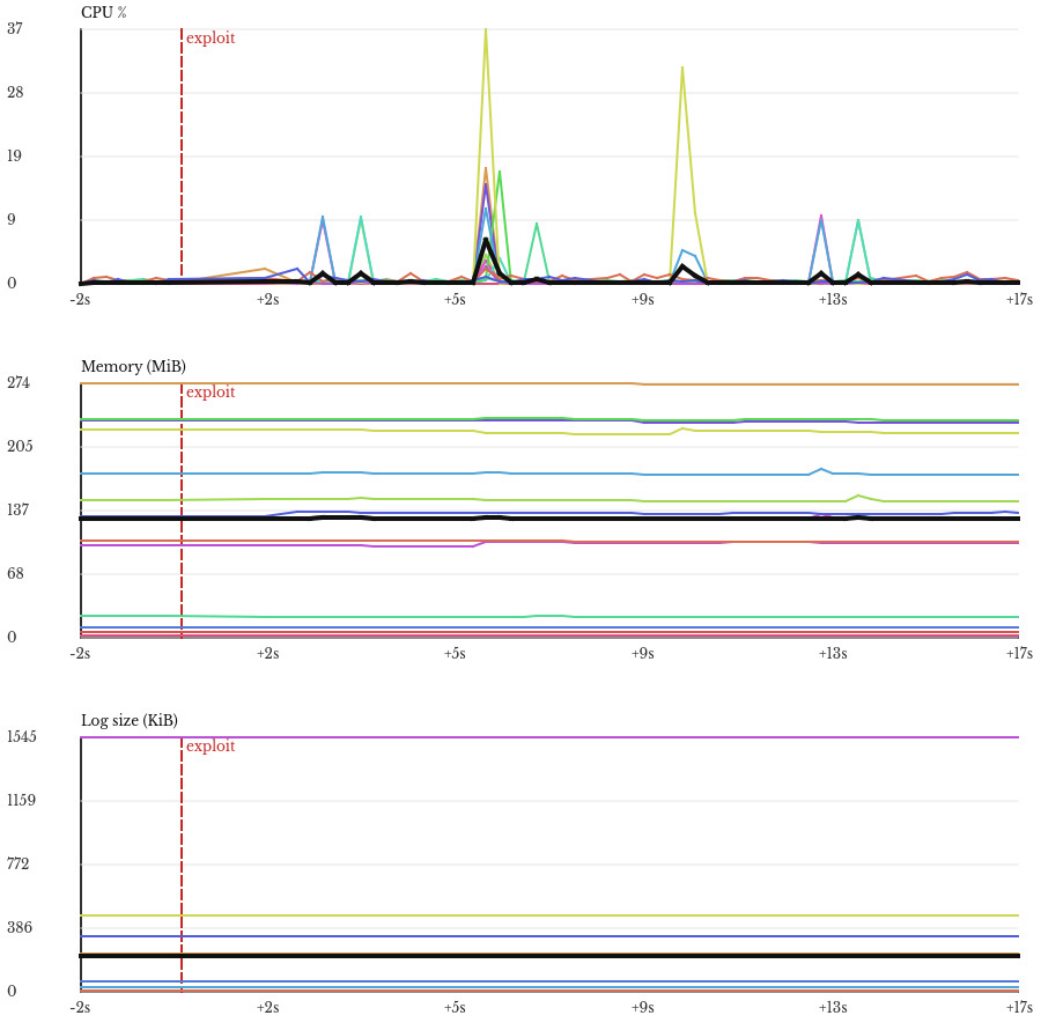
LayerSlider SQL injection
CVE-2024-2879 · severity CRITICAL
`GET /wp-admin/admin-ajax.php?action=ls_get_popup_markup&id=1'%20UNION%20SELECT%20user_login%2Cuser_pass%20FROM%20wp_users--%20-`
CVE exploit · Plugin / theme · layerslider · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 400
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+80), CVE PATTERNS DETECTED (+40), Automation detected (+5)

```
0
```

Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the

footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.



Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.1–0.1%)**	8 samples before the exploit
CPU, container average, after	**0.4% (95% CI 0.1–0.7%)**	56 samples after the exploit
CPU change after the exploit	**+0.3%, p = 0.028**	mean after vs mean before; unlikely to be chance (significant at 5%)
CPU peak after the exploit	**6.3%, 139.2 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**127.9 MiB (95% CI 127.7–128.1 MiB)**	8 samples before the exploit
Memory, container average, after	**127.7 MiB (95% CI 127.3–128.1 MiB)**	56 samples after the exploit
Memory change after the exploit	**-0.2 MiB, p = 0.303**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**128.4 MiB, 12.3 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

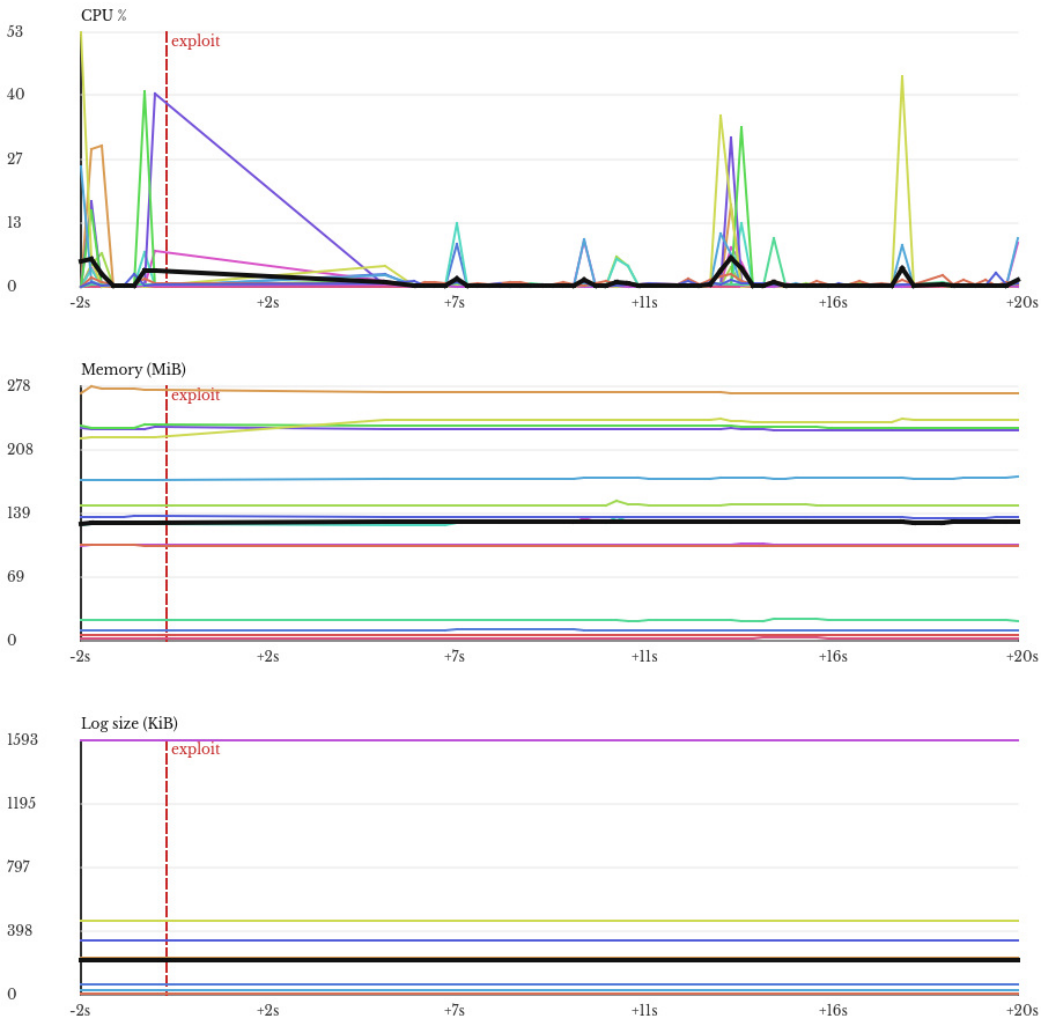
REST API content injection (defacement)
CVE-2017-1001000 · severity HIGH
`POST /wp-json/wp/v2/posts/1?id=1abc`
CVE exploit · WordPress core · Frontend

Score: 10/100 Route: PRODUCTION HTTP/2 400
Analyzer score verdict: clean
Signals: Suspicious headers (+5), Automation detected (+5)

Pretty print ☐


```
{"code":"rest_invalid_param","message":"Invalid parameter(s): id","data":{"status":400,"params":
{"id":"id is not of type integer."},"details":{"id":{"code":"rest_invalid_type","message":"id is
not of type integer.","data":{"param":"id"}}}}}
```


Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

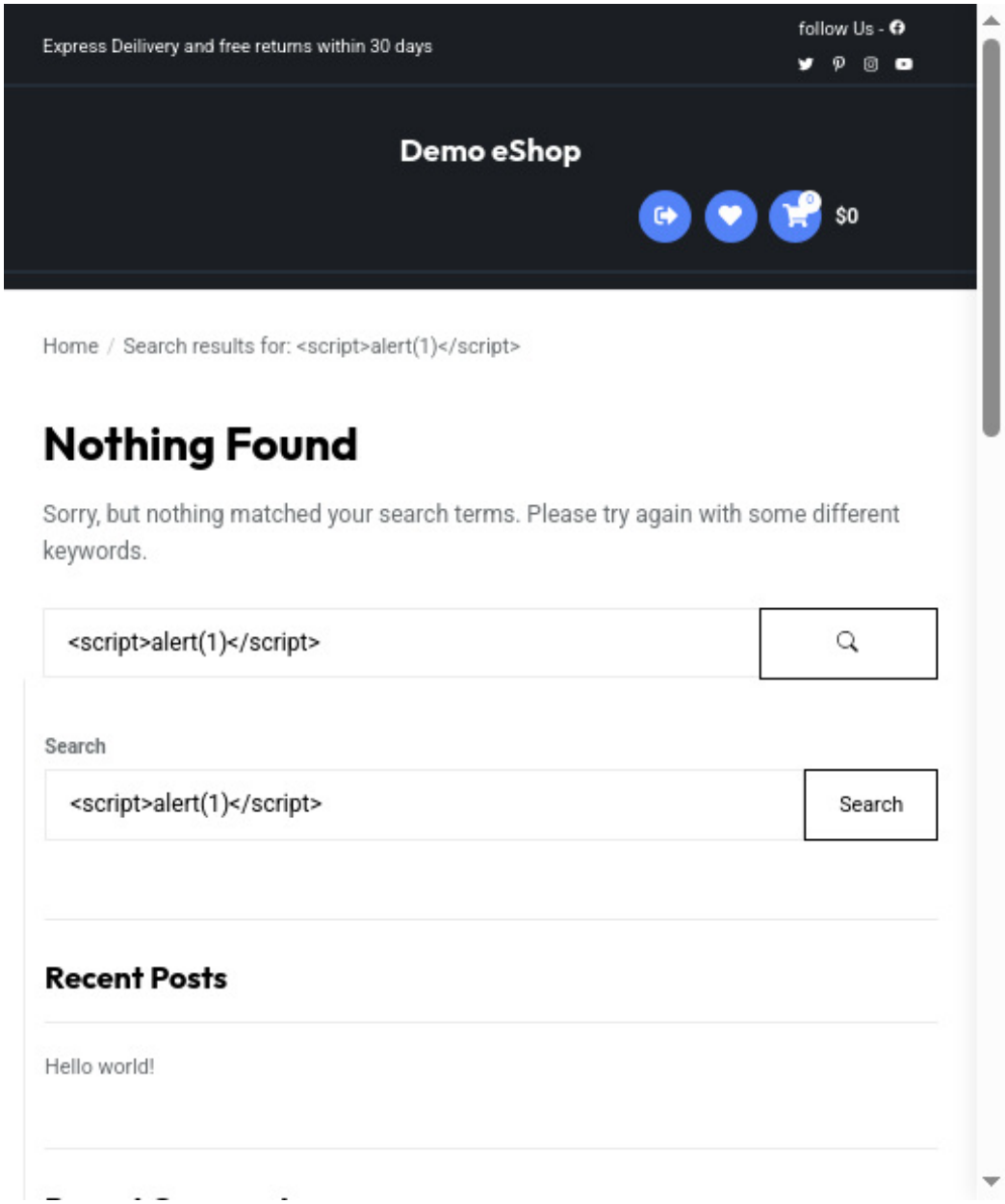


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**2.6% (95% CI 0.0–11.5%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.0–1.1%)**	55 samples after the exploit
CPU change after the exploit	**-2.0%, p = 0.332**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**5.8%, 1.5 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**127.9 MiB (95% CI 127.7–128.0 MiB)**	8 samples before the exploit
Memory, container average, after	**129.2 MiB (95% CI 129.0–129.4 MiB)**	55 samples after the exploit
Memory change after the exploit	**+1.3 MiB, p < 0.001**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**130.1 MiB, 10.9 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

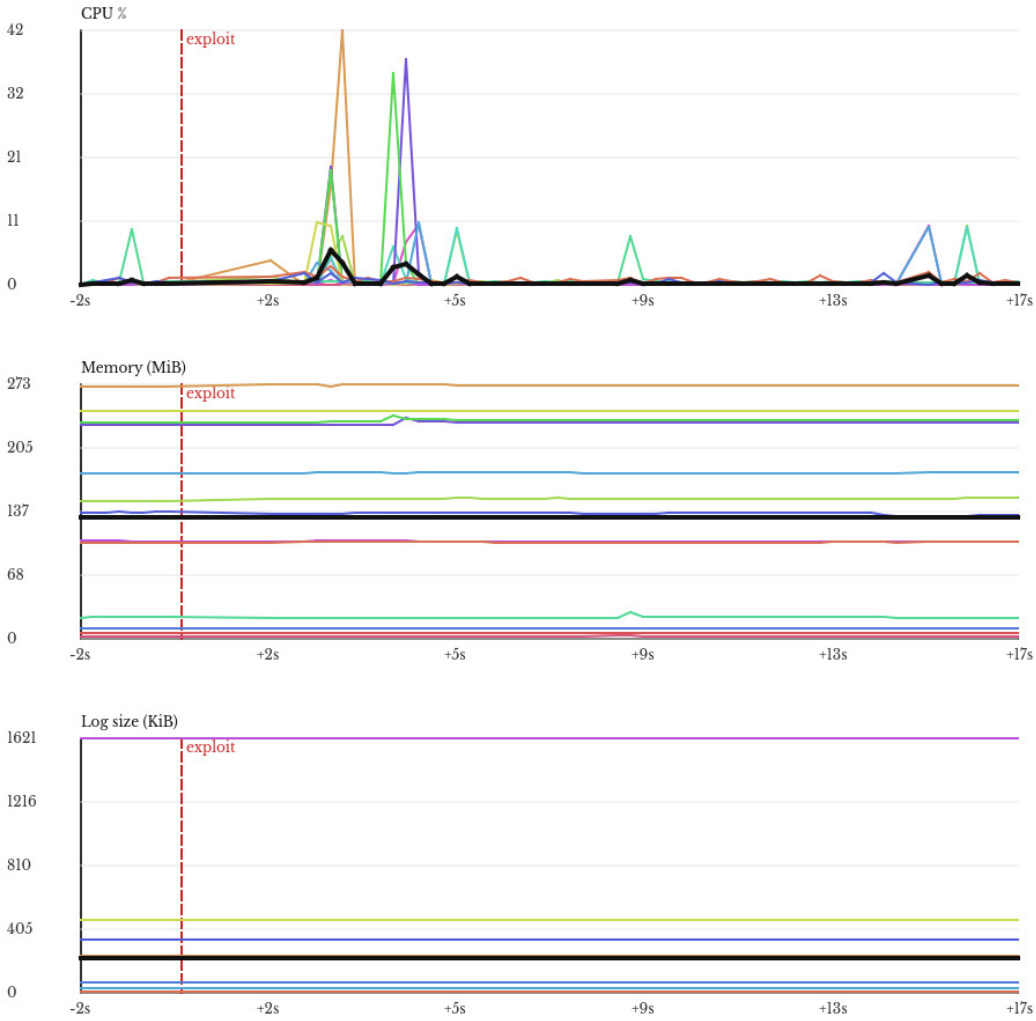
Script tag in a search form
GENERIC-XSS-001 · severity HIGH
`GET /?s=<script>alert(1)</script>`
Generic attack · WordPress core · Frontend

Score: 75/100 Route: HONEYPOT reason: rapid_automation_detected HTTP/2 200
Diverted to honeypot by rule "rapid_automation_detected" – a CVE/exploit match (or IP reputation) diverts regardless of the numeric score.
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+70), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.

CPU and memory were read from the containers' cgroup files every 0.25 s.

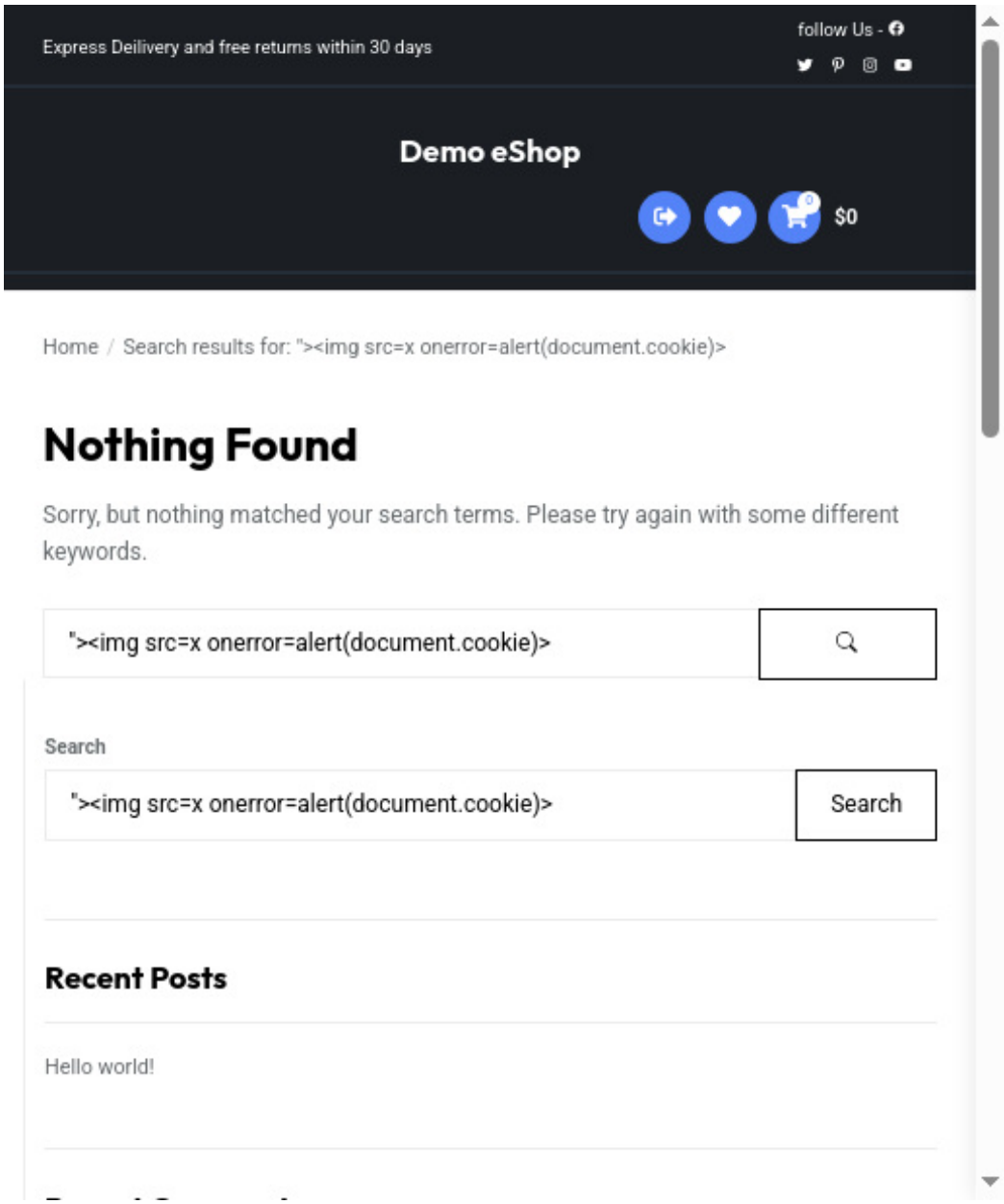


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

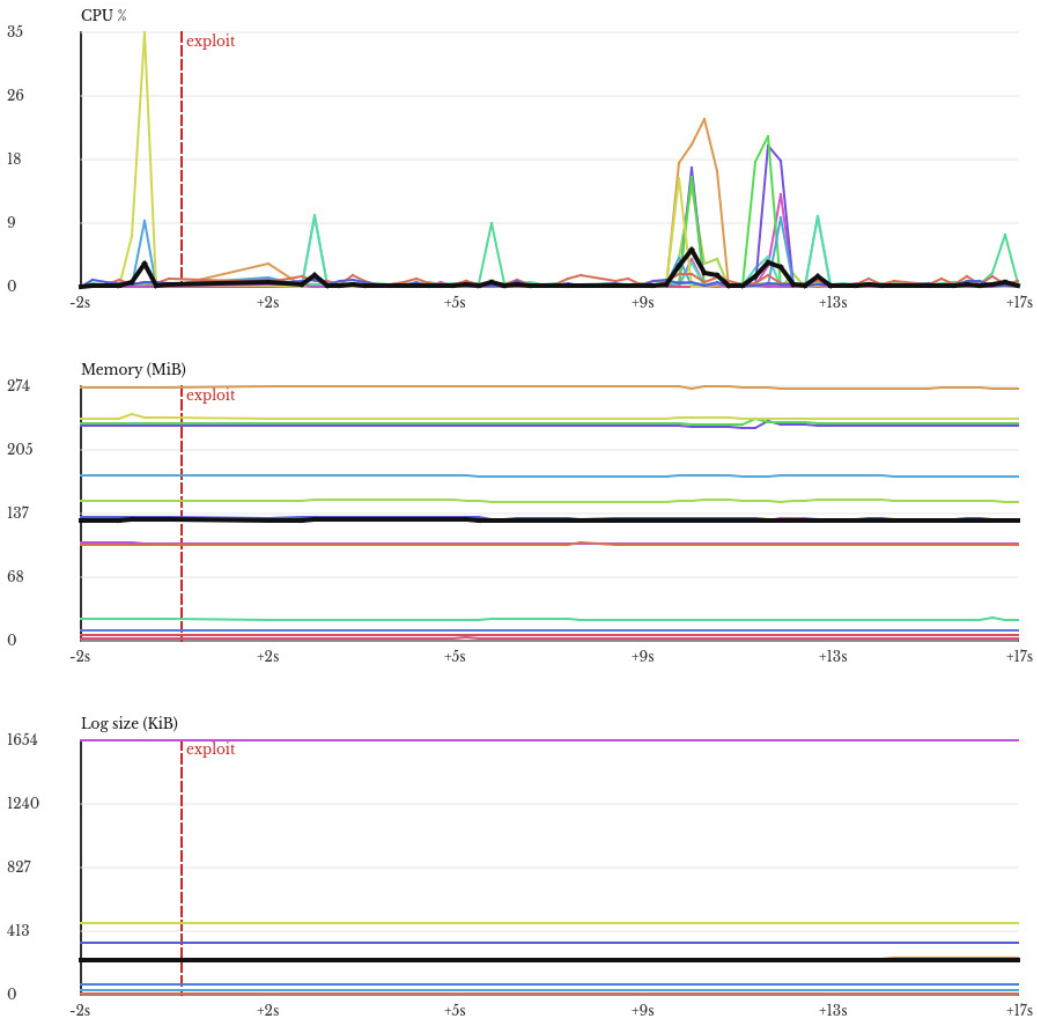
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.2% (95% CI 0.0–0.4%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.0–1.0%)**	55 samples after the exploit
CPU change after the exploit	**+0.3%, p = 0.209**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**5.8%, 26.6 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**129.2 MiB (95% CI 129.2–129.2 MiB)**	8 samples before the exploit
Memory, container average, after	**129.7 MiB (95% CI 129.6–129.9 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.5 MiB, p < 0.001**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**130.3 MiB, 53.0 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Event-handler XSS
GENERIC-XSS-002 · severity HIGH
`GET /?s=%22%3E%3Cimg%20src=x%20onerror=alert(document.cookie)%3E`
Generic attack · WordPress core · Frontend

Score: 80/100 Route: HONEYPOT reason: high_threat_score HTTP/2 200
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+75), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

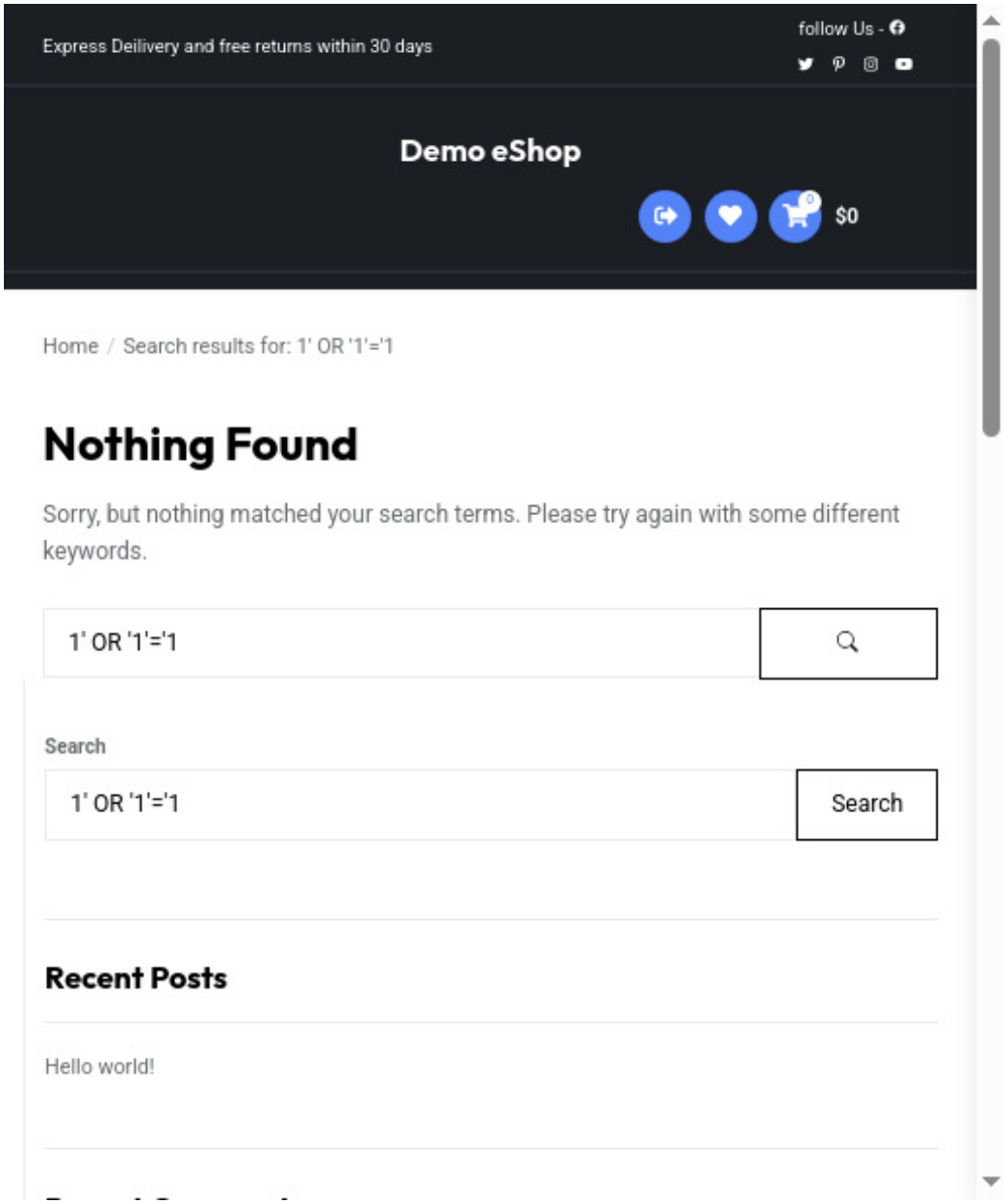


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.5% (95% CI 0.0–1.4%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.0–1.0%)**	55 samples after the exploit
CPU change after the exploit	**-0.0%, p = 0.962**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**5.0%, 4.3 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**129.5 MiB (95% CI 129.4–129.6 MiB)**	8 samples before the exploit
Memory, container average, after	**129.1 MiB (95% CI 128.9–129.4 MiB)**	55 samples after the exploit
Memory change after the exploit	**-0.3 MiB, p = 0.016**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**129.5 MiB, 0.3 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

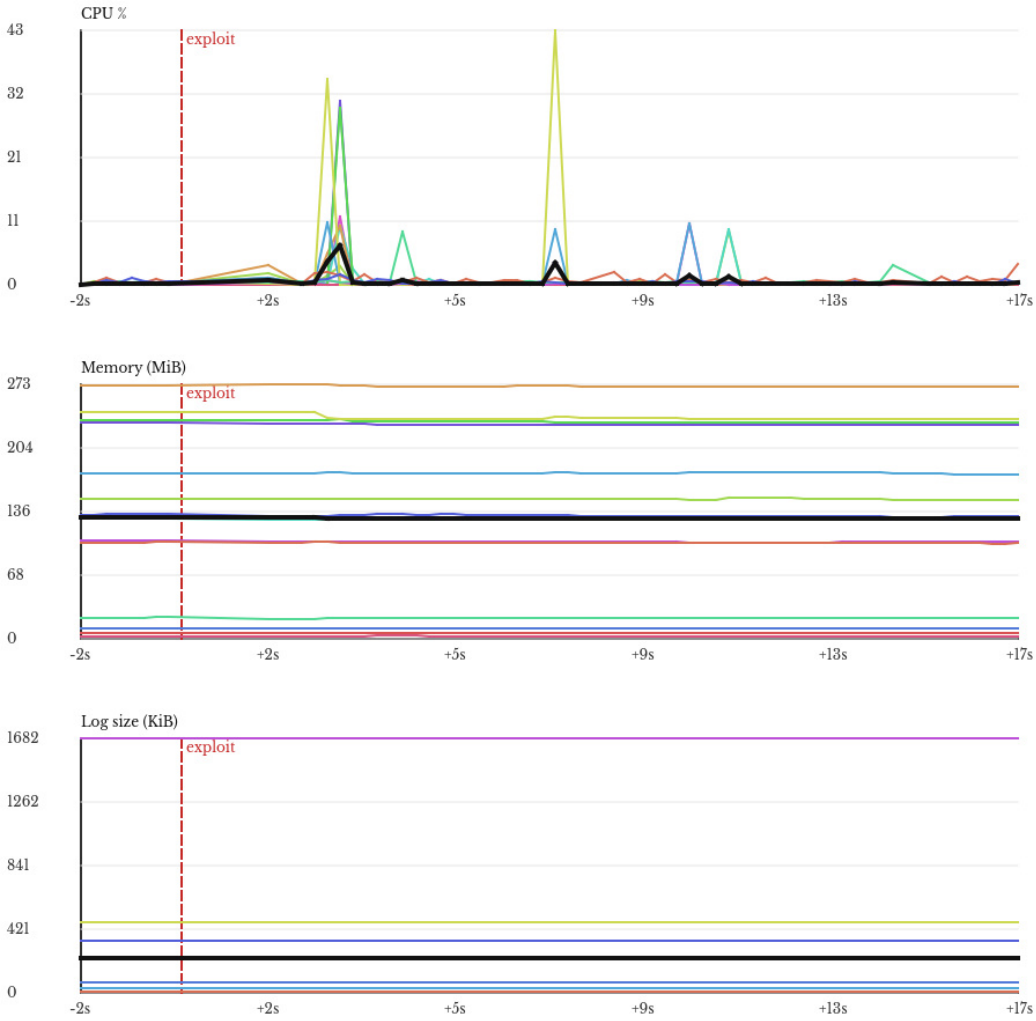
SQL injection in a search form
GENERIC-SQLI-001 · severity HIGH
`GET /?s=1'%20OR%20'1'='1`
Generic attack · WordPress core · Backend

Score: 70/100 Route: HONEYPOT reason: rapid_automation_detected HTTP/2 200
Diverted to honeypot by rule "rapid_automation_detected" – a CVE/exploit match (or IP reputation) diverts regardless of the numeric score.
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+65), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.

CPU and memory were read from the containers' cgroup files every 0.25 s.

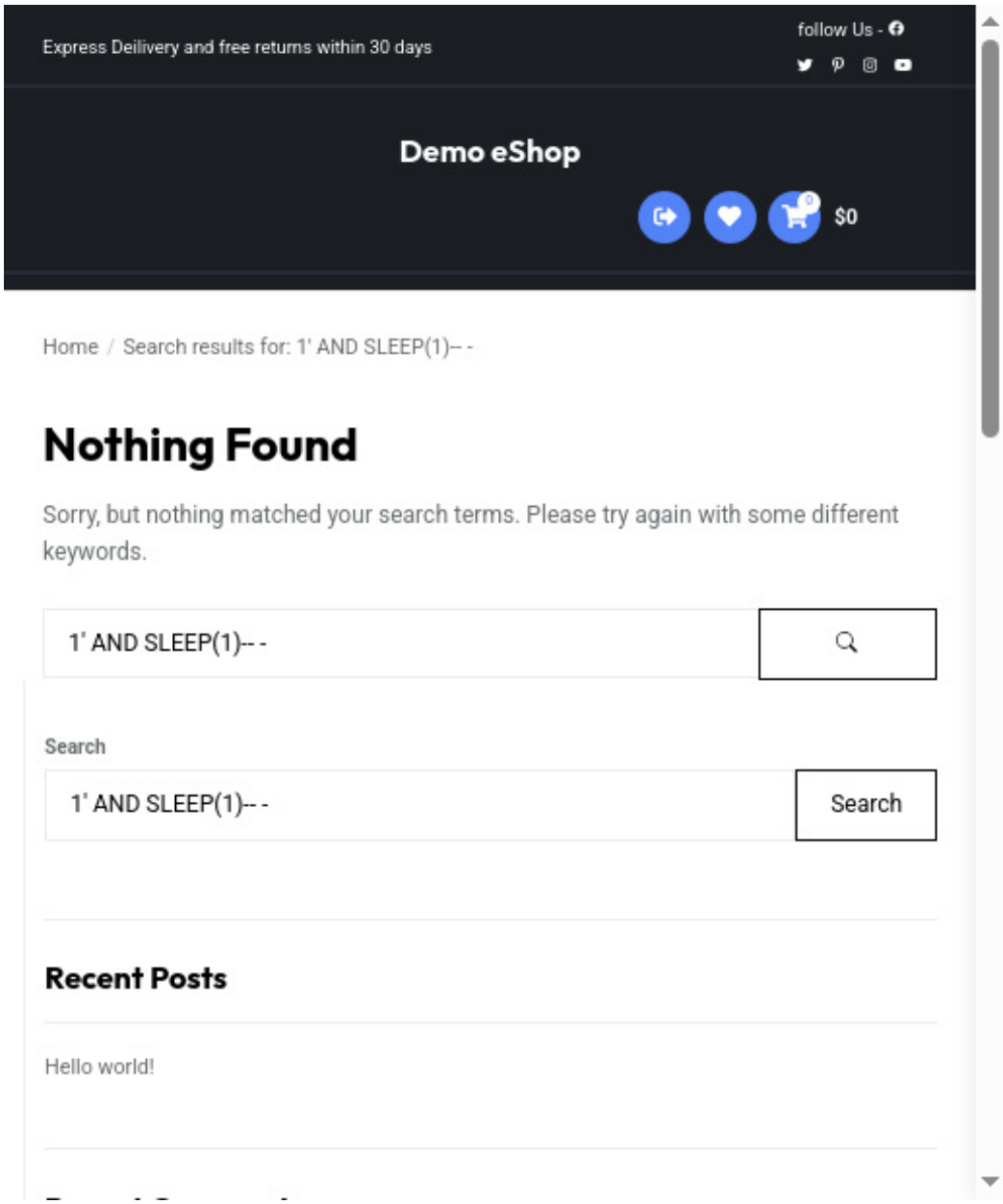


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

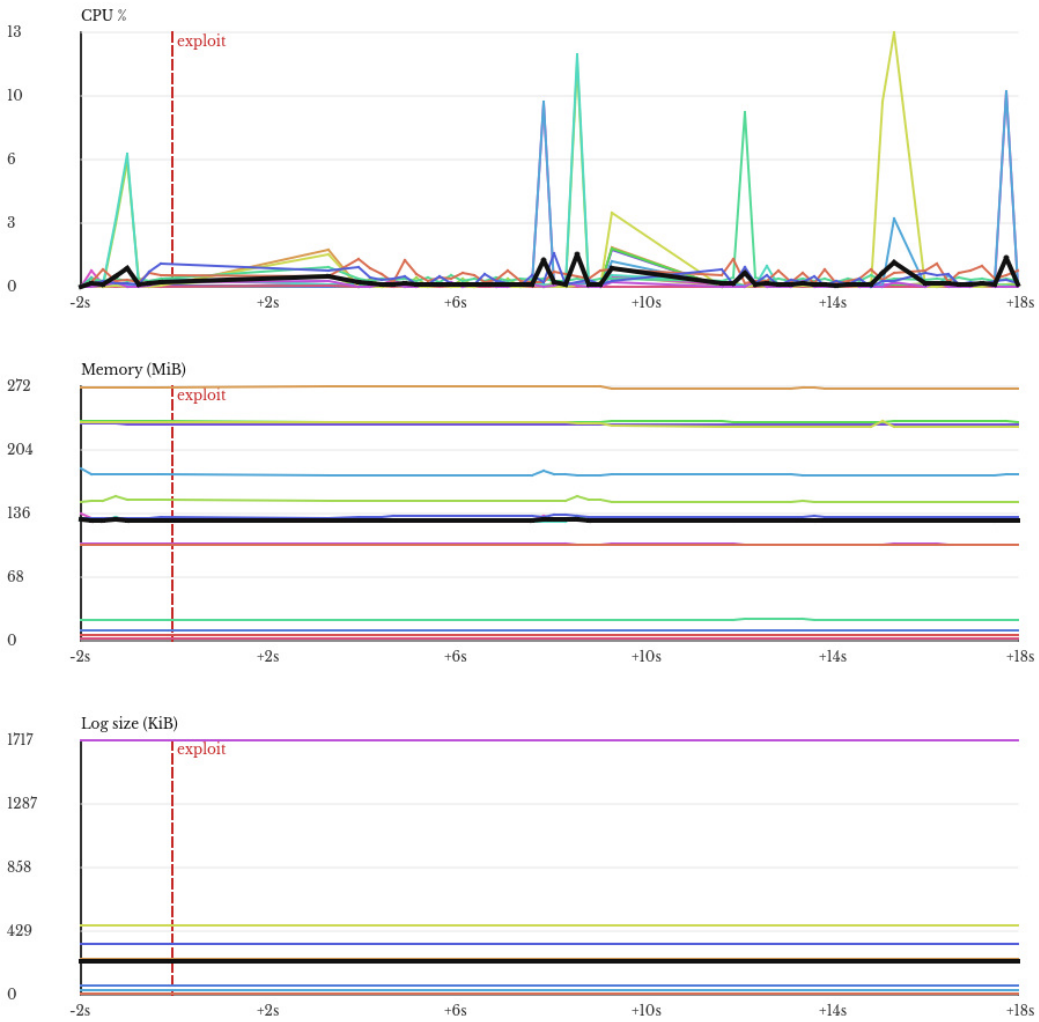
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.1–0.1%)**	8 samples before the exploit
CPU, container average, after	**0.4% (95% CI 0.0–0.8%)**	55 samples after the exploit
CPU change after the exploit	**+0.3%, p = 0.115**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**6.5%, 137.7 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**129.3 MiB (95% CI 129.2–129.4 MiB)**	8 samples before the exploit
Memory, container average, after	**128.2 MiB (95% CI 127.7–128.7 MiB)**	55 samples after the exploit
Memory change after the exploit	**-1.1 MiB, p = 0.006**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**128.9 MiB, -19.5 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Time-based blind SQL injection
GENERIC-SQLI-002 · severity HIGH
`GET /?s=1'%20AND%20SLEEP(1)--%20-`
Generic attack · WordPress core · Backend

Score: 30/100 Route: PRODUCTION HTTP/2 200
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+25), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

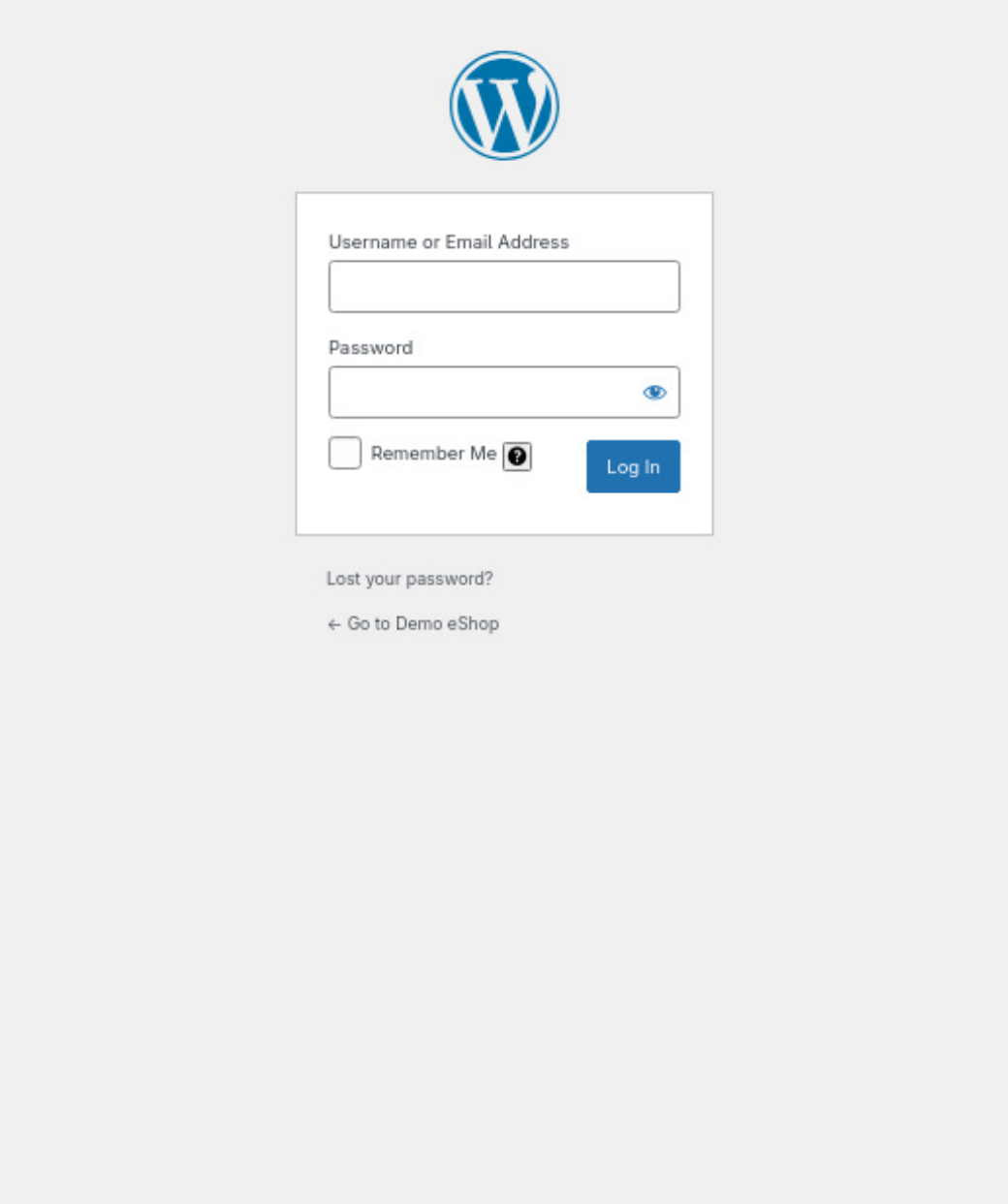


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

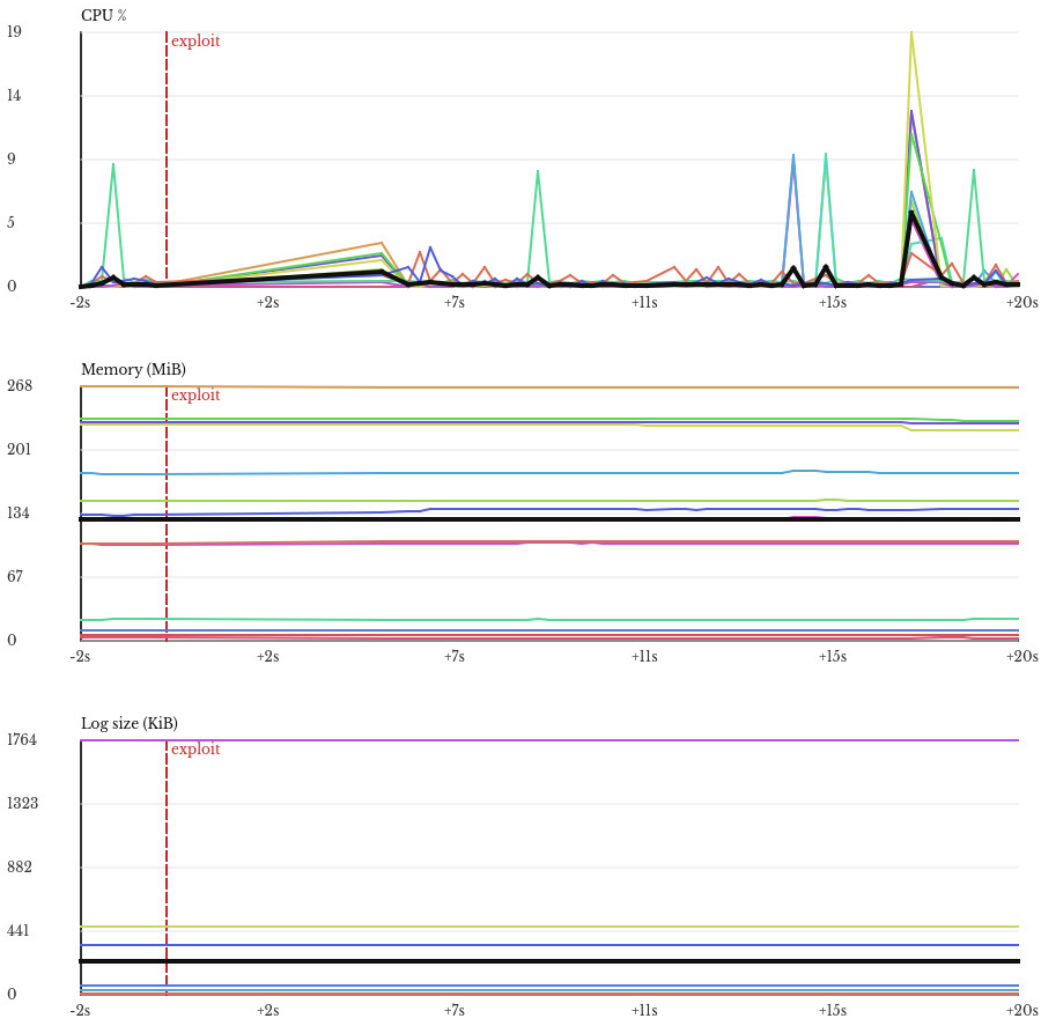
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.3% (95% CI 0.0–0.6%)**	8 samples before the exploit
CPU, container average, after	**0.3% (95% CI 0.2–0.4%)**	49 samples after the exploit
CPU change after the exploit	**-0.0%, p = 0.980**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**1.6%, 4.4 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**128.5 MiB (95% CI 128.2–128.8 MiB)**	8 samples before the exploit
Memory, container average, after	**128.0 MiB (95% CI 127.3–128.7 MiB)**	49 samples after the exploit
Memory change after the exploit	**-0.5 MiB, p = 0.094**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**129.0 MiB, 1.6 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

SQL injection in the login form (POST body)
GENERIC-SQLI-003 · severity HIGH
`POST /wp-login.php`
Generic attack · WordPress core · Backend

Score: 10/100 Route: PRODUCTION HTTP/2 200
Analyzer score verdict: clean
Signals: Suspicious headers (+5), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

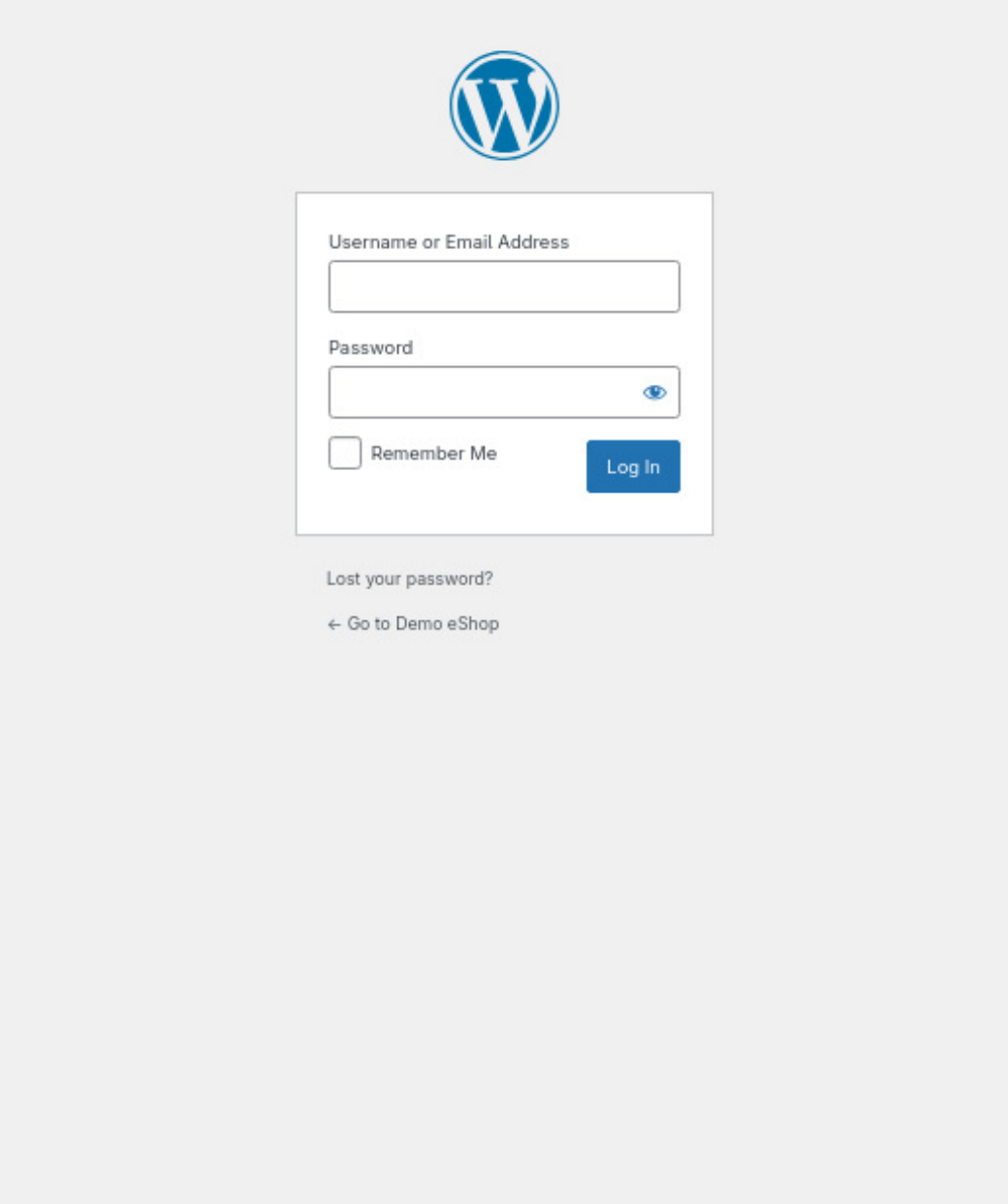


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

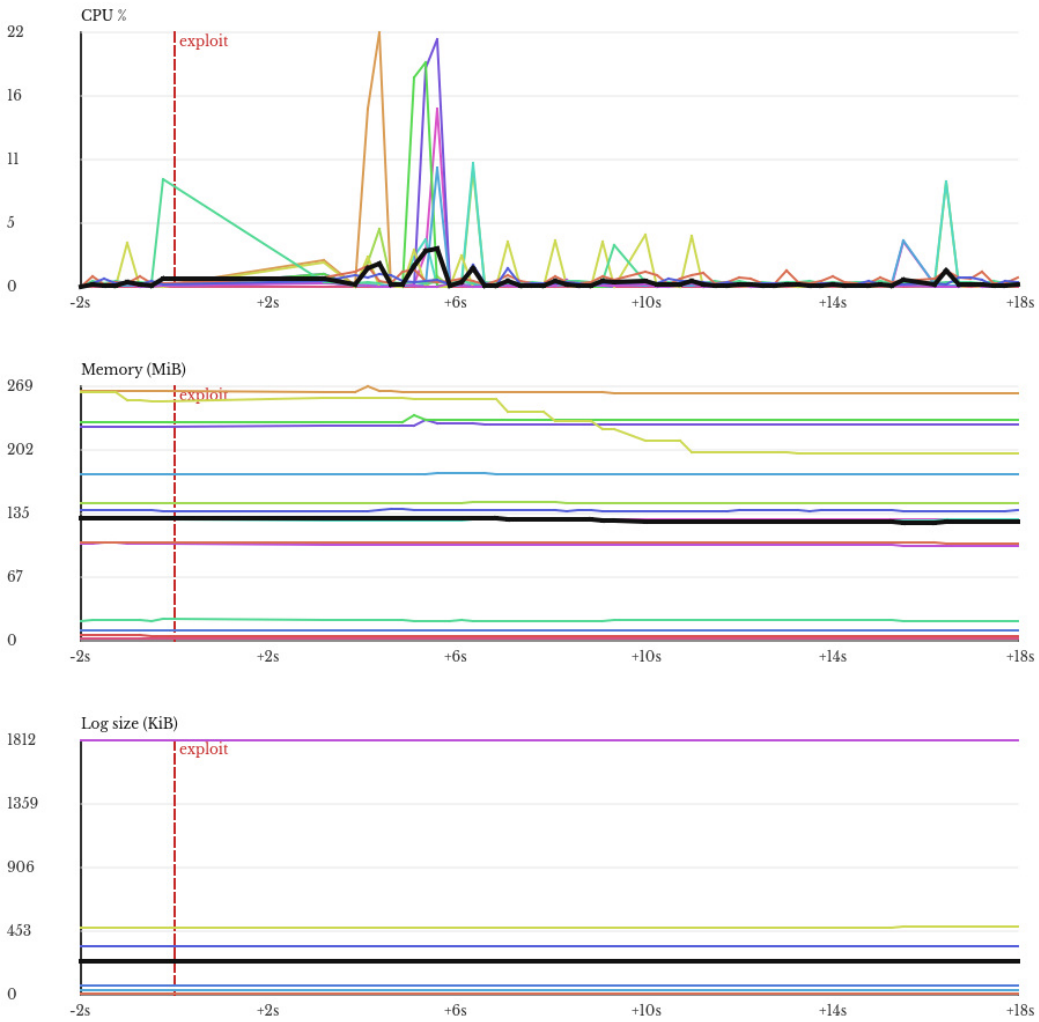
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.2% (95% CI 0.0–0.3%)**	8 samples before the exploit
CPU, container average, after	**0.3% (95% CI 0.1–0.5%)**	55 samples after the exploit
CPU change after the exploit	**+0.1%, p = 0.297**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**5.4%, 25.8 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**127.4 MiB (95% CI 127.2–127.5 MiB)**	8 samples before the exploit
Memory, container average, after	**127.7 MiB (95% CI 127.3–128.1 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.4 MiB, p = 0.060**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**128.0 MiB, 8.1 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Wordlist scan: sensitive paths
GENERIC-ENUM-001 · severity MEDIUM
`GET /admin`
Generic attack · WordPress core · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 302
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+20), Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

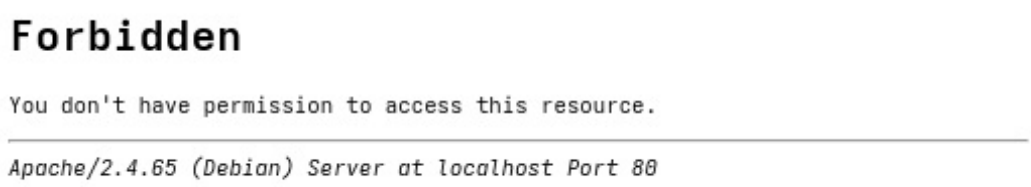


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

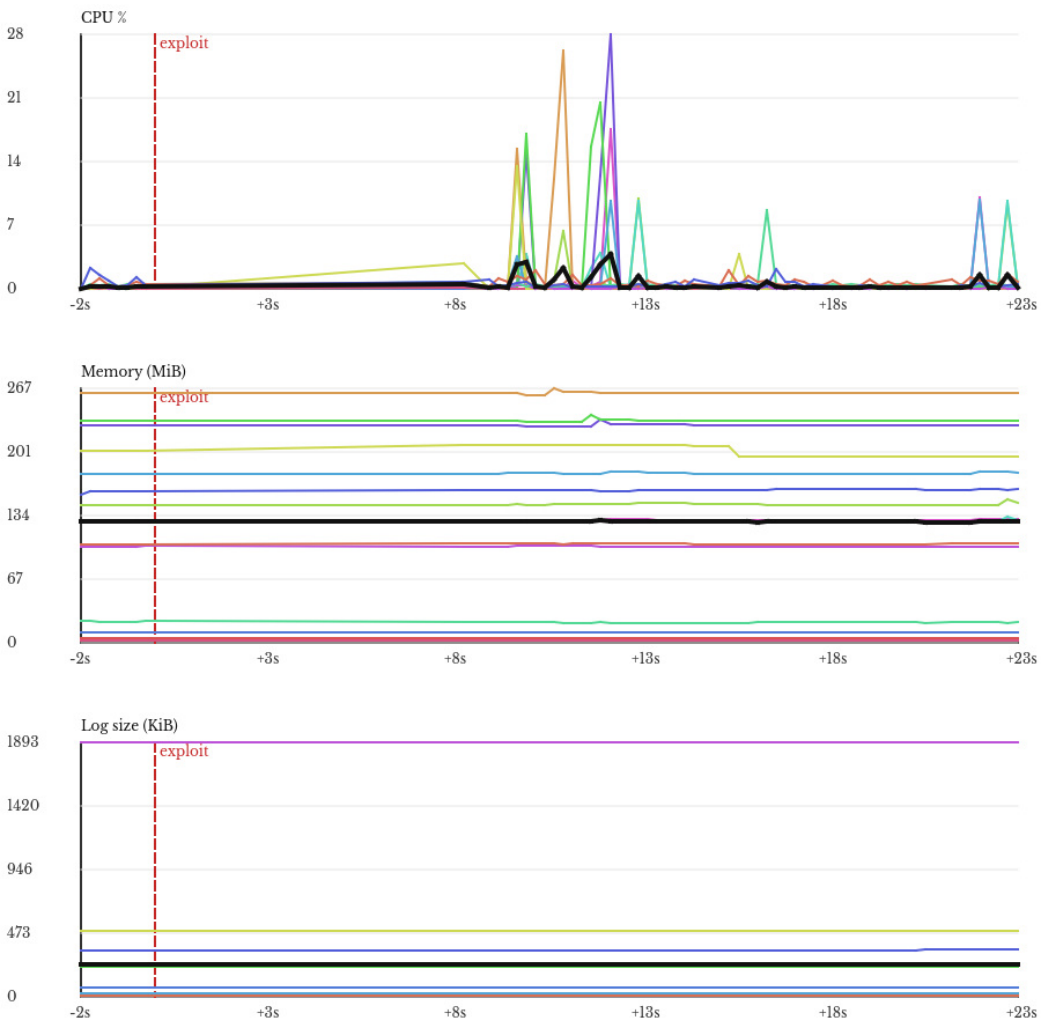
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.2% (95% CI 0.0–0.4%)**	8 samples before the exploit
CPU, container average, after	**0.4% (95% CI 0.1–0.7%)**	55 samples after the exploit
CPU change after the exploit	**+0.2%, p = 0.219**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**3.3%, 14.6 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**129.0 MiB (95% CI 126.0–132.1 MiB)**	8 samples before the exploit
Memory, container average, after	**126.5 MiB (95% CI 110.5–142.5 MiB)**	55 samples after the exploit
Memory change after the exploit	**-2.5 MiB, p = 0.285**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**129.3 MiB, 0.9 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Wordlist scan: plugin and theme discovery
GENERIC-ENUM-002 · severity MEDIUM
`GET /wp-content/plugins/akismet/readme.txt`
Generic attack · Plugin / theme · Backend

Score: 5/100 Route: PRODUCTION HTTP/2 403
Analyzer score verdict: clean
Signals: Automation detected (+5)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.



Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.1–0.2%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.2–0.8%)**	55 samples after the exploit
CPU change after the exploit	**+0.4%, p = 0.039**	mean after vs mean before; unlikely to be chance (significant at 5%)
CPU peak after the exploit	**3.8%, 48.4 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**126.6 MiB (95% CI 126.6–126.7 MiB)**	8 samples before the exploit
Memory, container average, after	**126.9 MiB (95% CI 125.7–128.1 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.3 MiB, p = 0.441**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**127.8 MiB, 12.8 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

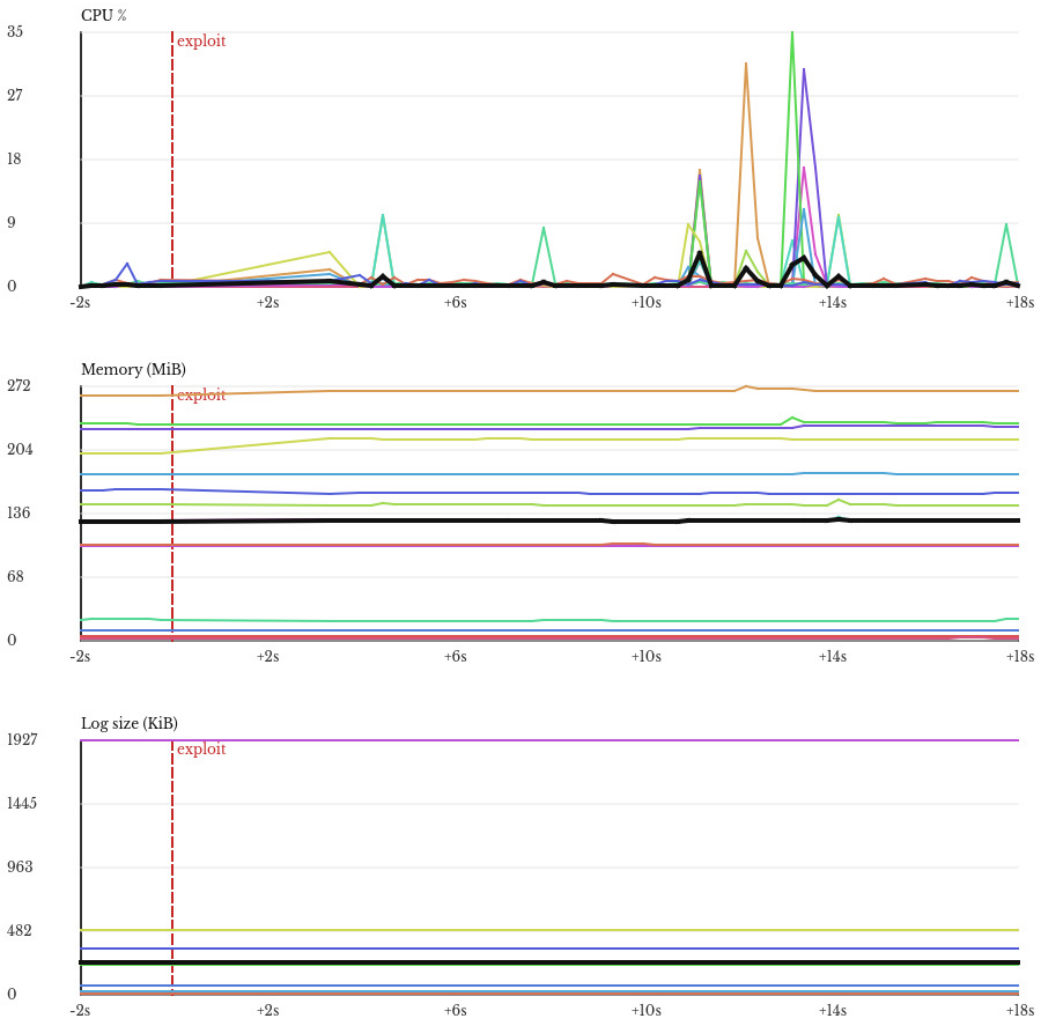
Insecure direct object reference: user IDs
GENERIC-IDOR-001 · severity MEDIUM
`GET /wp-json/wp/v2/users/1`
Generic attack · WordPress core · Backend

Score: 30/100 Route: PRODUCTION HTTP/2 401
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+25), Automation detected (+5)

Pretty print ☐

{"code":"rest_forbidden","message":"Sorry, you are not allowed to do that.","data":{"status":401}}

Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.



Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.0–0.2%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.1–0.8%)**	56 samples after the exploit
CPU change after the exploit	**+0.4%, p = 0.044**	mean after vs mean before; unlikely to be chance (significant at 5%)
CPU peak after the exploit	**4.5%, 56.2 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**126.7 MiB (95% CI 126.7–126.8 MiB)**	8 samples before the exploit
Memory, container average, after	**127.7 MiB (95% CI 127.5–128.0 MiB)**	56 samples after the exploit
Memory change after the exploit	**+1.0 MiB, p < 0.001**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**128.6 MiB, 60.0 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

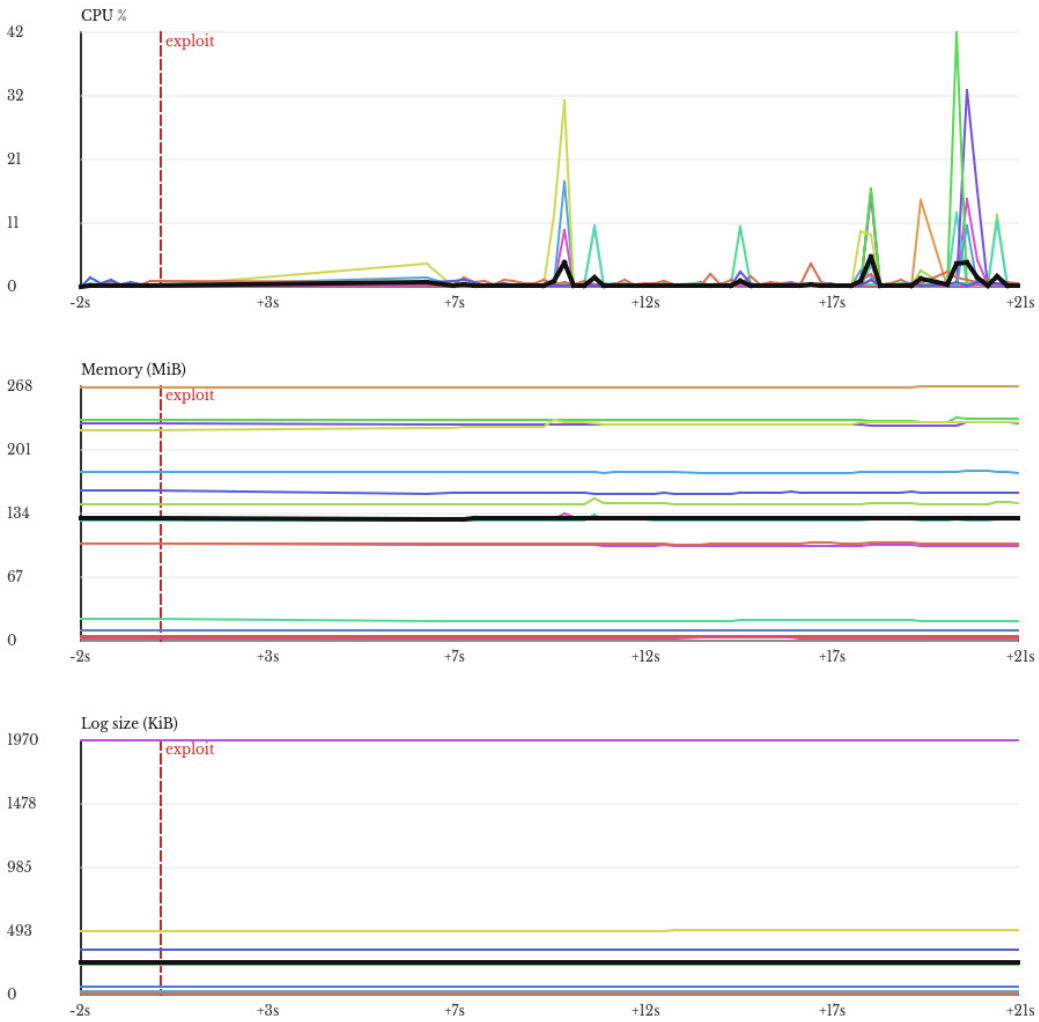
Insecure direct object reference: order IDs
GENERIC-IDOR-002 · severity MEDIUM
`GET /wp-json/wc/v3/orders/1`
Generic attack · Plugin / theme · woocommerce · Backend

Score: 5/100 Route: PRODUCTION HTTP/2 401
Analyzer score verdict: clean
Signals: Automation detected (+5)

Pretty print

{"code":"woocommerce_rest_cannot_view","message":"Sorry, you cannot view this resource.","data":
{"status":401}}

Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

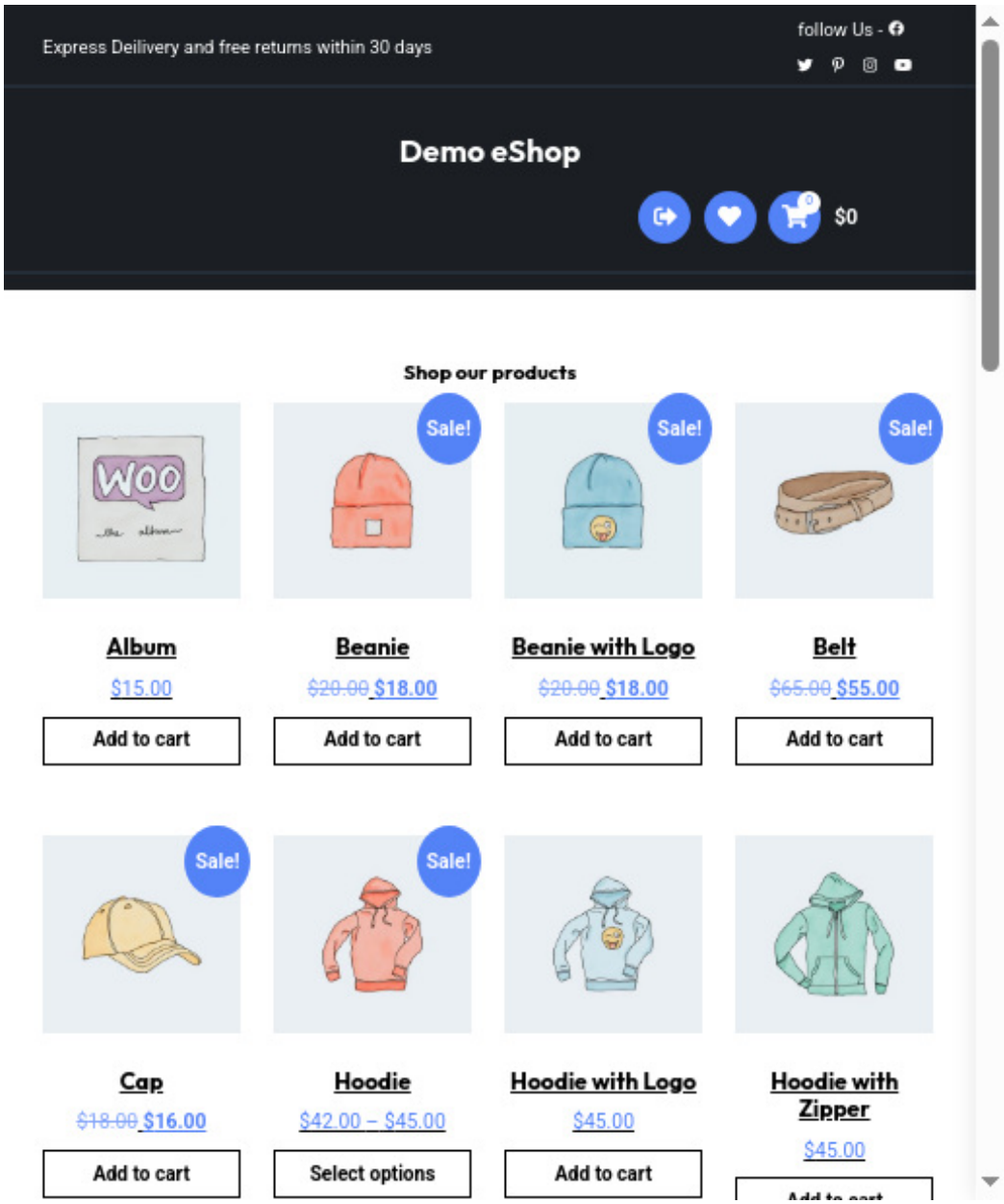


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

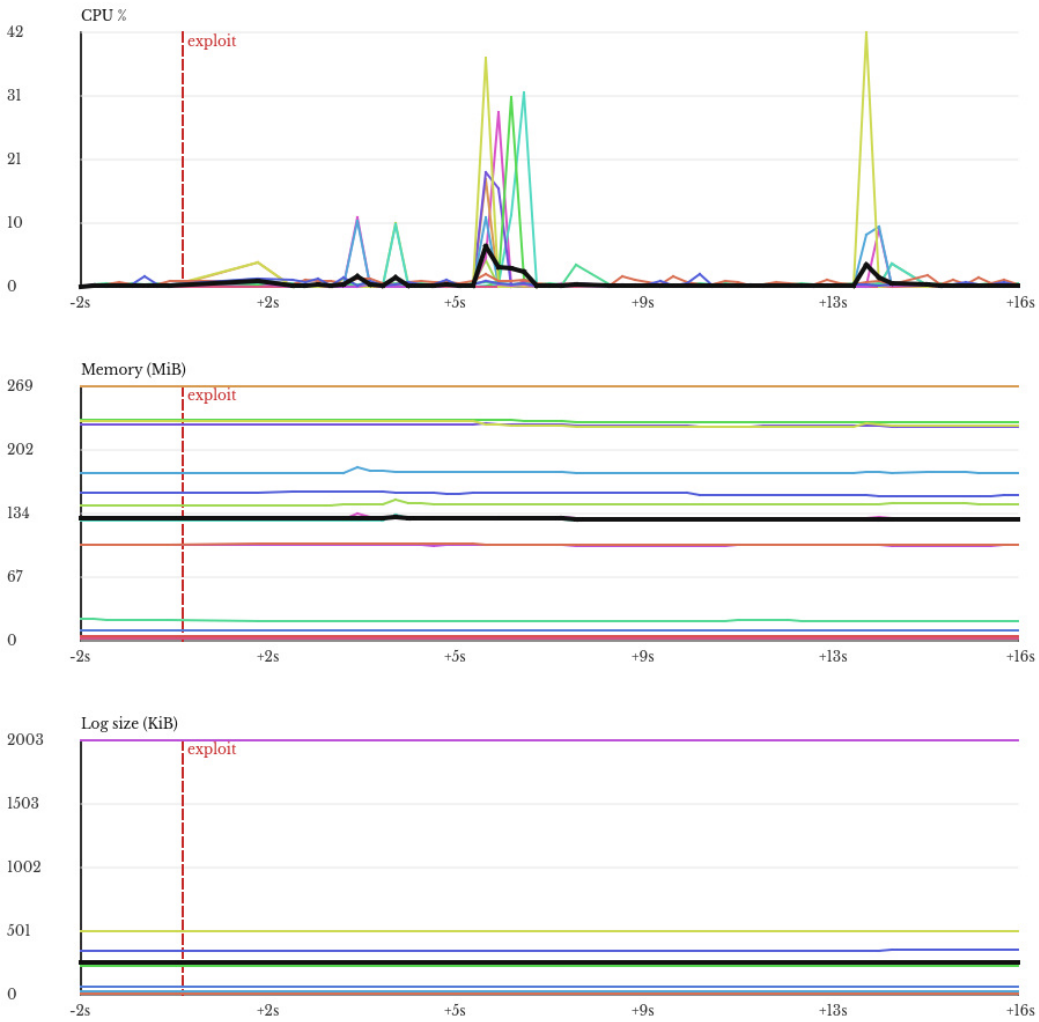
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.1–0.1%)**	8 samples before the exploit
CPU, container average, after	**0.6% (95% CI 0.2–0.9%)**	55 samples after the exploit
CPU change after the exploit	**+0.5%, p = 0.022**	mean after vs mean before; unlikely to be chance (significant at 5%)
CPU peak after the exploit	**4.9%, 85.7 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**128.3 MiB (95% CI 128.3–128.3 MiB)**	8 samples before the exploit
Memory, container average, after	**128.4 MiB (95% CI 128.2–128.6 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.1 MiB, p = 0.385**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**129.0 MiB, 1059.4 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Scanner tool: sqlmap
GENERIC-TOOL-001 · severity MEDIUM
`GET /`
Generic attack · WordPress core · Backend

Score: 50/100 Route: PRODUCTION HTTP/2 200
Analyzer score verdict: elevated but below threshold
Signals: Suspicious headers (+50)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

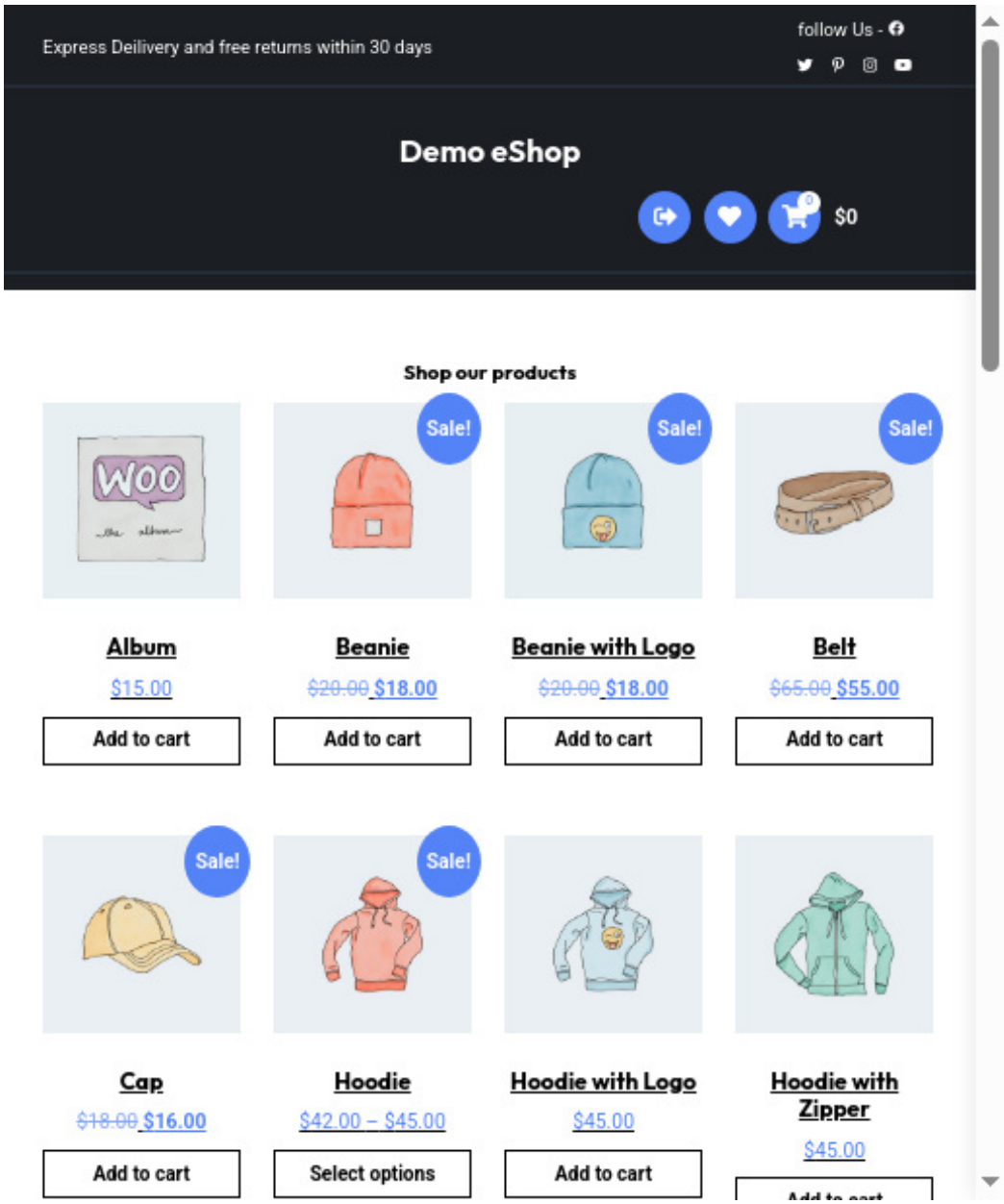


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

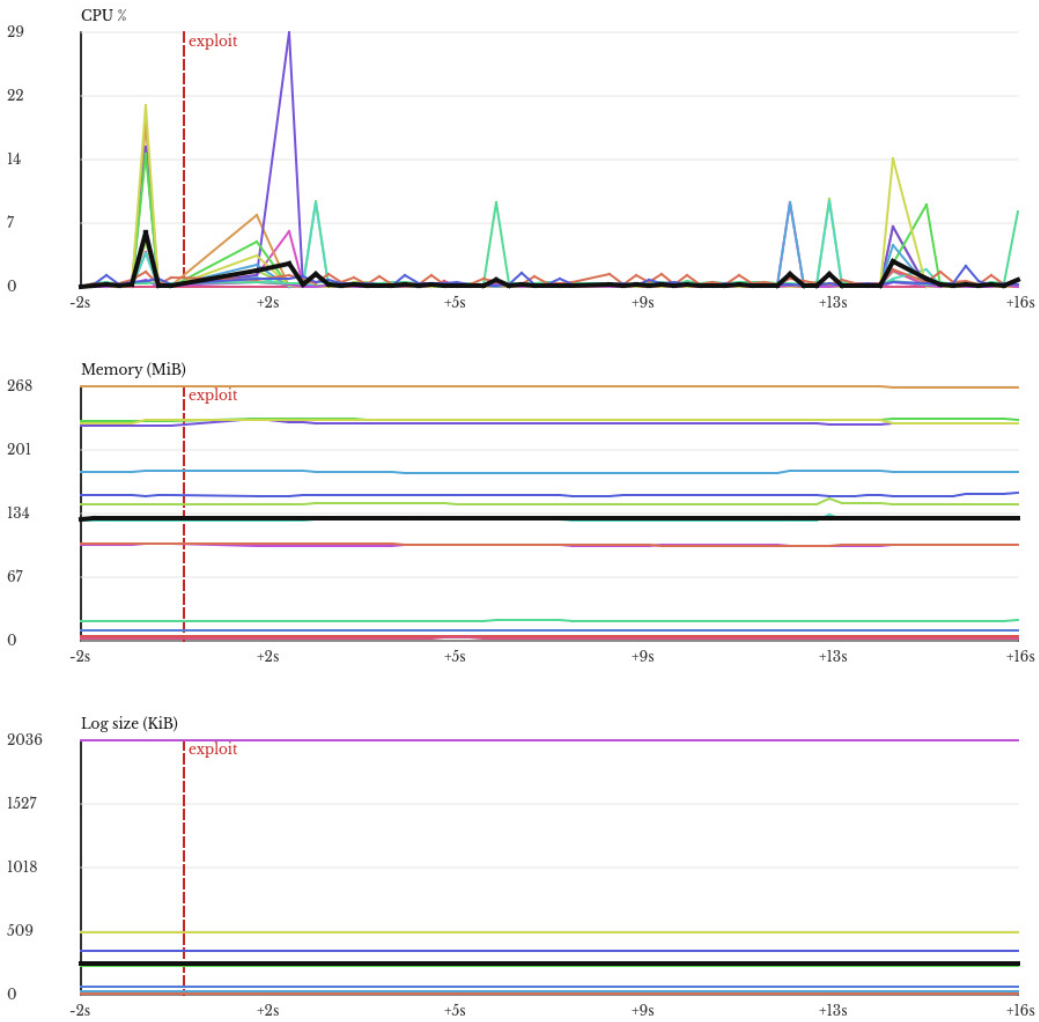
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.0–0.1%)**	8 samples before the exploit
CPU, container average, after	**0.6% (95% CI 0.0–1.1%)**	55 samples after the exploit
CPU change after the exploit	**+0.5%, p = 0.069**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**6.6%, 140.1 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**128.7 MiB (95% CI 128.5–128.8 MiB)**	8 samples before the exploit
Memory, container average, after	**128.3 MiB (95% CI 126.8–129.8 MiB)**	55 samples after the exploit
Memory change after the exploit	**-0.3 MiB, p = 0.377**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**129.6 MiB, 42.6 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Scanner tool: Nikto
GENERIC-TOOL-002 · severity MEDIUM
`GET /`
Generic attack · WordPress core · Backend

Score: 50/100 Route: PRODUCTION HTTP/2 200
Analyzer score verdict: elevated but below threshold
Signals: Suspicious headers (+50)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

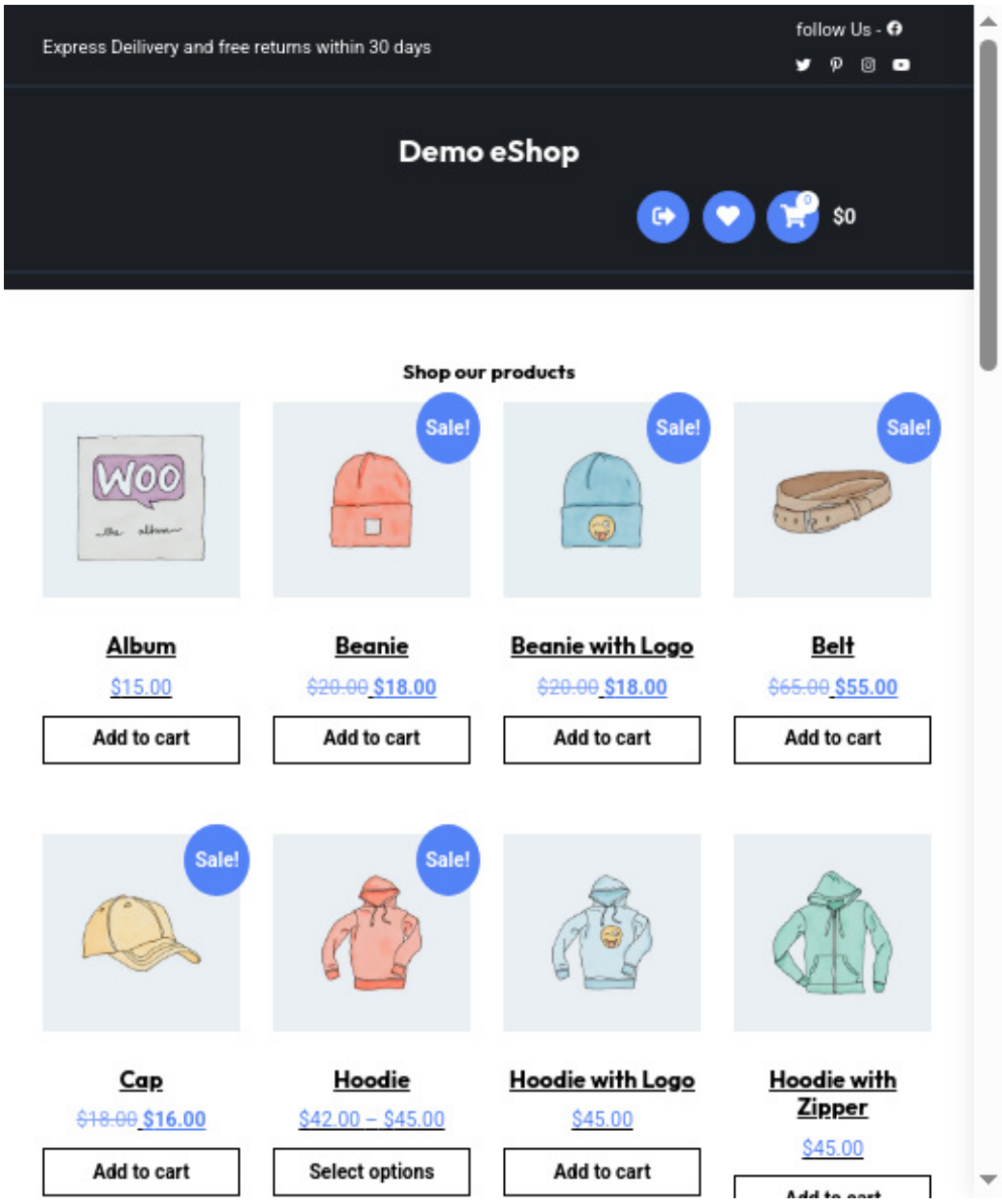


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

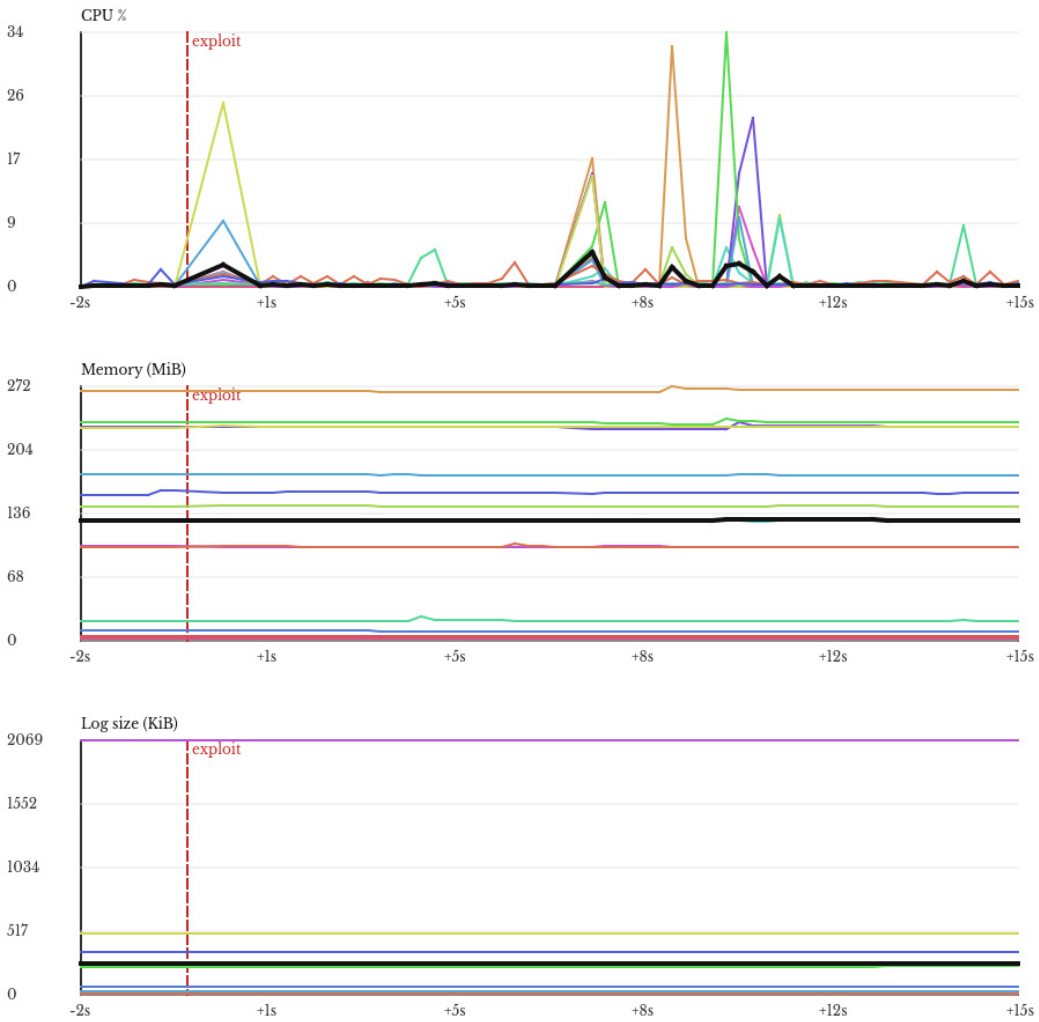
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.8% (95% CI 0.0–2.6%)**	8 samples before the exploit
CPU, container average, after	**0.3% (95% CI 0.1–0.5%)**	55 samples after the exploit
CPU change after the exploit	**-0.5%, p = 0.525**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**2.9%, 1.0 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**128.1 MiB (95% CI 126.7–129.4 MiB)**	8 samples before the exploit
Memory, container average, after	**128.4 MiB (95% CI 128.2–128.6 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.4 MiB, p = 0.165**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**129.1 MiB, 4.6 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Scanner tool: Metasploit
GENERIC-TOOL-003 · severity MEDIUM
`GET /`
Generic attack · WordPress core · Backend

Score: 0/100 Route: PRODUCTION HTTP/2 200
Analyzer score verdict: clean



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

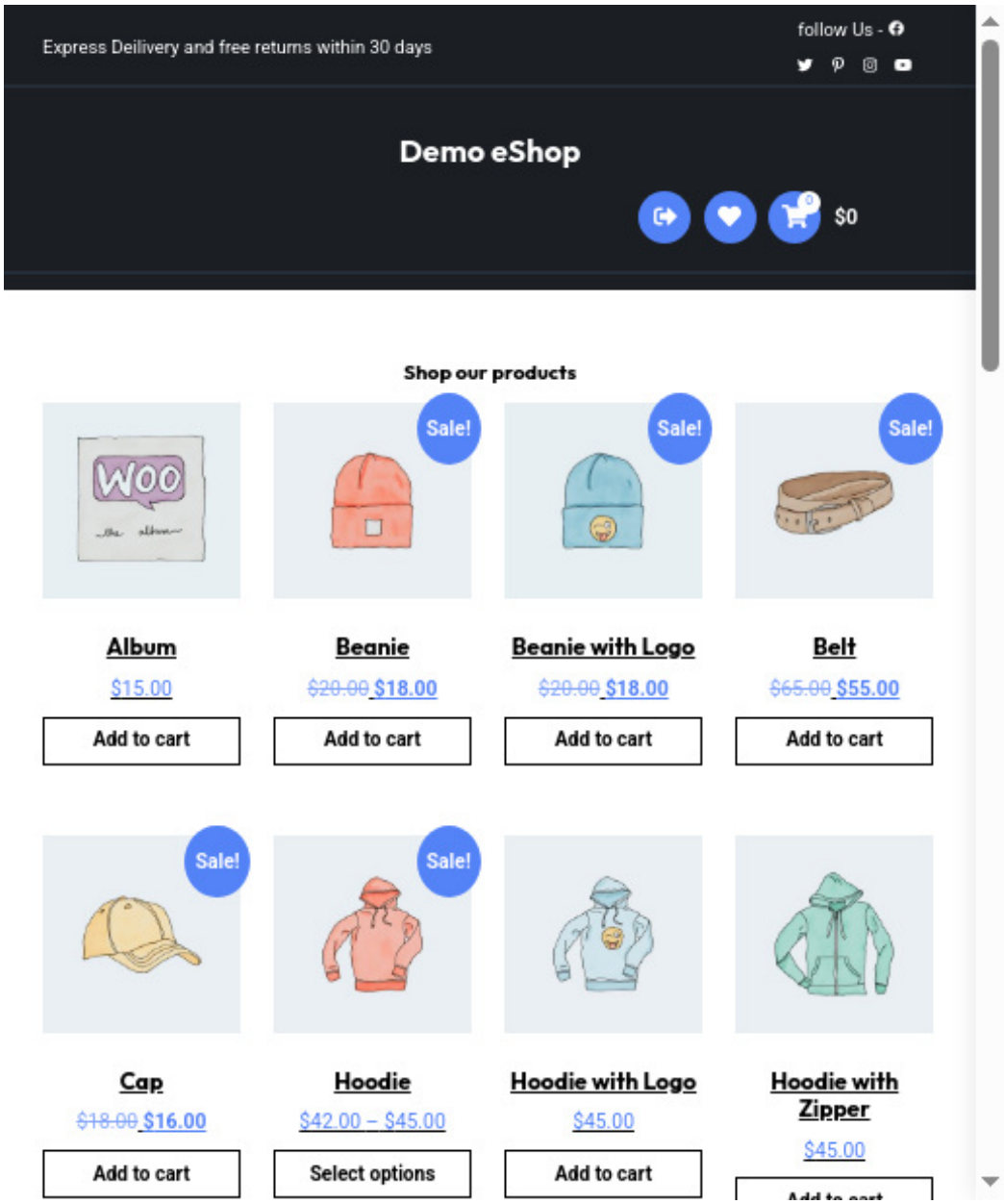


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

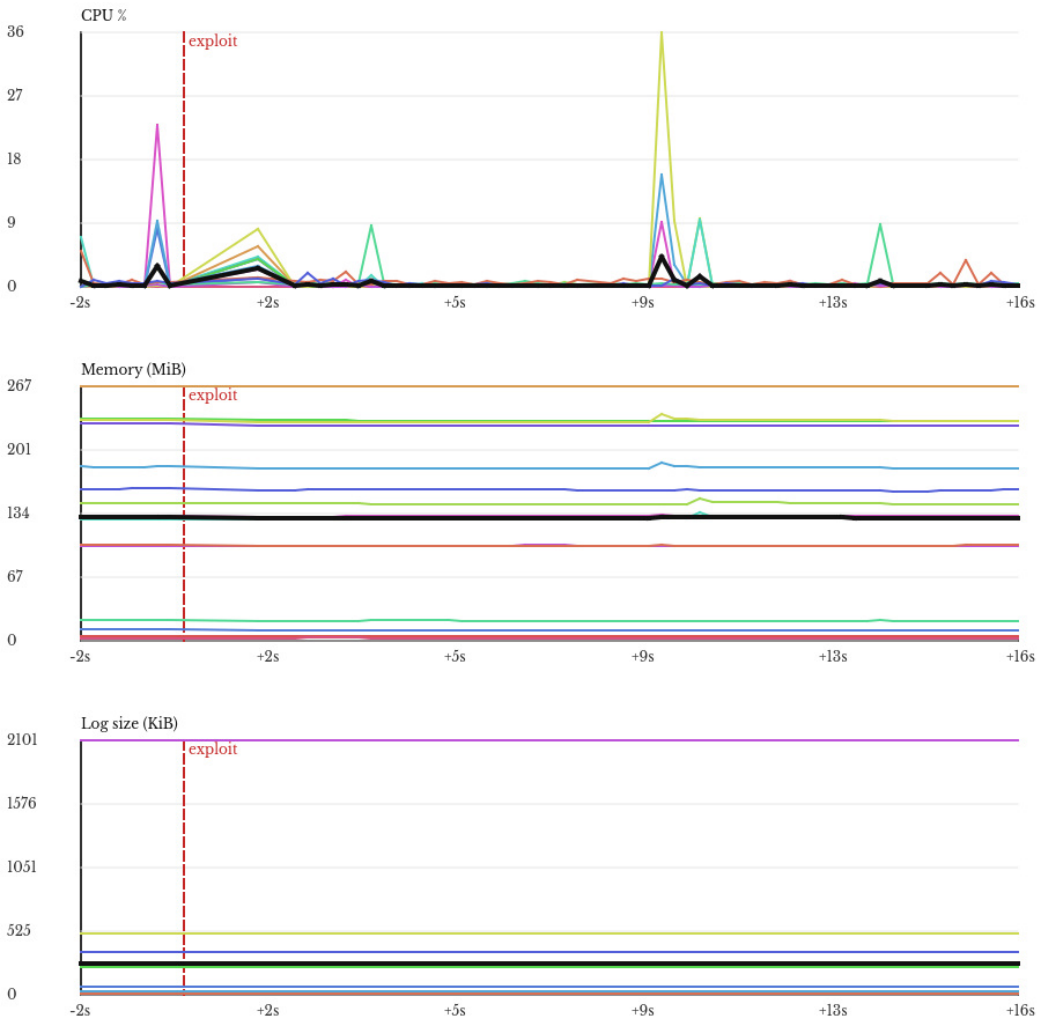
Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.1% (95% CI 0.0–0.2%)**	8 samples before the exploit
CPU, container average, after	**0.5% (95% CI 0.2–0.8%)**	55 samples after the exploit
CPU change after the exploit	**+0.4%, p = 0.020**	mean after vs mean before; unlikely to be chance (significant at 5%)
CPU peak after the exploit	**4.6%, 72.6 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**128.1 MiB (95% CI 127.8–128.5 MiB)**	8 samples before the exploit
Memory, container average, after	**128.3 MiB (95% CI 128.1–128.5 MiB)**	55 samples after the exploit
Memory change after the exploit	**+0.1 MiB, p = 0.253**	mean after vs mean before; could plausibly be chance (not significant at 5%)
Memory peak after the exploit	**128.9 MiB, 5.1 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Scanner tool: Gobuster
GENERIC-TOOL-004 · severity MEDIUM
`GET /`
Generic attack · WordPress core · Backend

Score: 50/100 Route: PRODUCTION HTTP/2 200
Analyzer score verdict: elevated but below threshold
Signals: Suspicious headers (+50)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.
CPU and memory were read from the containers' cgroup files every 0.25 s.

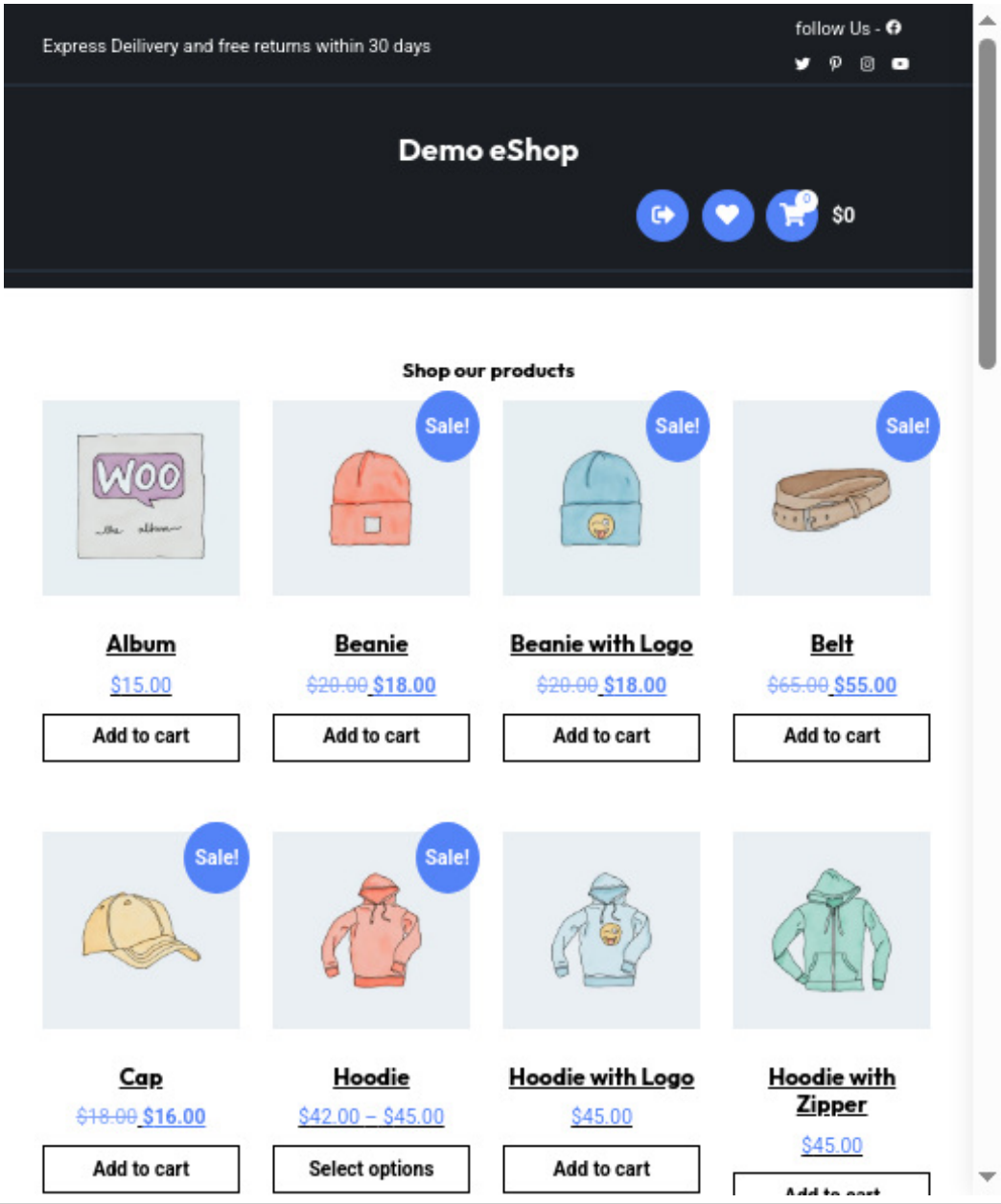


Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.5% (95% CI 0.0–1.4%)**	8 samples before the exploit
CPU, container average, after	**0.3% (95% CI 0.1–0.5%)**	55 samples after the exploit
CPU change after the exploit	**-0.2%, p = 0.507**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**4.2%, 3.8 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**129.2 MiB (95% CI 128.2–130.1 MiB)**	8 samples before the exploit
Memory, container average, after	**128.7 MiB (95% CI 128.6–128.9 MiB)**	55 samples after the exploit
Memory change after the exploit	**-0.5 MiB, p = 0.030**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**129.6 MiB, 3.5 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

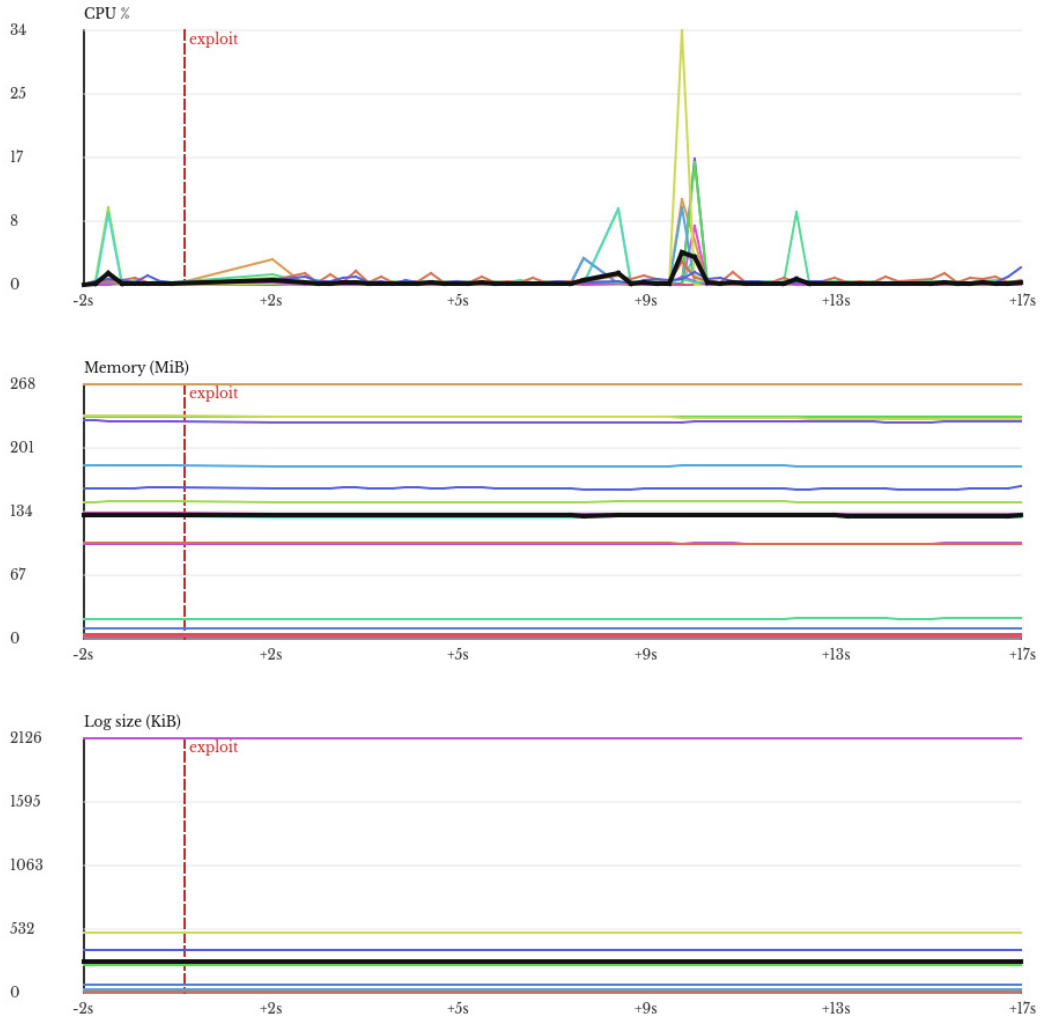
Scanner tool: WPScan
GENERIC-TOOL-005 · severity MEDIUM
`GET /`
Generic attack · Plugin / theme · Backend

Score: 70/100 Route: HONEYPOT reason: rapid_automation_detected HTTP/2 200
Diverted to honeypot by rule "rapid_automation_detected" – a CVE/exploit match (or IP reputation) diverts regardless of the numeric score.
Analyzer score verdict: elevated but below threshold
Signals: Suspicious headers (+50), Automation detected (+20)



Resource usage – ~2 s before to ~15 s after the exploit
Per-container CPU %, memory (MiB) and log-file size (KiB), sampled from ~2 seconds before this exploit fired to ~15 seconds after. Each thin coloured line is one container (see the legend below); the bold black line is the average across all containers. Time runs left→right and the dashed red line at **0 s** marks the moment the exploit was sent, so the readings to its left are the pre-exploit baseline and the spike to its right is the footprint the exploit and the honeypot's handling of it left on the stack.

CPU and memory were read from the containers' cgroup files every 0.25 s.



Legend: backup_service honeypot_content_sync honeypot_database_1 honeypot_database_2 honeypot_database_3 honeypot_eshop_1 honeypot_eshop_2 honeypot_eshop_3 log_capper production_database production_eshop reverse_proxy session_store suricata_ids vector_outbound **average**

Probability metrics: did the exploit move resource usage?		
CPU, container average, before	**0.3% (95% CI 0.0–0.7%)**	8 samples before the exploit
CPU, container average, after	**0.3% (95% CI 0.0–0.6%)**	55 samples after the exploit
CPU change after the exploit	**+0.0%, p = 0.857**	mean after vs mean before; could plausibly be chance (not significant at 5%)
CPU peak after the exploit	**4.3%, 8.5 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation
Memory, container average, before	**129.4 MiB (95% CI 129.4–129.5 MiB)**	8 samples before the exploit
Memory, container average, after	**129.1 MiB (95% CI 128.8–129.3 MiB)**	55 samples after the exploit
Memory change after the exploit	**-0.4 MiB, p = 0.018**	mean after vs mean before; unlikely to be chance (significant at 5%)
Memory peak after the exploit	**129.3 MiB, -8.2 SD above the baseline**	a short spike can be real even when the means don't differ significantly; more than ~3 SD is rarely normal fluctuation

Probability metrics: all individual exploits		
Requests that got a response	**32/32 = 100% (95% CI 89–100%)**	share of exploits the proxy answered (the rest failed to send)
Diverted to the honeypot (detection rate)	**20/32 = 62% (95% CI 45–77%)**	estimated chance that an exploit like these is caught; the CI shows how sure that is with this many exploits
Mean threat score	**63.2 (95% CI 50.7–75.8)**	typical isolated score out of 100; the CI is where the average of many such exploits would likely land
Spread of scores	**SD 34.7, median 70, range 0–100**	how much the scores differ from one exploit to the next
Score in the high band (≥80)	**13/32 = 41% (95% CI 26–58%)**	estimated chance an exploit like these scores in the red band on the Exploits page

2. Threat increase – repeated exploits

Threat state is reset once, then these two semi-randomly chosen exploits are run alternately for three rounds with **no reset between them**. The score ratchets up as the host IP's reputation accumulates:

- CVE-2023-28121 (WooCommerce Payments admin bypass)
- GENERIC-SQLI-001 (SQL injection in a search form)

Score progression: 100 → 95 → 100 → 100 → 100 → 100

Probability metrics: threat increase		
Score increase per request (trend)	**+0.4 points (95% CI -1.0 to +1.8), p = 0.441**	straight-line fit of score vs request number; the rise could plausibly be chance (not significant at 5%)
Fit quality (R^2)	**0.15**	share of the score variation the straight line explains (1 = perfect)
Requests diverted to the honeypot	**6/6 = 100% (95% CI 61–100%)**	share of the repeated requests that landed on the honeypot

Round 1: WooCommerce Payments admin bypass
CVE-2023-28121 · severity HIGH
`GET /wp-json/wp/v2/users`
CVE exploit · Plugin / theme · woocommerce-payments · Backend

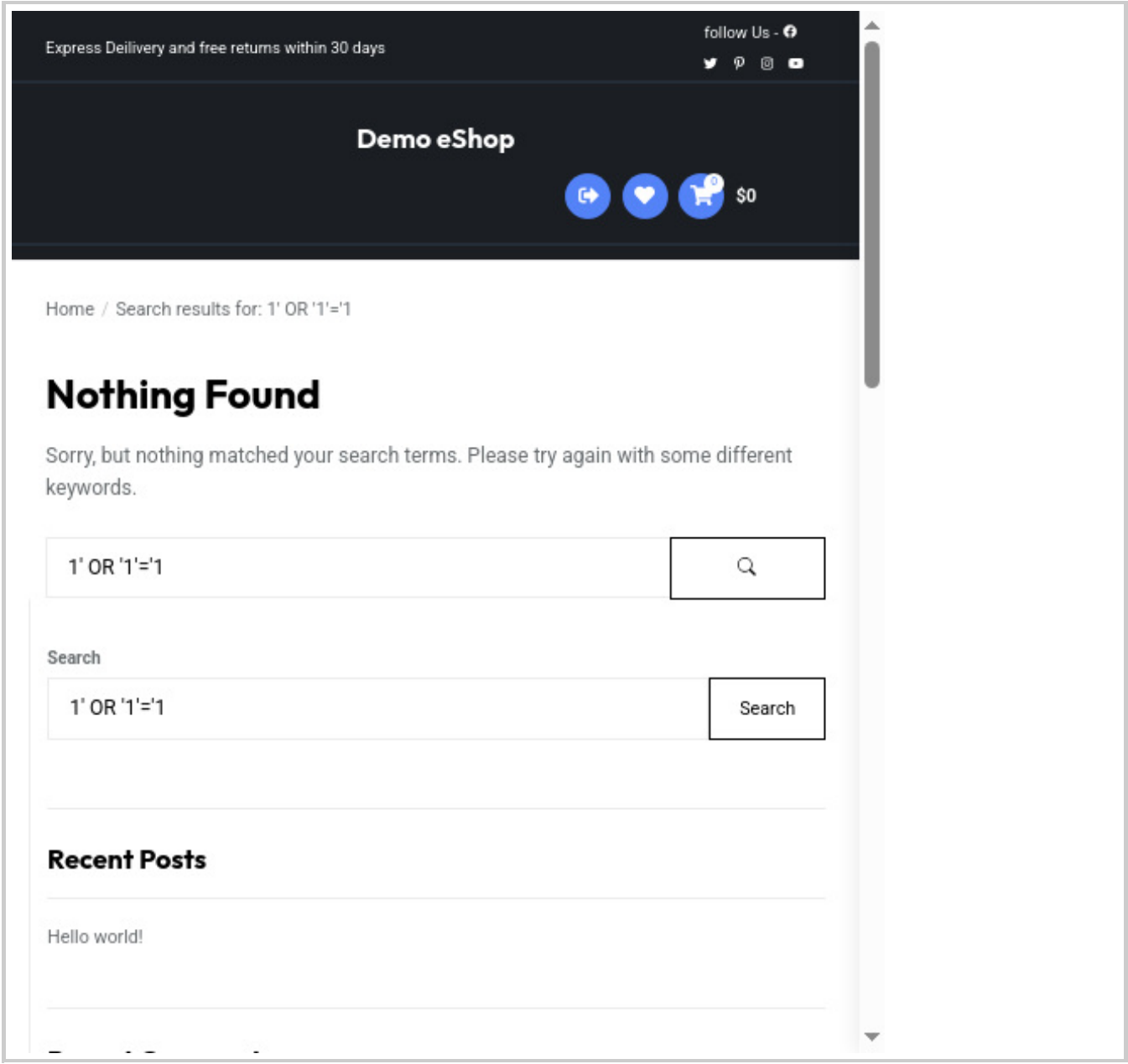
Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 200
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+25), Suspicious headers (+50), CVE PATTERNS DETECTED (+40), Automation detected (+5)

Pretty print

{"code":"rest_forbidden","message":"Sorry, you are not allowed to do that.","data":{"status":401}}

Round 1: SQL injection in a search form
GENERIC-SQLI-001 · severity HIGH
`GET /?s=1'%20OR%20'1'='1`
Generic attack · WordPress core · Backend

Score: 95/100 Route: HONEYPOT reason: high_threat_score HTTP/2 200
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+25), URI patterns matched (+65), Automation detected (+5)



Round 2: WooCommerce Payments admin bypass
CVE-2023-28121 · severity HIGH
`GET /wp-json/wp/v2/users`
CVE exploit · Plugin / theme · woocommerce-payments · Backend

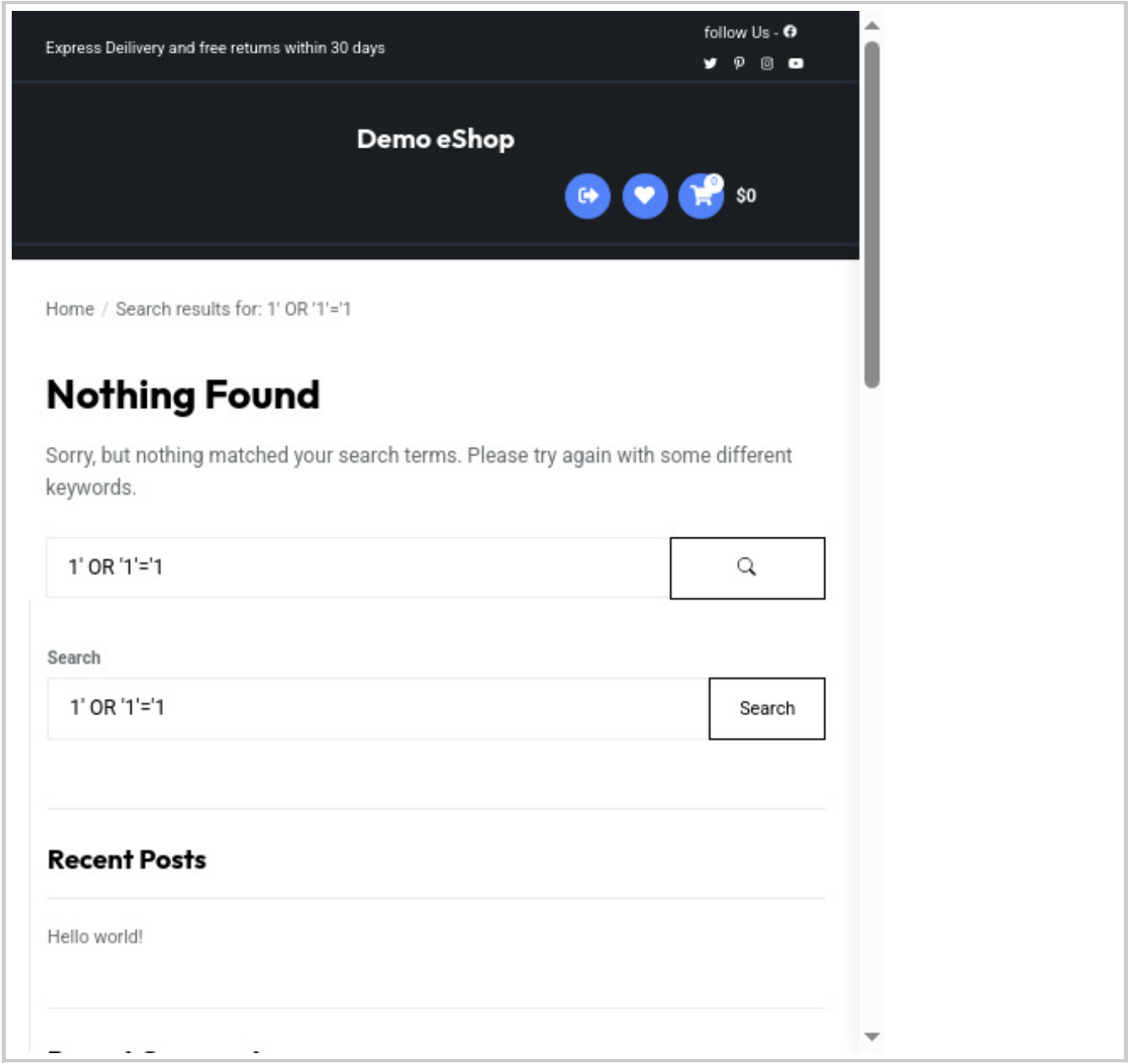
Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 200
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+25), Suspicious headers (+50), CVE PATTERNS DETECTED (+40), Automation detected (+5)

Pretty print


```
[{"id":1,"name":"admin","url":"http:\/\/localhost","description":"","link":"https:\/\/localhost\/a
uthor\/admin\/","slug":"admin","avatar_urls":
{"24":"https:\/\/secure.gravatar.com\/avatar\/258d8dc916db8cea2cafb6c3cd0cb0246efe061421dbd83ec3a3
50428cabda4f?
s=24&d=mm&r=g","48":"https:\/\/secure.gravatar.com\/avatar\/258d8dc916db8cea2cafb6c3cd0cb0246efe06
1421dbd83ec3a350428cabda4f?
s=48&d=mm&r=g","96":"https:\/\/secure.gravatar.com\/avatar\/258d8dc916db8cea2cafb6c3cd0cb0246efe06
1421dbd83ec3a350428cabda4f?s=96&d=mm&r=g"},"meta":[],"is_super_admin":true,"woocommerce_meta":
{"variable_product_tour_shown":"","activity_panel_inbox_last_read":"","activity_panel_reviews_last
_read":"","categories_report_columns":"","coupons_report_columns":"","customers_report_columns":""
,"orders_report_columns":"","products_report_columns":"","revenue_report_columns":"","taxes_report
_columns":"","variations_report_columns":"","dashboard_sections":"","dashboard_chart_type":"","das
hboard_chart_interval":"","dashboard_leaderboard_rows":"","order_attribution_install_banner_dismis
sed":"","homepage_layout":"","homepage_stats":"","task_list_tracked_started_tasks":"","android_app
_banner_dismissed":"","launch_your_store_tour_hidden":"","coming_soon_banner_dismissed":""},"eleme
ntor_introduction":"","_links":{"self":[{"href":"https:\/\/localhost\/wp-
json\/wp\/v2\/users\/1","targetHints":{"allow":["GET"]}}],"collection":
[{"href":"https:\/\/localhost\/wp-json\/wp\/v2\/users"}]}}]
```


Round 2: SQL injection in a search form
GENERIC-SQLI-001 · severity HIGH
`GET /?s=1'%20OR%20'1'='1`
Generic attack · WordPress core · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 200
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+65), Automation detected (+5)



Round 3: WooCommerce Payments admin bypass
CVE-2023-28121 · severity HIGH
`GET /wp-json/wp/v2/users`
CVE exploit · Plugin / theme · woocommerce-payments · Backend

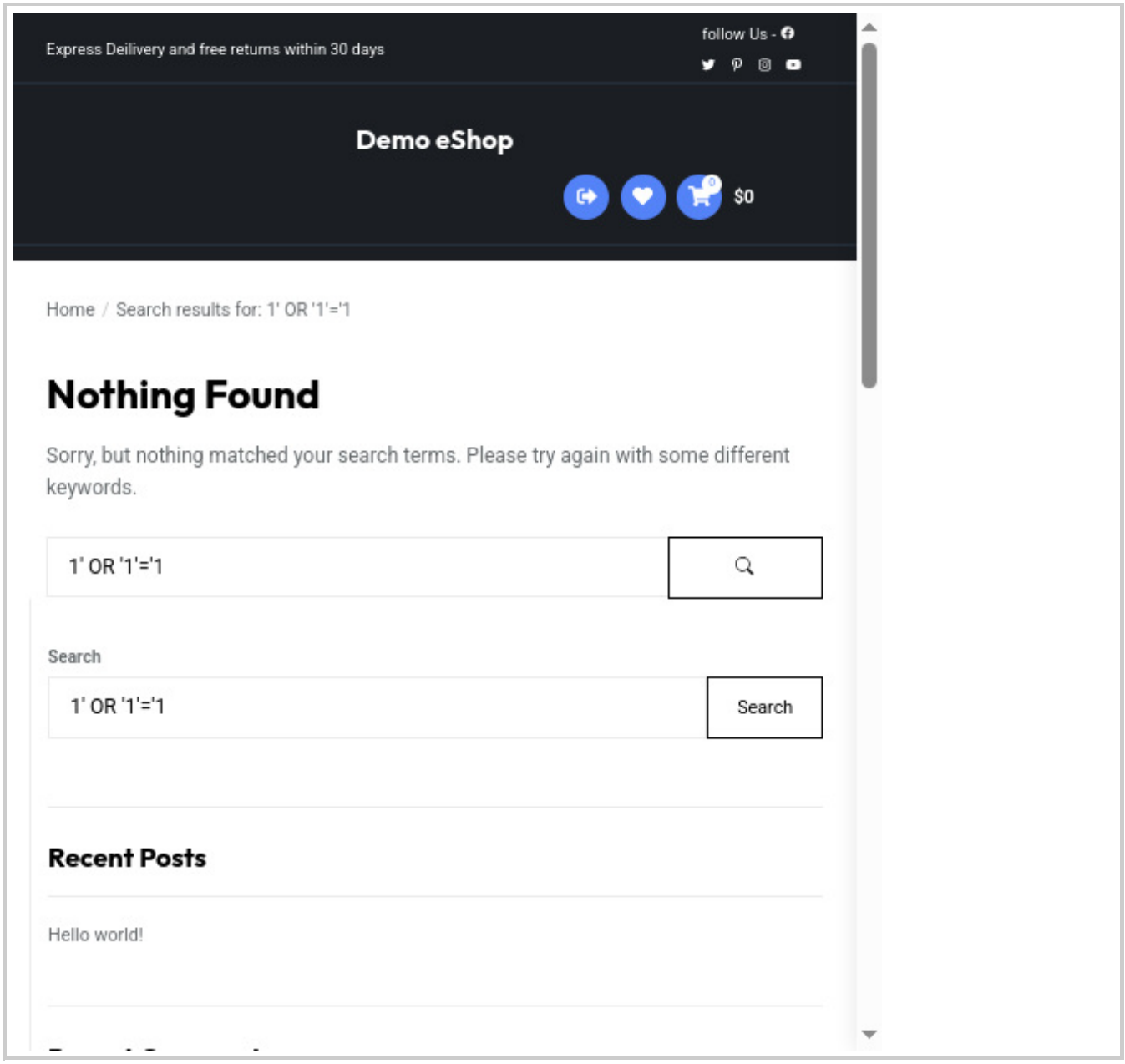
Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 200
Analyzer score verdict: SUSPICIOUS
Signals: URI patterns matched (+25), Suspicious headers (+50), CVE PATTERNS DETECTED (+40), Automation detected (+5)

Pretty print


```
[{"id":1,"name":"admin","url":"http:\/\/localhost","description":"","link":"https:\/\/localhost\/a
uthor\/admin\/","slug":"admin","avatar_urls":
{"24":"https:\/\/secure.gravatar.com\/avatar\/258d8dc916db8cea2cafb6c3cd0cb0246efe061421dbd83ec3a3
50428cabda4f?
s=24&d=mm&r=g","48":"https:\/\/secure.gravatar.com\/avatar\/258d8dc916db8cea2cafb6c3cd0cb0246efe06
1421dbd83ec3a350428cabda4f?
s=48&d=mm&r=g","96":"https:\/\/secure.gravatar.com\/avatar\/258d8dc916db8cea2cafb6c3cd0cb0246efe06
1421dbd83ec3a350428cabda4f?s=96&d=mm&r=g"},"meta":[],"is_super_admin":true,"woocommerce_meta":
{"variable_product_tour_shown":"","activity_panel_inbox_last_read":"","activity_panel_reviews_last
_read":"","categories_report_columns":"","coupons_report_columns":"","customers_report_columns":""
,"orders_report_columns":"","products_report_columns":"","revenue_report_columns":"","taxes_report
_columns":"","variations_report_columns":"","dashboard_sections":"","dashboard_chart_type":"","das
hboard_chart_interval":"","dashboard_leaderboard_rows":"","order_attribution_install_banner_dismis
sed":"","homepage_layout":"","homepage_stats":"","task_list_tracked_started_tasks":"","android_app
_banner_dismissed":"","launch_your_store_tour_hidden":"","coming_soon_banner_dismissed":""},"eleme
ntor_introduction":"","_links":{"self":[{"href":"https:\/\/localhost\/wp-
json\/wp\/v2\/users\/1","targetHints":{"allow":["GET"]}}],"collection":
[{"href":"https:\/\/localhost\/wp-json\/wp\/v2\/users"}]}}]
```

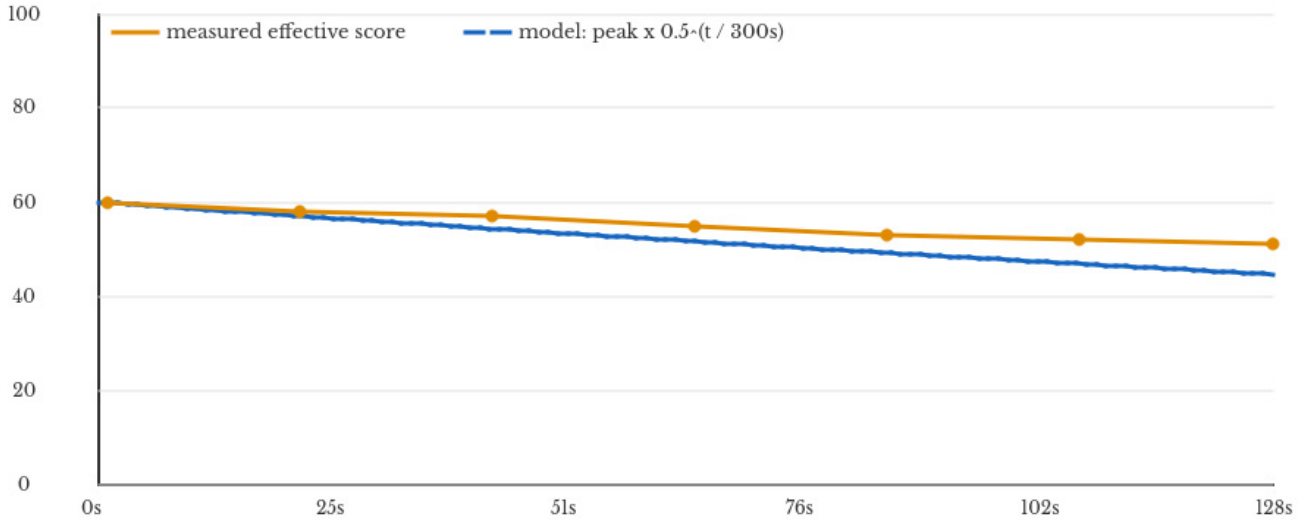

Round 3: SQL injection in a search form
GENERIC-SQLI-001 · severity HIGH
`GET /?s=1'%20OR%20'1'='1`
Generic attack · WordPress core · Backend

Score: 100/100 Route: HONEYPOT reason: high_threat_score HTTP/2 200
Analyzer score verdict: elevated but below threshold
Signals: URI patterns matched (+65), Automation detected (+5)



3. Threat decay

This scenario is **measured on a clean host IP, before the exploit scenarios above are run** (it is presented last only for readability). That matters: the effective score a request sees folds in the IP's reputation, which the IDS continuously re-imports from Suricata's alert log – so once the IP has been hammered by the other scenarios, every "benign" probe keeps picking that reputation back up and the score never falls, no matter how thoroughly state is flushed. The scenario **un-poisons the host IP**, seeds one moderate, decayable peak and then samples the effective score fading over the window while sending only benign requests. The peak was seeded with a single moderate exploit (**CVE-2023-2986 (Abandoned Cart Lite checkout token forgery)**). Measured samples are plotted against the analytic decay curve the router itself applies.



Elapsed	Effective score
1s	60
22s	58
43s	57
65s	55
86s	53
107s	52
128s	51

Probability metrics: threat decay		
Measured half-life (exponential fit)	**529s (95% CI 475–596s)**	configured: 300s – outside the CI, so the measured decay differs from the configured one
Fit quality (R^2)	**0.99**	how closely the samples follow an exponential curve (1 = perfect)
Typical gap to the model curve (RMSE)	**3.8 points**	average distance between each measured score and the dashed curve

4. Test categorization

The individual tests regrouped. **Diverted** means the attacker was routed to the honeypot on some request of the test; **On request 1** counts tests caught immediately. Tests without their own request series are sent up to three times, so a later detection shows whether repetition tips the score. Peak CPU and memory are the container average after the first request.

Generic attack vs CVE exploit	Tests	Diverted to honeypot (any request)	On request 1	Later	Mean score	Peak CPU	Peak MiB
CVE exploit	14	13/14 = 93% (95% CI 69–99%)	13/14	–	82	5.7%	127
Generic attack	18	15/18 = 83% (95% CI 61–94%)	7/18	request 2, 2, 2, 2, 2, 2, 3, 3	49	4.6%	129

WordPress core	Tests	Diverted to	On request 1	Later	Mean score	Peak CPU	Peak MiB

vs plugin / theme		honeypot (any request)					
WordPress core	16	14/16 = 88% (95% CI 64–97%)	6/16	request 2, 2, 2, 2, 2, 2, 3, 3	50	4.7%	129
Plugin / theme	16	14/16 = 88% (95% CI 64–97%)	14/16	–	76	5.4%	127

Frontend vs backend	Tests	Diverted to honeypot (any request)	On request 1	Later	Mean score	Peak CPU	Peak MiB
Frontend	3	2/3 = 67% (95% CI 21–94%)	2/3	–	55	5.5%	130
Backend	29	26/29 = 90% (95% CI 74–96%)	18/29	request 2, 2, 2, 2, 2, 2, 3, 3	64	5.0%	128

On which request is each test detected?

ID	Test	Requests	Route per request	Diverted on
CVE-2023-28121	WooCommerce Payments admin bypass	1	honeypot	**request 1**
CVE-2023-2986	Abandoned Cart Lite checkout token forgery	1	honeypot	**request 1**
CVE-2025-4403	Drag & Drop file upload MIME-type bypass	1	honeypot	**request 1**
CVE-2025-2266	CWMP unauthenticated options update	1	honeypot	**request 1**
CVE-2025-47577	Gift Voucher file upload RCE	1	honeypot	**request 1**
CVE-2024-2387	Advanced Form Integration SQL injection	1	honeypot	**request 1**
CVE-2025-10142	PagSeguro Connect path traversal	1	honeypot	**request 1**
CVE-2024-50508	WP File Upload directory traversal	1	honeypot	**request 1**
GENERIC-WP-001	wp-config.php disclosure	3	production → production → honeypot	**request 3**
GENERIC-WP-002	Installer file access	2	production → honeypot	**request 2**
GENERIC-WP-003	User enumeration via ?author=	1	honeypot	**request 1**
GENERIC-WP-004	XML-RPC probe	1	honeypot	**request 1**
CVE-2024-25600	Bricks Builder unauthenticated RCE	1	honeypot	**request 1**
CVE-2024-27956	WP Automatic SQL injection	1	honeypot	**request 1**
CVE-2024-1071	Ultimate Member SQL injection	1	honeypot	**request 1**
CVE-2022-0739	BookingPress SQL injection	1	honeypot	**request 1**
CVE-2024-2879	LayerSlider SQL injection	1	honeypot	**request 1**
CVE-2017-1001000	REST API content injection (defacement)	3	production → production → production	**not within 3 requests**
GENERIC-XSS-001	Script tag in a search form	1	honeypot	**request 1**
GENERIC-XSS-002	Event-handler XSS	1	honeypot	**request 1**
GENERIC-SQLI-001	SQL injection in a search	1	honeypot	**request 1**

	form			
GENERIC-SQLI-002	Time-based blind SQL injection	2	production → honeypot	**request 2**
GENERIC-SQLI-003	SQL injection in the login form (POST body)	3	production → production → honeypot	**request 3**
GENERIC-ENUM-001	Wordlist scan: sensitive paths	1	honeypot	**request 1**
GENERIC-ENUM-002	Wordlist scan: plugin and theme discovery	6	production → production → production → production → ? → production	**not within 6 requests**
GENERIC-IDOR-001	Insecure direct object reference: user IDs	2	production → honeypot	**request 2**
GENERIC-IDOR-002	Insecure direct object reference: order IDs	4	production → production → production → production	**not within 4 requests**
GENERIC-TOOL-001	Scanner tool: sqlmap	2	production → honeypot	**request 2**
GENERIC-TOOL-002	Scanner tool: Nikto	2	production → honeypot	**request 2**
GENERIC-TOOL-003	Scanner tool: Metasploit	3	production → production → production	**not within 3 requests**
GENERIC-TOOL-004	Scanner tool: Gobuster	2	production → honeypot	**request 2**
GENERIC-TOOL-005	Scanner tool: WPScan	1	honeypot	**request 1**

Resource usage and honeypot pool reuse

Peak CPU and memory per category are in the tables above. The honeypot pool registry at the end of the run shows how the pool reacted to the host's resources:

Growing normally – the host has room, so every new attacker gets a pool of their own and a spare is built after each assignment.
Ready pools: 0 (none). Free (unowned) spares: 0. Attackers per pool: none. Round-robin reuse assignments so far: 0.